FUNDAMENTALS OF DATA SCIENCE QUESTION BANK SOLVED

Unit-1

2 MARKS QUESTION

1. Define is Data Mining.

Data mining in data science refers to the process of discovering patterns, correlations, and useful information from large datasets using statistical, mathematical, and computational techniques. It involves extracting hidden insights from data to make informed decisions and predictions.

2. What is knowledge discovery in database?

Knowledge Discovery in Databases (KDD) is the overall process of discovering useful knowledge from data. This process involves several steps, including data cleaning, data integration, data selection, data transformation, data mining, pattern evaluation, and knowledge presentation.

3. List any 4 steps of Knowledge discovery (KDD).

1. Data Cleaning: Removing noise and inconsistent data to improve the quality of the dataset.
2. Data Integration: Combining data from multiple sources to create a unified dataset.
3. Data Selection: Choosing relevant data to analyse from the larger dataset.
4. Data Transformation: Converting data into a suitable format or structure for mining.
5. Data Mining: Applying algorithms to extract patterns or models from the data. 6. Pattern Evaluation: Identifying the truly interesting patterns representing knowledge based on some criteria.
7. Knowledge Presentation: Presenting the mined knowledge in an understandable form, often using visualization techniques.

4. What is Loosely coupled DBMS and tightly coupled DBMS?

In data science, a loosely coupled DBMS (Database Management System) is characterized by a high degree of independence between its various components. This means that the database system and its applications can interact without being tightly integrated, allowing for greater flexibility and easier updates or modifications to individual components without affecting the entire system.

Tightly coupled DBMS has components that are highly interdependent, leading to more cohesive and efficient performance since all parts are designed to work seamlessly together. This close integration often results in better optimization and speed but can make the system more rigid and harder to modify or scale independently.

5.What is prediction and description in Data Mining?

Prediction: In data mining, prediction involves using data to make informed guesses about unknown future events. It uses historical data to predict outcomes. For example, predicting whether a customer will purchase a product based on their past behaviour.

- **Description**: Description in data mining focuses on finding patterns and relationships within the data to provide a meaningful summary. It aims to describe the main features of the data, such as identifying trends, clusters, or outliers. For instance, describing customer segments based on their purchasing behaviour.

6. List any four Discovery Driven Tasks

1.Clustering
2.Classification
3. Association Rule Mining

4.Anomaly Detection

7.Define Support and Confidence

- **Support**: In association rule mining, support measures the frequency of cooccurrence of items in a dataset. It indicates how often an itemset appears in the dataset.
- **Confidence**: Confidence measures the reliability of the rule. It shows how often a rule is found to be true. It is the probability of finding the consequent item in a transaction given that the transaction also contains the antecedent item.

8.Write the objectives of clustering.

• Grouping Similar Data Points
• Discovering Inherent Patterns
• Data Summarization and Compression.
• Data Exploration and Understanding
• Pattern Recognition and Classification • Anomaly Detection and Outlier Identification
• Decision Making and Knowledge Discovery:
• Improving Data Preprocessing and Feature Engineering

9) Define Rough Set.

The rough sets theory has recently become a popular theory in the field of data mining. the theory, introduce by Pawlak in the early 1980s, provides a formal framework for the automated transformation of data into knowledge. the rough set theory though mathematically simple, has displayed its fruitfulness in a variety of data mining areas. a rough set is a pair of approximations-lower approximation and upper approximation using this approximation, the rough set theory develops tools to discover rules from the given databases.

10) What is sequence mining and spatial data mining?

sequence mining refers to the process of discovering interesting sequential patterns or sub sequences from data. It involves identifying patterns in sequences of events or items over time, where the order of occurrences matters. Sequence mining is widely used in various domains such as web usage mining, bioinformatics, finance, and customer behaviour analysis.
Spatial mining refers to the process of discovering patterns, trends, and relationships in spatial datasets. Spatial data includes information about the geographic location and attributes of objects, such as points of interest, regions, or routes. Spatial mining techniques are used to extract valuable insights from spatial data, enabling better decision-making in various domains.

11.**write the subtasks of web mining?**

•Resource finding retrieving document intended for the web. •Information selection and preprocessing: automatically selecting and preprocessing specific information from resources retrieved from the web.
•Generalization: to automatically discover general pattern at individual web sites as well as across multiple sites.
•Analysis: validation and/or interpretation of the mined patterns.

12.Expand KDT and IE?

KDT - knowledge Discovery in Text.
IE - Information Extraction.

13.What is web mining and Text mining?

Web Mining is the use of the data mining technique automatically discover and extract information document/services. Discovering useful information from the world wide web and its usage pattern.

Text mining text data mining can be described as the process of extracting essential data from standard language text. all the data that we generate via text message, documents, email, files are written in common language text. Text mining is primarily used to draw useful pattern from such data.

14.List any two issues and challenges in data mining?

1. **Data Quality and Preprocessing** 2. **Scalability and Efficiency**

15.List the two application areas of data mining?

1. **Healthcare:** Data mining is used to analyze patient data, improve diagnostics, personalize treatment plans, and predict disease outbreaks. It helps in identifying patterns and correlations in medical records, enhancing patient care and operational efficiency.
2. **Finance:** In the financial sector, data mining assists in fraud detection, risk management, customer segmentation, and credit scoring. It enables institutions to analyze large volumes of transaction data to uncover trends, anomalies, and predictive insights that inform decision-making and strategy development.

16.Explain the uses if Data Visualization?

1. **Simplifying Complex Data:** By transforming large datasets into visual formats like charts, graphs, and maps, data visualization makes complex data more accessible and understandable.
2. **Identifying Patterns and Trends**: Visualization helps in quickly spotting patterns, trends, and outliers that might not be apparent in raw data. This facilitates datadriven decision-making and insight discovery.
3. **Communicating Insights:** Effective visualizations communicate findings and insights clearly and compellingly, making it easier for stakeholders to grasp critical information and take informed actions.
4. **Facilitating Comparisons:** Visualization allows for easy comparison of different data sets, enabling users to analyze relationships, correlations, and differences across variables.
5. **Enhancing Engagement**: Interactive visualizations can engage users more effectively than static data presentations, encouraging exploration and deeper analysis.

17) Define Machine Learning?

Machine learning is the automation of a learning process and learning is equivalent to the construction of rules based on observations. This is a broad field which includes not only learning from examples, but also reinforcement learning, learning with a teacher, etc.

18)Define Supervised and Unsupervised Learning?

Supervised learning means learning from examples, where a training set is given which acts as examples for the classes. The system finds a description of each class. Once the description (and hence a classification rule) has been formulated, it is used to predict the class of previously unseen objects. This is similar to discriminate analysis which occurs in statistics.

Unsupervised learning is learning from observation and discovery. In this mode of learning, there is no training set or prior knowledge of the classes. The system analyses the given set of data to observe similarities emerging out of the subsets of the data. The outcome is a set of class descriptions, one for each class, discovered in the environment. This is similar to cluster analysis in statistics.

19)Explain the two fundamental goals of Data Mining?

Prediction: Prediction makes use of existing variables in the database in order to predict unknown or future values of interest

Description: description focuses on finding patterns describing the data and the subsequent presentation for user interpretation.

20.Define Discovery Model? Give an example?

A discovery model in data mining refers to a process or methodology used to identify patterns, correlations, and insights within a dataset without predefined notions of what those patterns might be. This model aims to uncover previously unknown and potentially useful information from data.
Example: Market Basket Analysis is a common example of a discovery model. It is used in retail to discover product purchase patterns. By analyzing transaction data, this model identifies sets of products frequently bought together, such as "bread and butter," helping retailers optimize inventory, layout, and promotional strategies.

21.What is Frequent Episodes? List its types?

Frequent episodes refer to patterns or sequences of events that occur frequently within a given sequence of data. These patterns are particularly useful in time series analysis, event prediction, and sequence mining. Identifying frequent episodes can help in understanding the underlying structure and relationships within the data. Types of frequent episodes include:
1. **Serial Episodes**
2. **Parallel Episodes**

22.What is Deviation Detection? Give example?

Deviation detection, also known as anomaly detection, is a technique used to identify unusual patterns or outliers within a dataset that do not conform to expected behavior. These anomalies can represent errors, fraud, unusual events, or genuine but rare occurrences that require attention and further investigation.
Example:
Suppose you are monitoring the temperature of a server room in a data center. The temperature typically fluctuates within a certain range due to variations in server activity and environmental conditions. However, if the temperature suddenly spikes to an extremely high value that is well outside the normal range, it could indicate a malfunctioning cooling system or a fire hazard. In this case, deviation detection techniques would analyze the temperature data in real-time and flag the unusually high temperature as an anomaly. This would trigger alerts to the data center staff, prompting them to investigate the cause of the anomaly and take corrective action, such as repairing the cooling system or evacuating the area in case of a fire. By detecting deviations from normal behavior, deviation detection helps in maintaining the reliability, efficiency, and safety of systems and processes.

6 Marks question

1.Write the difference between KDD and Data Mining.

DISTINGUISH BETWEEN KDD AND DATA MINING

KDD	DATA MINING
1. KDD refers to a process of identifying valid, novel, potentially useful, and ultimately understandable patterns and relationships in data. 2. It consists of various steps like data selection, cleaning, integration, transformation, and data mining. 3. It is iterative(MEANS rules that can be applied Repeatedly) 4. KDD is the representation of structure of data with which knowledge can be acquired.	1. Data Mining refers to a process of extracting useful and valuable information or patterns from large data sets. 2. It is a step inside the KDD process which deals with identifying patterns. 3. It is Non-iterative. 4. It works with large amount of data and is concerned with algorithmic means by which patterns are recognized.

2) Define KDD process. Explain the different stages of KDD. (6)

Knowledge discovery in databases (KDD) is the process of discovering useful knowledge from a collection of data.

This widely used data mining technique is a process that includes data preparation and selection, data cleaning, data integration, prior knowledge on data sets and interpreting accurate solutions from the observed results.

STAGES OF KDD

The stages of KDD, starting with the raw data and finishing with the extracted knowledge, are given below.

◆ **SELECTION**

This stage is concerned with selecting or segmenting the data that are relevant to some criteria. For example, for credit card customer profiling, we extract the type of transactions for each type of customers and we may not be interested in details of the shop where the transaction takes place.

◆ **PREPROCESSING**

Preprocessing is the data cleaning stage where unnecessary information is removed. (For example, it is unnecessary to note the sex of a patient when studying pregnancy!) When the data is drawn from several sources, it is possible that the same information is represented in different sources in different formats. This stage reconfigures the data to ensure a consistent format, as there is a possibility of inconsistent formats.

TRANSFORMATION

The data is not merely transferred across, but transformed in order to be suitable for the task of data mining. In this stage, the data is made usable and navigable.

DATA MINING

This stage is concerned with the extraction of patterns from the data.

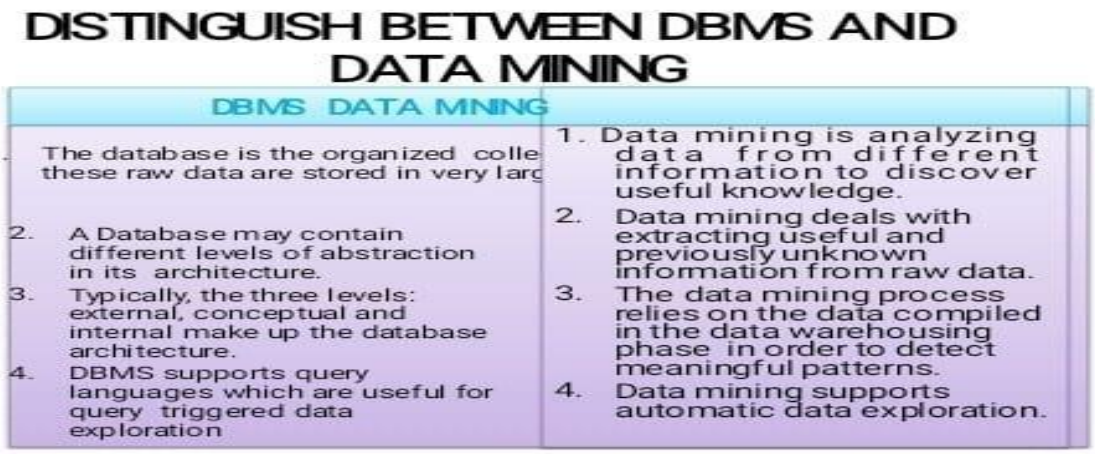
INTERPRETATION AND EVALUATION

The patterns obtained in the data mining stage are converted into knowledge, which in turn, is used to support decision-making.

DATA VISUALIZATION

Data visualization makes it possible for the analyst to gain a deeper, more intuitive understanding of the data and as such can work well alongside data mining. Data mining allows the analyst to focus on certain patterns and trends and explore them in- depth using visualization. On its own, data visualization can be overwhelmed by the volume of data in a database but in conjunction with data mining can help with exploration.

3.Distinguish between DBMS and Data Mining.



4) Explain interaction of Data Mining Systems with Relational DBMS

A relational database is a collection of tables, each of which is assigned a unique name. Each table consists of a set of attributes (columns or fields) and usually stores a large set of tuples (records or rows). Each tuple in a relational table represents an object identified by a unique key and described by a set of attribute values. A semantic data model, such as an entity-relationship (ER) data model, is often constructed for relational databases. An ER data model represents the database as a set of entities and their relationships.

Consider the following example.

A relational database for ALL Electronics. The All Electronics company is described by the following relation tables: customer, item, employee, and branch. Fragments of the tables described here are shown in Figure 1.6.

- The relation customer consists of a set of attributes, including a unique customer identity number (cust ID), customer name, address, age, occupation, annual income, credit information, category, and so on.
- Similarly, each of the relations item, employee, and branch consists of a set of attributes describing their properties.
- Tables can also be used to represent the relationships between or among multiple relation tables. For our example, these include purchases (customer purchases items, creating a sales transaction that is handled by an employee), items sold (lists the items sold in a given transaction), and works at (employee works at a branch of All Electronics).

Relational data can be accessed by database queries written in a relational query language, such as SQL, or with the assistance of graphical user interfaces. In the latter, the user may employ a menu, for example, to specify attributes to be included in the query, and the constraints on these attributes.

A given query is transformed into a set of relational operations, such as join, selection, and projection, and is then optimized for efficient processing. A query allows retrieval of specified subsets of the data. Suppose that your job is to analyse the All Electronics data. Through the use of relational queries, you can ask things like “Show me a list of all items that were sold in the last quarter.” Relational languages also include aggregate functions such as sum, avg (average), count, max (maximum), and min (minimum). These allow you to ask things like “Show me the total sales of the last month, grouped by branch,” or “How many sales transactions occurred in the month of December?” or “Which sales person had the highest amount of sales?"

When data mining is applied to relational databases, we can go further by searching for trends or data patterns. For example, data mining systems can analyse customer data to predict the credit risk of new customers based on their income, age, and previous credit information. Data mining systems may also detect deviations, such as items whose sales are far from those expected in comparison with the previous year. Such deviations can then be further investigated (e.g., has there been a change in packaging of such items, or a significant increase in price?).
Relational databases are one of the most commonly available and rich information repositories, and thus they are a major data form in our study of data mining.

5.Explain the following Data Mining techniques.
i) Verification Model ii) Discovery Model

VERIFICATION MODEL

In this process of data mining, the user makes a hypothesis and tests the hypothesis on the data to verify its validity. The emphasis is on the user who is responsible for formulating the hypothesis and issuing the query on the data to affirm or negate the hypothesis.

In a supermarket, for example, with a limited budget for a mailing campaign to launch a new product, it is important to identify the section of the population most likely to buy the new product. The user formulates a hypothesis to identify potential customers and their common characteristics. Historical data about transactions and demographic information can then be queried to reveal comparable purchases and the characteristics shared by those purchasers. The whole operation can be repeated by successive refinements of hypotheses until the required limit is reached. The user may come up with a new hypothesis or may refine the existing one and verify it against the database.

DISCOVERY MODEL

The discovery model differs in its emphasis, in that it is the systematically discovering important information hidden in the data: The data is sifted in search of frequently occurring patterns, trends and generalizations about the data without intervention or guidance from the user. The manner in which the rules are discovered depends on the class of the data mining application.

An example of such a model is a supermarket database, which is mined to discover the particular groups of customers to target for a mailing campaign. The data is searched with no hypothesis in mind other than for the system to group the customers according to the common characteristics found.

Ø The typical discovery driven tasks are
· Discovery of association rules
· Discovery of classification rules
· Clustering
· Discovery of frequent episodes · Deviation detection.
These tasks are of an exploratory nature and cannot be directly handed over to currently available database technology.

6) Write a note on i) Discovery of Association Rules ii) Discovery of Classification Rules

DISCOVERY OF ASSOCIATION RULES

An association rule is an expression of the form $X => Y$, where X and Y are
the sets of items. The intuitive meaning of such a rule is that the transaction of the database which contains X tends to contain Y. Given a database, the goal is to discover all the rules that have the support and confidence greater than or equal to the minimum support and confidence, respectively.

Let $L = \{l1, l2, \ldots, lm\}$ be a set of literals called items. Let D, the database, be a set of transactions, where each transaction T is a set of items. T supports an item x, if x is in T. T is said to support a subset of items X, if T supports each item x in X, $X => Y$ holds with *confidence* c, if c% of the transactions in D that support X also support Y. The rule $X => Y$ has *support* s in the transaction set D if s% of the transactions in D

support X____Y. *Support* means how often *X* and *Y* occur together as a percentage of the total transactions. *Confidence* measures how much a particular item is dependent on another.
Thus, the association with a very high support and confidence is a pattern that occurs often in the database that should be obvious to the end user. Patterns with extremely low support and confidence should be regarded as of no significance. Only patterns with a combination of intermediate values of confidence and support provide the user with interesting and previously unknown information.

DISCOVERY OF CLASSIFICATION RULES

Classification involves finding rules that partition the data into disjoint groups. The input for the classification is the training data set, whose class labels are already known. Classification analyses the training data set and constructs a model based on the class label, and aims to assign a class label to the future unlabelled records. Since the class field is known, this type of classification is known as supervised learning. A set of classification rules are generated by such a classification process, which can be used to classify future data and develop a better understanding of each class in the database. We can term this as *supervised learning* too.

There are several classification discovery models. They are: the decision trees, neural networks, genetic algorithms and the statistical models like linear/geometric discriminates. The applications include the credit card analysis, banking, medical applications and the like. Consider the following example.
The domestic flights in our country were at one time only operated by Indian Airlines. Recently, many other private airlines began their operations for domestic travel. Some of the customers of Indian Airlines started flying with these private airlines and, as a result, Indian Airlines lost these customers. Let us assume that Indian Airlines wants to understand why some customers remain loyal while others leave. Ultimately, the airline wants to predict which customers it is most likely to lose to its competitors. Their aim to build a model based on the historical data of loyal customers versus customers who have left.

7) Write a note on i) Clustering ii) Frequent Episodes

Clustering:

Clustering is a method of grouping data into different groups, so that the data in each group share similar trends and patterns. Clustering constitutes a major class of data mining algorithms. The algorithm attempts to automatically partition the data space into a set of regions or clusters, to which the examples in the table are assigned, either deterministically or probability-wise. the goal of the process sis to identify all sets of similar examples in the data in some optional fashion. Clustering according to similarity is a concept which appears in many disciplines. If a measure of similarity is available, then there are a number of techniques for forming clusters. Another approach is to build set functions that measure some particular property of groups. this latter approach achieves what is known as optimal partitioning.

The objectives of clustering are: ▪ To uncover natural groupings
 To initiate hypothesis about the data
 To find consistent and valid organization of the data.

A retailer may want to know where similarities exists in his customer base, so that he can create and understand different groups. He can use the existing database of the different customers or more specially , different transactions collected over a period of time. The clustering methods will help him in identifying different categories of customers. During the discovery process, the differences between data sets can be discovered in order to separate them into different groups and similarity between data sets can be used to group similar data together.

FREQUENT EPISODES

Frequent episodes are the sequence of events that occur frequently, close to each other and are extracted from the time sequences. How close it has to be to consider it as frequent is domain dependent. This is given by the user as the input and the output are the prediction rules for the time sequences.

Given a set R of event types, an event is a pair (4, 1) where A ∈ R is an event type and is an integer, we can calculate the occurrence time of the event. An event sequence s of R is a triple (Ts, Tc, S), where T's Te are integers. T's is the starting time and Te is the ending time.

S={(A1, 1), (A2, 12), (A, 1)) is the ordered sequence of events, such that A, ∈ R and Tst Te for all i 1, 2... n-1.

These episodes can be of three types. The serial episodes which occur in sequence. The parallel episodes in which there are no constraints on the order of the event types A and B given. And the non-serial and non-parallel episodes which occur in a sequence

if the occurrences of A and B precede an occurrence of C, and there is no constraint on

the relative order of A and B given.

The applications include telecommunications, and share market analysis, and these are mainly used for temporal data.

8) Write a note on i) Genetic Algorithms ii) Rough Set Techniques

Generic Algorithms

Genetic algorithms are a relatively new computing paradigm, inspired by Darwin's theory of evolution. A population of individuals, each representing a possible solution to a problem, is initially created at random. Then pairs of individuals combine (crossover) to produce offspring for the next generation. A mutation process is also used to randomly modify the genetic structure of some members of each new generation. The algorithm runs to generate solutions for successive generations. The probability of an individual reproducing is proportional to the goodness of the solution it represents. Hence, the quality of the solutions in successive generations improves. The process is terminated when an acceptable or optimum solution is found, or after some fixed time limit. Genetic algorithms are appropriate for problems which require optimization, with respect to some computable criterion.

This paradigm can also be applied to data mining problems. Here, the quantity to be minimized is often the number of classification errors on a training set. However, the mining of large data sets by genetic algorithms has only recently become practical due to the availability of affordable, high-speed computers. But there is hardly any special genetic algorithm designed to suit data mining problems.

ROUGH SETS TECHIQUES

The rough sets theory has recently become a popular theory in the field of data mining. The theory, introduced by Pawlak in the early 1980s, formal framework for the automated transformation of data into Knowledge. The rough sets theory, though mathematically simple, has displayed its fruitfulness in a variety of data mining areas. The rough set is a pir of approximations---lower approximation and upper approximation sets. The Lower approximation is also said to be positive cases and the upper approximation is said to be possible cases. using these approximation, the rough set theory develops tools to be discover rules from the given databases. A wide range of applications utilize the ideas of the theory. Medical and data analysis, aircraft pilot performance evaluation, image processing and voice recognition are a few examples. Almost inevitably, the database used for data mining will contain imperfections, such as noise, unknown values or errors due to inaccurate +measuring equipment. The rough set theory comes

handy for dealing with these types of problems, as it is a tool for handling vagueness and uncertainty inherit to decision situations. An important result from the theory is that it simplifies the search for dominating attributes leading to specific properties, or just rules pending in the data.

9) Write a note on Support Vector Machines(SVM)

Support Vector Machines(SVM) is based on statistical learning theory and is increasingly becoming useful in data mining. Main idea to non-linearly map the dataset into a high dimensional feature space and use linear discriminator to classify data. Its success has been demonstrated in the areas of regression, classification and decision tree construction.

- Unlike other classification techniques, which attempt to minimize the error of classification SVMs incorporate structured risk minimization which minimizes an upper bound on the generalized error.
- Consider a simple case, if A and B are two sets, the idea is to determine from infinite number of planes correctly separating A and B with smallest generalization error. SVMs select the plane which maximizes the margin separating the two classes. The margin is defined as the distance between the separating hyperplane to the nearest point of A, plus the distance from the hyperplane to the nearest point in B.
-

10) Explain any two discovery driven tasks.

The typical discovery driven tasks are

- Discovery of association rules
- Discovery of classification rules
- Clustering
- Discovery of frequent episodes
- Deviation detection.

These tasks are of an exploratory nature and cannot be directly handed over to currently available database technology. We shall concentrate on these tasks now.

Discovery of Association Rules

An association rule is an expression of the form $X \Rightarrow Y$, where X and Y are the sets of items. The intuitive meaning of such a rule is that the transaction of the database which contains X tends to contain Y. Given a database, the goal is to discover all the rules that have the support and confidence greater than or equal to the minimum support and confidence, respectively.

Let $L=\{l_1,l_2,...,l_m\}$ be a set of literals called items. Let D, the database, be a set of transactions, where each transaction T is a set of items. T supports an item x, if x is in T. T is said to support a subset of items X, if T supports each item x in X. $X \Rightarrow Y$ holds with *confidence* c, if c% of the transactions in D that support X also support Y. The rule $X \Rightarrow Y$ has *support* s in the transaction set D if s% of the transactions in D support $X \cup Y$. *Support* means how often X and Y occur together as a percentage of the total transactions. *Confidence* measures how much a particular item is dependent on another.

Thus, the association with a very high support and confidence is a pattern that occurs often in the database that should be obvious to the end user. Patterns with extremely low support and confidence should be regarded as of no significance. Only patterns with a combination of intermediate values of confidence and support provide the user with interesting and previously unknown information. We shall study the techniques to discover association rules in Chapter 4.

Clustering

Clustering is a method of grouping data into different groups, so that the data in each group share similar trends and patterns. Clustering constitutes a major class of data

mining algorithms. The algorithm attempts to automatically partition the data space into a set of regions or clusters, to which the examples in the table are assigned, either deterministically or probability-wise. The goal of the process is to identify all sets of similar examples in the data, in some optimal fashion.

Clustering according to similarity is a concept which appears in many disciplines. If a measure of similarity is available, then there are a number of techniques for forming clusters. Another approach is to build set functions that measure some particular property of groups. This latter approach achieves what is known as optimal partitioning.

The objectives of clustering are:

- to uncover natural groupings
- to initiate hypothesis about the data
- to find consistent and valid organization of the data.

A retailer may want to know where similarities exist in his customer base, so that he can create and understand different groups. He can use the existing database of the different customers or, more specifically, different transactions collected over a period of time. The clustering methods will help him in identifying different categories of customers. During the discovery process, the differences between data sets can be discovered in order to separate them into different groups, and similarity between data sets can be used to group similar data together. Chapter 5, we shall discuss in detail about using the clustering algorithm for data mining tasks.

11) Write a note on the following:
i)Sequence Mining ii)Web Mining iii)Spatial Data Mining

SEQUENCE MINING:-

It consists sequence of ordered elements or events (with or without time) Sequential pattern mining is a mining of frequently occurring ordered events or subsequence patterns.

Example:-

Customer buys a Laptop, then pen drive, then mouse, then keyboard……

WEB MINING:-

Web mining is the use of the data mining technic automatically discover and extract information like documents/services.

Discovering useful information from the world wide web and its usage pattern.

SPATIAL DATA MINING:-

Spatial data mining is the branch of data mining that deals with spatial(location) data it is a process of discovering interesting and previously unknown, but potentially useful, patterns from large spatial datasets. Extracting interesting and useful patterns from spatial datasets is more difficult than extracting the corresponding patterns from traditional numeric and categorical data due to the complexity of spatial data types, spatial relationship and spatial autocorrelation.

12) List and explain the issues and challenges in Data Mining The difficulties in data mining can be categorized as 1.Limited information.
2.Noise or missing data.
3.User interaction and prior knowledge
4.Uncertainty and irrelevant fields.
5.Size, updates and irrelevant fields.

1. Limited Information:-

A database is often designed for purposes other then that or data mining and, sometimes some attributes which are essential for knowledge discovery of the application domain are not present in the data.

2.Noise and missing data:-

Noisy data are data with a of additional large amount meaningless information in it called noise. Missing data can be treated in a number of ways:- ->Simply disregarding missing values.
->Omitting corresponding records.
->Treating missing data as a special included additionally in the value to be attribute domain. Kinds of Noise

3.User interaction and prior knowledge:-

An analyst is usually not a KDD expert, but simply a person making use of the data by means of the available KDD techniques. Since the KDD process is by definition interactive and iterative, it is challenging to provide a high performance, rapidresponse environment that also assists the users in the proper selection and matching of the appropriate techniques to achieve their goals.

4.Uncertainity:-

This refers to the severity of error and the degree of noise in the data precision is an important consideration in a discovery system.

5.Size,updates and irrelevant fields:-

Database tend to be large and dynamic, in that their contents are keep changing as information is added, modified or remove.

13) Explain the different application areas of Data Mining.

DM APPLICATION AREAS

1. Business Transaction Data:-

Modern business processes are consolidating with millions of customers and billions of their transactions. Business enterprises require necessary information for their effective functioning in today's competitive world. For example, they would like know: "Is this transaction fraudulent?"; "Which customer is likely to migrate?", and "What product is this customer most likely to buy next?".

2. Electronic Commerce Data:-

Electronic commerce produce large data sets in which the analysis of marketing patterns and risk patterns is critical.

3. Scientific and Engineering Data;-

Scientific data and metadata tend to be more complex in structure than business data. In addition, scientists and engineers are making increasing use of simulation and systems with application domain knowledge.

4. **Health care data:-**

Hospitals, health care organizations, insurance companies, and the concerned government agencies accumulate large collections of data about patients and health care-related details.

5. Web Data:-

The data on the web is growing not only in volume but also in complexity. Web data now includes not only text, audio and video material, but also streaming data and numerical data.

6. Multimedia Documents:-

Today's technology for retrieving multimedia items on the web is far from satisfactory. On the other hand, an increasingly large number of matters are on the web and the number of users is also growing explosively. It is becoming harder to extract meaningful information from the archives of multimedia data as the volume grows. **7. Data web:-**

The Extensible Markup Language (XML) is an emerging language for working with data in networked environments. As this infrastructure grows, data mining is expected to be a critical enabling technology for the emerging data web.

Unit-II

2 marks questions

1) What is data warehouse?

Data warehouse is a subject oriented integrated time varient and nonvolatile collection of data in support of management 's decision making process.

2) How are organizations using the information from data warehouses?

Organization use data warehouse in data science to extract valueable insights for decision-making.This includes analysing historical trends,identifing patterns,and making prediction based on the data stored within the warehouse.

3) Expand OLTP and OLAP.

OLTP : Online Transaction Processing

OLAP : Online Analytical Processing

4) What is data cube? How it is defined?

Data cube allows data to be mode led and viewed in multiple dimension dimension are the perspective or entities with respect to which an organization wants to keep records.

5. Define dimensions and facts.

Dimensions: Dimensions are descriptive attributes or characteristics of data that provide context to the facts. They define the structure of data and help categorize and describe the facts in a meaningful way. Examples include time, geography, and product categories.

Facts: Facts are quantitative data or measurements that are the focus of analysis in a data warehouse or data mart. They usually consist of numerical values and are subject to aggregation. Examples include sales amounts, profit margins, and quantities sold.

6. Define base cuboid and apex cuboid.

Base Cuboid: The base cuboid is the multi-dimensional representation of the most detailed level of data in a data cube. It contains all possible dimensions at their finest granularity, representing the raw data before any aggregation.

Apex Cuboid: The apex cuboid, also known as the top or 0-D cuboid, is the highest level of summarization in a data cube. It contains the most aggregated data, often just a single value representing the summary of all data points.

7. Define measure and mention any two categories.

Measure: A measure is a numeric data value that quantifies a business process, typically used for analysis and decision-making. Measures are the actual values that are analyzed within a data cube.

Categories of Measures:

1. Additive Measures: Measures that can be summed across any of the dimensions. Example: Total Sales.
2. Non-additive Measures: Measures that cannot be summed across any dimension. Example: Ratios and percentages.

8. Define concept hierarchy. Give an example.

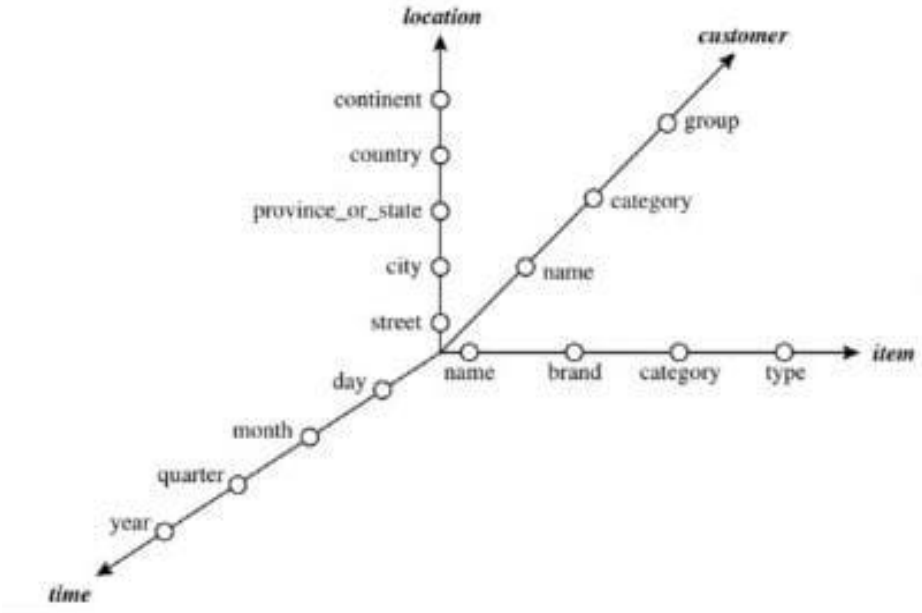
Concept Hierarchy: A concept hierarchy is a system of organizing data into multiple levels of abstraction or granularity. It defines a sequence of mappings from a set of low-level concepts to higher-level, more general concepts.
Example: A geographical concept hierarchy might look like this:
- Country > State > City > Neighborhood
In this hierarchy, data can be analyzed at different levels, from the country level down to the neighborhood level, allowing for more detailed or more generalized analysis.

9) Write any two differences between OLAP system and Statistical database.

	OLAP system	Statistical database
Purpose	Designed for business intelligence, enabling quick, multidimensional data analysis for decision-making	Designed for detailed statistical analysis and modelling, often used in scientific and research contexts
Data handling	Organize data in multidimensional cubes, optimized for aggregation and exploration across various dimensions	Handle data in tabular or matrix formats, focused on performing complex statistical computations and analysis

10) What is Starnet Model? Give an example through diagram.

A Starnet model consists of radial lines emanating from a central point, where each line represents a concept hierarchy for a dimension. Each abstraction level in the hierarchy is called a footprint. These represent the granularities available for use by OLAP operations such as drill-down and roll-up.



11) List any two benefits of Business analyst by having data warehouse.

- A data warehouse may provide a competitive advantage by presenting relevant information from which to measure performance and make critical adjustments in order to help win over competitors.
- A data warehouse can enhance business productivity because it is able to quickly and efficiently gather information that accurately describes the organization.

12) List any two data warehouse models from the architecture point of view.

1.The enterprise warehouse
2.The data mart
3.The virtual warehouse.

13. List any four back-end tools and utilities included in data warehouse.

1. Data Extraction Tools: These tools are used to extract data from various source systems, such as databases, flat files, and other data sources.
2. Data Cleaning Tools: These tools help in detecting and correcting errors and inconsistencies in the data to improve its quality.
3. Data Transformation Tools: These tools are used to convert data from its original format or structure into a format suitable for analysis and reporting.
4. Data Loading Tools: These tools facilitate the process of loading transformed data into the data warehouse.

14. Define data cleaning and data integration.

Data Cleaning: Data cleaning, also known as data cleansing, is the process of identifying and correcting errors, inconsistencies, and inaccuracies in the data to improve its quality. This includes handling missing values, removing duplicates, and correcting data entry errors.
Data Integration: Data integration is the process of combining data from different sources to provide a unified view. It involves merging data from various databases, applications, and systems to ensure that all relevant data is available for analysis in a cohesive format.

15. Define data transformation and data reduction.

Data Transformation: Data transformation is the process of converting data from its original format into a format suitable for analysis. This includes operations such as normalization, aggregation, generalization, and attribute construction to enhance the quality and usability of the data.
Data Reduction: Data reduction involves reducing the volume of data while maintaining its analytical value. This is achieved through techniques like dimensionality reduction, data compression, and aggregation, making the data easier to store and process without significant loss of information.

16. What is Dimensionality Reduction? Mention any one method used for Dimensionality Reduction.

Dimensionality Reduction: Dimensionality reduction is the process of reducing the number of random variables or features under consideration. It aims to simplify the data set while retaining as much important information as possible, which helps in reducing computational complexity and improving model performance.

Method for Dimensionality Reduction:
- Principal Component Analysis (PCA): PCA is a statistical technique that transforms the original data into a new set of uncorrelated variables, called principal components, which are ordered by the amount of variance they capture from the data.

17) List any two methods for data discretization.

1. Equal Width Binning: This method divides the range of data into equal width intervals and assigns values to each interval. It's straightforward but can be sensitive to outliers.
2. Equal Frequency Binning: Here, the data is divided into intervals with an equal number of data points in each interval. This method can handle outliers better but may result in uneven intervals.

18) Write the difference between Operational database and Data Warehouse?

1.Purpose:

Operational Database: Designed for transactional processing and day-to-day operations of an organization. It is optimized for insert, update, and delete operations.
Data Warehouse: Used for analytical purposes, providing a centralized repository of integrated data from multiple sources. It's optimized for querying and analysis.

2.Data Structure:

Operational Database: Typically normalized to minimize redundancy and support transactional integrity.
Data Warehouse: Often denormalized to optimize for query performance and analytics.

3.Data Freshness:

Operational Database: Contains real-time or near-real-time data reflecting the current state of operations.
Data Warehouse: Aggregates data from various operational sources periodically (daily, weekly, etc.), providing a historical view.

4.Usage:

Operational Database: Supports day-to-day operations such as order processing, inventory management, etc.
Data Warehouse: Used for business intelligence, reporting, trend analysis, and decision-making.

5.Schema:

Operational Database: Designed with an application-specific schema that may evolve over time.
Data Warehouse: Utilizes a dimensional or star schema optimized for analytical queries and reporting.

6.Size and Scope:

Operational Database: Handles relatively smaller datasets, focusing on current operational needs.
Data Warehouse: Manages large volumes of data, often spanning multiple years, for historical analysis and reporting.

19) Define ROLAP and MOLAP?

ROLAP (Relational Online Analytical Processing) is a data modeling approach in data science where data is stored in a relational database management system (RDBMS) and queried using SQL. It provides flexibility in data storage and retrieval, as it leverages the relational database structure.

MOLAP (Multidimensional Online Analytical Processing) is another data modeling approach where data is stored in a multidimensional cube structure optimized for OLAP operations. It precalculates and stores aggregations to enhance query performance, making it suitable for complex analytical queries involving multiple dimensions.

20) What is Data mart? List its types?

A data mart is a subset of a data warehouse that is designed for a specific business line or department within an organization. It contains a focused subset of data relevant to the needs of a particular group of users.

Types of data marts :

1.Dependent Data Mart: This type of data mart directly relies on a data warehouse for its data.

2.Independent Data Mart: This data mart is created without the need for a data warehouse and is often used in smaller organizations or for specific projects.

3.Distributed Data Mart: Data is distributed across different physical locations or across different departments within an organization.

4.Virtual Data Mart: Instead of physically storing data, this type of data mart provides a virtual view of data from different sources, often through data virtualization techniques.

21. Explain Virtual Warehouse

Virtual Warehouse: A virtual warehouse is a type of data warehouse that does not physically store data. Instead, it provides a unified view of data that is stored across various physical sources. It uses data virtualization techniques to integrate data from different sources in real-time, allowing users to query and analyze data as if it were stored in a single location. This approach reduces the need for data replication and storage but may have performance implications depending on the complexity of queries and the underlying data sources.

22. What type of information should a Metadata Repository include?

A Metadata Repository should include the following types of information:

1. Source Metadata: Information about the origin of the data, such as source databases, tables, columns, data types, and data extraction methods.
2. Transformation Metadata: Details of the data transformation processes, including mappings, rules, logic, and algorithms used to convert source data into the target format.
3. Load Metadata: Information related to the loading process, such as load schedules, batch jobs, data load history, and error logs.
4. Structural Metadata: Descriptions of data structures, including data models, schemas, dimensions, facts, and hierarchies.
5. Business Metadata: Definitions and descriptions of business terms, metrics, and key performance indicators (KPIs), ensuring that all stakeholders have a common understanding of the data.

6. Operational Metadata: Information on the operational aspects of the data warehouse, such as usage statistics, performance metrics, and access controls.

23. What is Metadata Repository?

Metadata Repository: A metadata repository is a centralized database that stores metadata, which is data about data. It contains detailed information about the data's origin, structure, transformations, usage, and governance. The repository acts as a reference point for understanding, managing, and utilizing the data within a data warehouse or other data management systems. It helps ensure data consistency, quality, and compliance by providing comprehensive documentation and a single source of truth for metadata.

24. Explain the approaches to data cleaning as a process

Approaches to data cleaning typically involve several steps:
1. Data Profiling: Analyzing the data to understand its structure, content, and quality issues. This involves identifying missing values, outliers, duplicates, and inconsistencies.
2. Data Standardization: Ensuring that data conforms to predefined standards and formats. This may involve correcting formats, standardizing units of measurement, and ensuring consistency in naming conventions.
3. Data Enrichment: Enhancing the quality of the data by adding missing information or supplementing it with external data sources.
4. Data Deduplication: Identifying and removing duplicate records to ensure that each data entry is unique.
5. Data Validation: Implementing rules and checks to ensure data accuracy and consistency. This includes range checks, referential integrity checks, and logical consistency checks.
6. Data Correction: Correcting any identified errors and inconsistencies in the data. This can be done manually or through automated processes.
7. Data Integration: Merging data from different sources and ensuring that it is coherent and consistent.
8. Monitoring and Maintenance: Continuously monitoring data quality and implementing processes to maintain and improve data quality over time. This includes regular audits and updates to data cleaning procedures.
These steps ensure that the data used for analysis and decision-making is accurate, reliable, and fit for purpose.

25). Define Binning?

Binning is a top-down splitting technique based on a specified number of bins.These methods are also used as discretization methods for numerosity reduction and concept hierarchy generation. For example, attribute values can be discretized by applying equal-width or equal-frequency binning, and then replacing each bin value by the bin mean or median, as in smoothing by bin means or smoothing by bin medians, respectively. These techniques can be applied recursively to the resulting partitions in order to generate concept hierarchies. Binning does not use class information and is therefore an unsupervised discretization technique.

26). What are the use of Data scrubbing tool and Data Auditing tool?

Data scrubbing tools use simple domain knowledge to detect errors and make corrections in the data. These tools rely on parsing and fuzzy matching techniques when cleaning data from multiple sources. Data auditing tools used to find discrepancies by analyzing the data to

discover rules and relationships, and detecting data that violate such conditions. They are variants of data mining tools.

27). What is Entity Identification Problem?

The Entity Identification Problem refers to the challenge of matching equivalent real-world entities from multiple data sources during data integration. When combining data from various sources such as databases, data cubes, or flat files, it's essential to ensure that entities representing the same real-world object are correctly identified and merged.

28) .List the processes involved in data transformation?

- Smoothing,
- Aggregation,
- Generalization
- Normalization,
- Attribute construction

29). What is the use of Attribute Subset Selection?

Attribute subset selection is used to detect and remove irrelevant, weakly relevant, or redundant attributes or dimensions. This process improves model accuracy, reduces computational cost, simplifies models, and helps prevent overfitting.

30). What is Histograms? Mention the use of Multidimensional histograms

Histograms use binning to approximate data distributions and are a popular form of data reduction A histogram for an attribute, a partitions the data distribution of A into disjoint subsets, or buckets. If each bucket represents only a single attribute-value/frequency pair, the buckets are called singleton buckets.

Use of Multidimensional histograms:

Multidimensional histograms can capture dependencies between attributes. Such histograms have been found effective in approximating data with up to five attributes. More studies are needed regarding the effectiveness of multidimensional histograms for very high dimensions.

Long answer questions

1) Define and Explain Data Warehouse

Data Warehouse Definition: A data warehouse is a centralized repository designed to store, manage, and facilitate the analysis of large volumes of structured and semi-structured data. It aggregates data from multiple sources, including transactional databases, business applications, and external data sources, to provide a unified view of the organization's data.

Explanation:

1. Subject-Oriented: Data warehouses are designed around key subjects or business areas, such as sales, finance, and customer information, rather than being organized by individual transactions. This focus on specific subjects enables more comprehensive and targeted analysis.
2. Integrated: Data from diverse sources is integrated into a consistent format in the data warehouse. This integration involves cleaning, transforming, and consolidating data to resolve inconsistencies and ensure data quality.
3. Non-volatile: Once data is entered into a data warehouse, it remains static and is not updated in real-time. Historical data is preserved to enable trend analysis, reporting, and business intelligence. Data warehouses are updated periodically through batch processes.

4. Time-variant: Data warehouses maintain historical data, providing a temporal context for analysis. This allows users to analyze changes over time, track trends, and make time-based comparisons, which are crucial for strategic decision-making.
5. Optimized for Query and Analysis: Unlike operational databases, data warehouses are optimized for complex queries and data analysis rather than transaction processing. They use indexing, partitioning, and other techniques to enhance query performance and support large-scale data retrieval.
Architecture Components:
- Data Sources: Various internal and external data sources, including transactional databases, CRM systems, ERP systems, and more.
- ETL Processes: Extract, Transform, Load processes that extract data from source systems, transform it into a suitable format, and load it into the data warehouse.
- Data Storage: Centralized storage for integrated data, typically implemented using relational databases or columnar storage.
- Metadata: Data about the data, including definitions, source information, transformation rules, and data lineage.
- Access Tools: Tools and applications for querying, reporting, data mining, and business intelligence, providing users with the ability to access and analyze the data.

2) Explain Six Differences Between Operational Database System and Data Warehouse
1. Purpose:
- Operational Database: Designed for day-to-day transaction processing and data management in real-time. It supports the operations of a business, such as order processing, inventory management, and customer interactions.
- Data Warehouse: Designed for analytical processing and decision support. It stores historical data to enable complex queries, reporting, and analysis, providing insights for strategic decision-making.
2. Data Structure:
- Operational Database: Highly normalized to reduce redundancy and ensure data integrity. Tables are designed to efficiently handle transactional updates and real-time operations.
- Data Warehouse: Denormalized to optimize query performance and data retrieval. Data is often organized in star or snowflake schemas to facilitate fast and efficient analysis.
3. Data Update Frequency:
- Operational Database: Data is frequently updated in real-time as transactions occur. It requires high availability and quick response times to support business operations.
- Data Warehouse: Data is updated periodically through batch processes (e.g., daily, weekly). The focus is on data accuracy and historical consistency rather than real-time updates.
4. Query Type:
- Operational Database: Supports simple, short, and routine queries that are predictable and focused on individual records or small data sets (e.g., retrieving a customer's order history).
- Data Warehouse: Supports complex, long-running queries that involve large volumes of data and require aggregation, filtering, and joining of multiple data sets (e.g., sales trend analysis over the last five years).
5. Data Volume:
- Operational Database: Manages relatively small volumes of data compared to a data warehouse, focusing on current data that is relevant for ongoing operations.

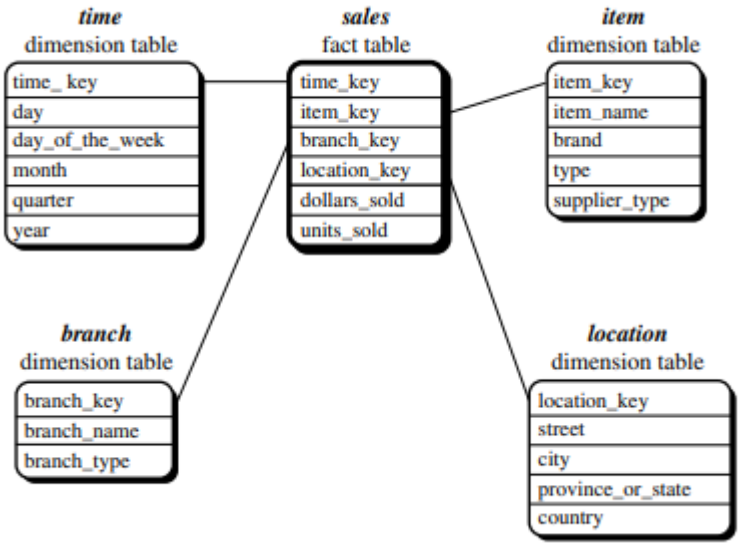
- Data Warehouse: Handles large volumes of historical data from various sources, enabling comprehensive analysis over extended time periods.

6. Users:
   - Operational Database: Used primarily by frontline employees, such as clerks, salespeople, and customer service representatives, who require immediate access to current data for operational tasks.
   - Data Warehouse: Used by data analysts, business analysts, managers, and executives who need to analyze data and generate reports for strategic planning and decision-making.
7. Performance Optimization:
   - Operational Database: Optimized for high transaction throughput and quick response times for individual record retrievals. Uses indexing, caching, and optimized SQL queries to support frequent, small-scale transactions.
   - Data Warehouse: Optimized for read-intensive operations and complex queries. Uses techniques such as indexing, partitioning, and parallel processing to efficiently handle large-scale data retrieval and aggregation.
8. Data Consistency:
   - Operational Database: Ensures immediate consistency of data through ACID (Atomicity, Consistency, Isolation, Durability) properties to maintain data integrity during transactions.
   - Data Warehouse: Ensures eventual consistency, with data being integrated and cleansed from multiple sources, focusing on accuracy and completeness for analytical purposes.

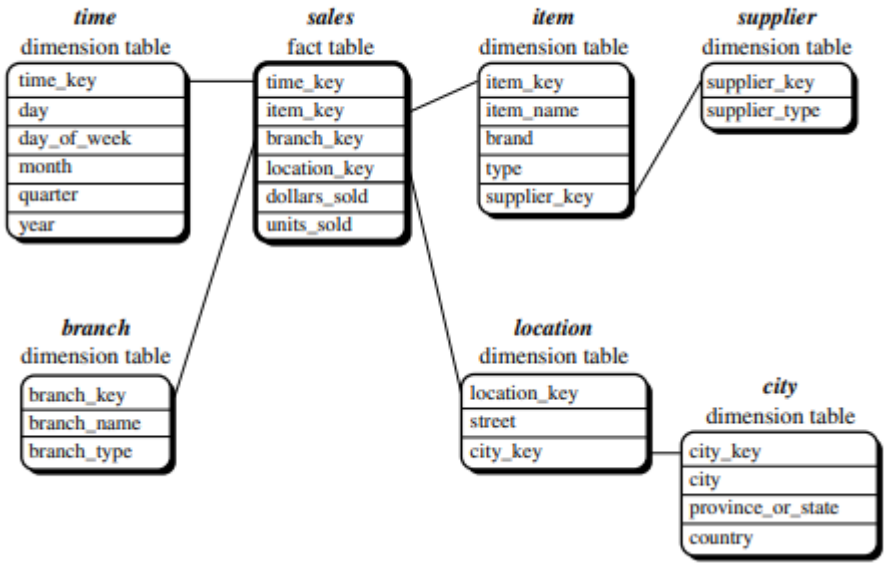
By understanding these differences, organizations can effectively utilize both operational databases and data warehouses to meet their specific data processing and analysis needs.

3) Explain the following Multidimensional database schema with example. (3 marks each)
i.) Star schema
ii.) Snowflake schema
iii.) Fact constellation schema

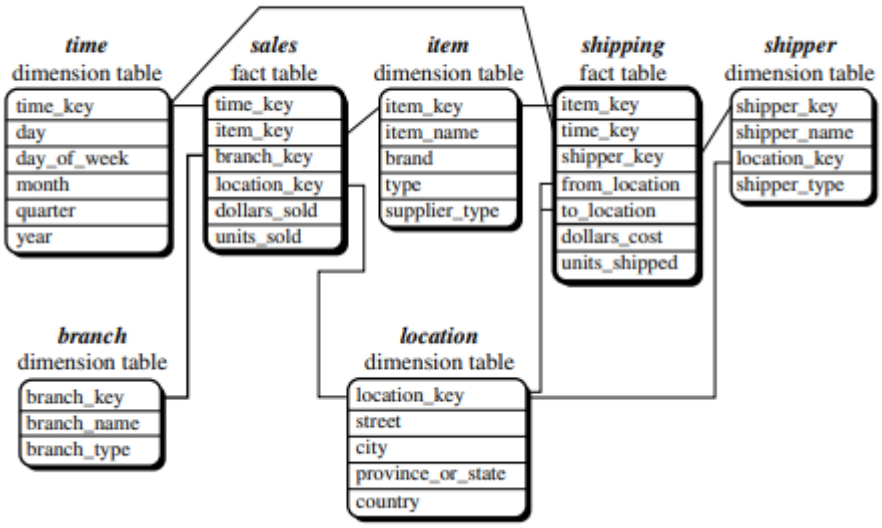
i) Star schema. A star schema for AllElectronicssales is shown in Figure 3.4. Sales are considered along four dimensions, namely,time, item, branch, and location. The schema contains a central fact table for sales that contains keys to each of the four dimensions, along with two measures: dollars sold and units sold. To minimize the size of the fact table, dimension identifiers (such as time key and item key) are system-generated identifiers. Notice that in the star schema, each dimension is represented by only one table, and each table contains a set of attributes. For example, the location dimension table contains the attribute set {location key, street, city, province or state, country}. This constraint may introduce some redundancy. For example, “Vancouver” and “Victoria” are both cities in the Canadian province of British Columbia. Entries for such cities in the location dimension table will create redundancy among the attributes province or state and country, that is, (..., Vancouver, British Columbia, Canada) and (..., Victoria, British Columbia, Canada). Moreover, the attributes within a dimension table may form either a hierarchy (total order) or a lattice (partial order).



ii) The snowflake schema is a variant of the star schema model, where some dimension tables are normalized, thereby further splitting the data into additional tables. The resulting schema graph forms a shape similar to a snowflake.
. A snowflake schema for AllElectronics sales is given in Figure 3.5. Here, the sales fact table is identical to that of the star schema in Figure 3.4. The main difference between the two schemas is in the definition of dimension tables. The single dimension table for item in the star schema is normalized in the snowflake schema, resulting in new item and supplier tables. For example, the item dimension table now contains the attributes item key, item name, brand, type, and supplier key, where supplier key is linked to the supplier dimension table, containing supplier key and supplier type information. Similarly, the single dimension table for location in the star schema can be normalized into two new tables: location and city. The city key in the new location table links to the city dimension. Notice that further normalization can be performed on province or state and country in the snowflake schema shown in Figure 3.5, when desirable



iii) A fact constellation schema is shown in Figure 3.6. This schema specifies two fact tables, sales and shipping. The sales table definition is identical to that of the star schema (Figure 3.4). The shipping table has five dimensions, or keys: item key, time key, shipper key, from location, and to location, and two measures: dollars cost and units shipped. A fact constellation schema allows dimension tables to be shared between fact tables. For example, the dimensions tables for time, item, and location are shared between both the sales and shipping fact tables. In data warehousing, there is a distinction between a data warehouse and a data mart. A data warehouse collects information about subjects that span the entire organization, such as customers, items, sales, assets, and personnel, and thus its scope is enterprise-wide. For data warehouses, the fact constellation schema is commonly used, since it can model multiple, interrelated subjects. A data mart, on the other hand, is a department subset of the data warehouse that focuses on selected subjects, and thus its scope is departmentwide. For data marts, the star or snowflake schema are commonly used, since both are geared toward modeling single subjects, although the star schema is more popular and efficient.



4) Define Measure. Explain different categories of measures. (4)

As we have noted, the summary measure is essentially the main theme of the analysis in a multidimensional model. A measure value is computed for a given cell by aggregating the data corresponding to the respective dimension-value sets defining the cell. The measures can be categorized into 3 groups based on the kind of aggregate function used.

Distributive A numeric measure is distributive if it can be computed in a distributed manner as follows. Suppose the data is partitioned into a few subsets. The measure can be simply the aggregation of the measures of all partitions. For example, *count, sum, min* and *max* are distributive measures.

Algebraic An aggregate function is algebraic if it can be computed by an algebraic function with some set of arguments, each of which may be obtained by a distributive measure. For example, *average* is obtained by sum/count.

Other examples of distributive functions are *standard-deviations* and *center-of-mass*.

Holistic An aggregate function is holistic if there is no constant bound on the storage size needed to describe a subaggregate. That is, there does not exist an algebraic function that can be used to compute this function. Examples of such functions, are *median, mode, most-frequent*, etc.

EXAMPLE 2.2

Let us consider another example at this stage. A store called, *Deccan Electronics* may

create a sales data warehouse in order to keep records of the store's sales with respect to the time, product and location. Thus, the dimensions are *time, product* and *location*. These dimensions allow the store to keep track of things like monthly sales of items, and the locations at which the items were sold. A dimension table for *product* contains the attributes *item name, brand*, and *type*. The attributes *shop, manager, city, region, state,* and *country* describe the dimension *location*. These attributes are related by a total order forming a hierarchy, such as *shop* < *city* < *state* < *country*. This hierarchy is shown in Figure 2.3. An example of a partial order for the *time* dimension are the attributes *week, month, quarter* and *year*. The sales data warehouse includes the sales amount in rupees and the total number of units sold. Note that we can have more than one numeric measure. Figure 2.4 shows the multidimensional model for such

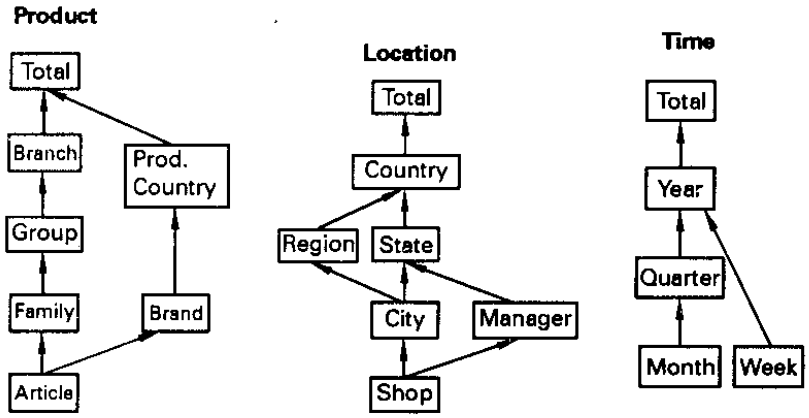


Figure 2.3 Dimension Schema

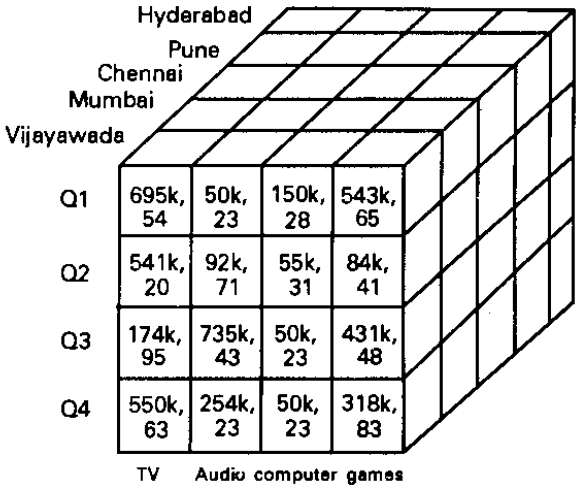


Figure 2.4 Data Cube for Example 2

5) Explain any four/six OLAP operations. (4/6)

Once we model our data warehouse in the form of a multidimensional data cube, it is necessary to explore the different analytical tools with which to perform the complex analysis of data. These data analysis tools are called OLAP (On-Line Analytical Processing). OLAP is mainly used to access the live data online and to analyze it. OLAP tools are designed in order to accomplish such analyses on very large databases. Hence, OLAP provides a user-friendly environment for interactive data analysis. Whilst data warehousing is primarily targeted at providing consolidated data for further analysis, OLAP provides the means to analyze those data in an application-oriented manner.

In the multidimensional model, the data are organized into multiple dimensions and each dimension contains multiple levels of abstraction. Such an organization provides the users with the flexibility to view data from different perspectives. There exist a number of OLAP operations on data cubes which allow interactive querying and analysis of the data. According to the underlying multidimensional view with classification hierarchies defined upon the dimensions, OLAP systems provide specialized data analysis methods. In this section, we study the basic OLAP operations for a multidimensional model.

Slicing This operation and the following one, Dicing, are used for reducing the data cube by one or more dimensions. The slice operation performs a selection on one dimension of the given cube, resulting in a subcube. Figure 2.5 shows a slice operation where the sales data are selected from the central cube for the dimension time, using the criteria *time*= '*Q2*'.

slice $_{\text{time='Q2'}}$ ***C*[quarter, city, product] = *C*[city, product]**

Dicing This operation is for selecting a smaller data cube and analyzing it from different perspectives. The dice operation defines a subcube by performing a selection on two or more dimensions. Figure 2.6 shows a dice operation on the central cube based on the following selection criteria, which involves three dimensions: *(location= "Mumbai" or "Pune"*) and (time = "*Q*l" or "*Q2*").

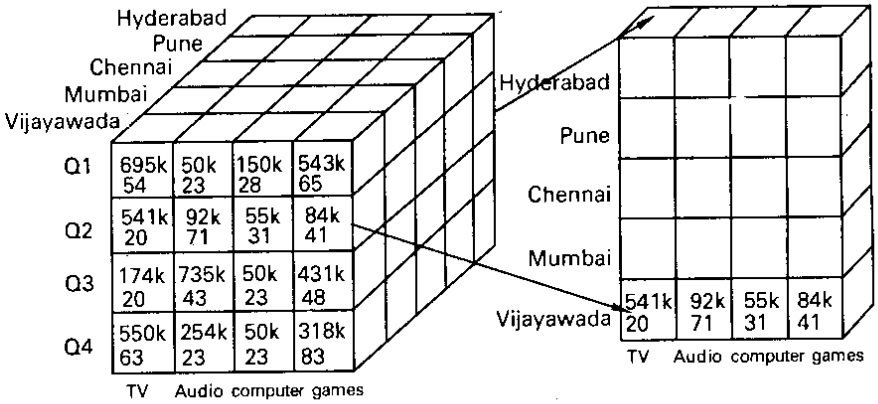


Figure 2.5 Slice Operation

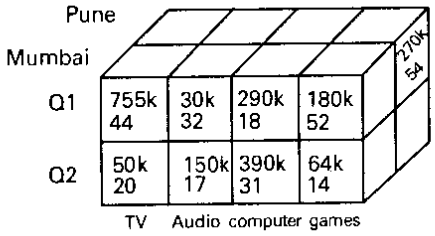


Figure 2.6 Dice Operation

$$\textbf{dice}_{\text{time='Q1' or 'Q2' and location = "Mumbai" or "Pune"}}\ \boldsymbol{C}[\textbf{quarter, city, product}]$$
$$= \boldsymbol{C}[\textbf{quarter}', \textbf{city}', \textbf{product}].$$

Where **quarter′ and city′** have truncated domains such as {Q1, Q2} and {Mumbai, Pune}, respectively.

Drilling This operation is meant for moving up and down along classification hierarchies. The different instances of drilling operations may be distinguished as follows:

Drill-up This operation deals with switching from a detailed to an aggregated level within the same classification hierarchy. The drill-up operation (also called as **roll-up** operation) performs aggregation on a data cube, either by *climbing-up a dimension hierarchy* or by *dimension* reduction. Figure 2.7 shows the result of a roll-up operation performed on the central cube by climbing up the dimension hierarchy for the *location* given in Figure 2.3. The roll-up operation aggregates the data by ascending the location hierarchy from the level of the city to the level of *province.* In other words, rather than grouping the data by the city, the resulting cube groups the data by the province.

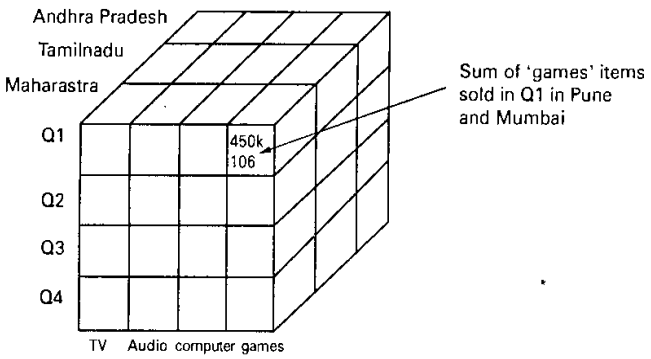


Figure 2.7 Roll-up Operation

roll-up$_{time}$ *C*[quarter, city, product] = *C*[quarter, province, product]

Here, each data cell of the resulting cuboid is the aggregation of the data cells that are merged due to the roll-up operation. In other words, the measures stored in the data cells, ***C*[Q1, Mumbai, computer]** and ***C*[Q1, Pune, computer]**, are added to determine the measure to be stored at ***C*[Q1, Maharashtra, computer]** (Maharashtra is a province or state in India and the cities Mumbai and Pune are in this province).

When roll-up is performed by dimension reduction, one or more dimensions are removed from the given cube. For example, consider a employment data cube containing only the two dimensions, profession and time. Roll-up may be performed by removing, say, the sex dimension, resulting in an aggregation of the total employment by profession and by year.

Drill-down This operation is concerned with switching from an aggregated to a more detailed level within the same classification hierarchy. Drill-down is the reverse

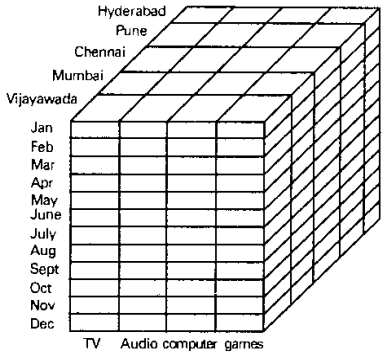


Figure 2.8 Drill-down Operation

of roll-up. It navigates from less detailed data to more detailed data. Drill-down can be realized by either *stepping-down* a dimension hierarchy or *introducing additional dimensions.*

Figure 2.8 shows the result of a drill-down operation performed on the central cube by stepping down a dimension hierarchy for a time period defined as *day* < *month* < *quarter* < *year*. Drill-down occurs by descending in the time hierarchy from the level of a quarter to the level of a month. The resulting data cube details the total sales per month rather than summarized by quarter. Drill-down can also be performed by adding new dimensions to a cube. For example, a drill-down on the given cube can occur by introducing an additional dimension, such as *customer-type.*

Drill-within It is switching from one classification to a different one within the same dimension;

Drill-across It means switching from a classification in one dimension to a different classification in a different dimension.

Pivot (rotate) Pivot (also called "rotate") is a visualization operation which rotates the data axes in order to provide an alternative presentation of the same data. Other examples include rotating the axes in a 3-D cube, or transforming a 3-D cube into a series of 2-D planes.

Other OLAP operations may include ranking the top-N or bottom-N items in lists, as well as computing moving averages, growth rates, interests, and internal rates of return, depreciation, currency conversions, and statistical functions. The results of these operations are typically visualized in a cross-tabular form, i.e., mapped to a grid-oriented, two-dimensional layout structure.

6) Explain different views regarding the business analysis framework design of a datawarehouse. (4)

Designing a data warehouse involves various perspectives to ensure it meets business analysis requirements. Here are the key views:

Top-Down View:

Description: This approach starts with the overall needs of the business. It emphasizes understanding the business requirements, identifying key performance indicators (KPIs), and establishing a data warehouse that supports strategic decision-making.

Focus: It ensures alignment with business objectives and focuses on a comprehensive view of the organization's data needs.

Data Source View:

Description: This view focuses on the sources of data that feed into the data warehouse. It involves understanding the different operational systems, databases, external data sources, and the data integration processes required to gather and consolidate data.

Focus: It emphasizes data extraction, transformation, and loading (ETL) processes to ensure data quality and consistency.

Data Warehouse View:

Description: This perspective centers on the structure and organization of the data within the data warehouse itself. It involves designing the schema (star schema, snowflake schema), defining fact and dimension tables, and ensuring efficient data storage and retrieval.
Focus: It focuses on how data is stored, indexed, and optimized for query performance.
Business Query View:
Description: This view considers how end-users will interact with the data warehouse. It involves understanding the types of queries users will run, the reports they need, and the OLAP tools or dashboards they will use for analysis.
Focus: It emphasizes user accessibility, ease of use, and the ability to perform complex analyses and generate insights.

7) Explain the steps in data warehouse design process. (4)

A data warehouse can be built using a top-down approach, a bottom-up approach, or a combination of both. The top-down approach starts with the overall design and planning. It is useful in cases where the technology is mature and well known, and where the business problems that must be solved are clear and well understood. The bottom-up approach starts with experiments and prototypes. This is useful in the early stage of business modeling and technology development. It allows an organization to move forward at considerably less expense and to evaluate the benefits of the technology before making significant commitments. In the combined approach, an organization can exploit the planned and strategic nature of the top-down approach while retaining the rapid implementation and opportunistic application of the bottom-up approach. From the software engineering point of view, the design and construction of a data warehouse may consist of the following steps: planning, requirements study, problem analysis, warehouse design, data integration and testing, and finally deployment of the data warehouse. Large software systems can be developed using two methodologies: the waterfall method or the spiral method. The waterfall method performs a structured and systematic analysis at each step before proceeding to the next, which is like a waterfall, falling from one step to the next. The spiral method involves the rapid generation of increasingly functional systems, with short intervals between successive releases. This is considered a good choice for data warehouse development, especially for data marts, because the turnaround time is short, modifications can be done quickly, and new designs and technologies can be adapted in a timely manner.

In general, the warehouse design process consists of the following steps:

1. Choose a business process to model, for example, orders, invoices, shipments, inventory, account administration, sales, or the general ledger. If the business process is organizational and involves multiple complex object collections, a data warehouse model should be followed. However, if the process is departmental and focuses on the analysis of one kind of business process, a data mart model should be chosen.
2. Choose the grain of the business process. The grain is the fundamental, atomic level of data to be represented in the fact table for this process, for example, individual transactions, individual daily snapshots, and so on.
3. Choose the dimensions that will apply to each fact table record. Typical dimensions are time, item, customer, supplier, warehouse, transaction type, and status.

4. Choose the measures that will populate each fact table record. Typical measures are numeric additive quantities like dollars sold and units sold.

8) Explain the three-tier architecture of Data warehouse with neat diagram. (6)

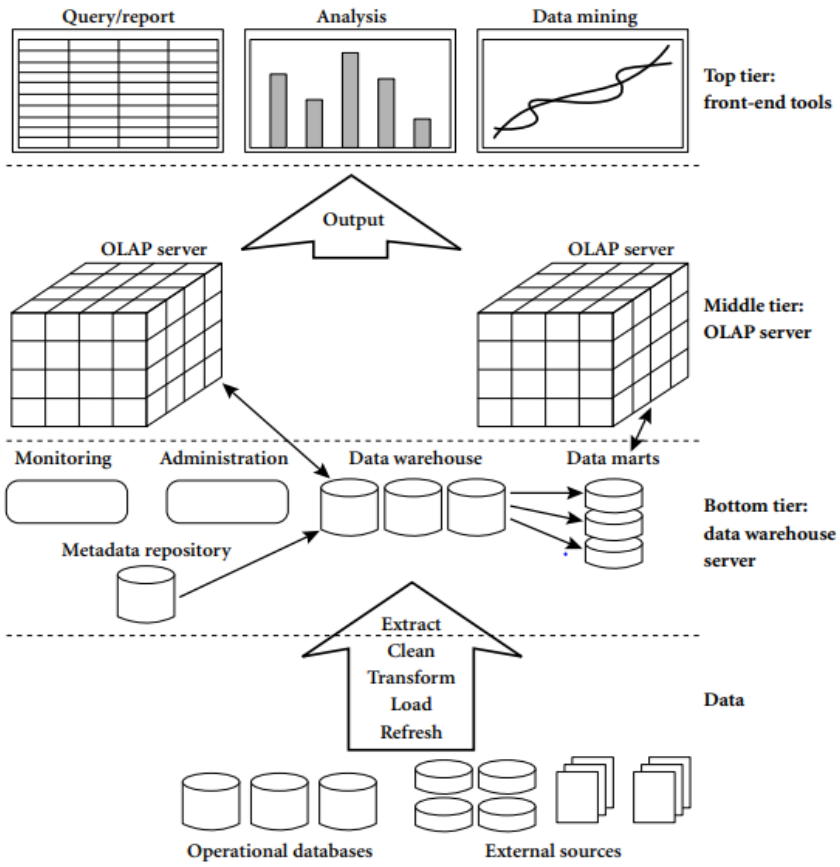


Figure 3.12 A three-tier data warehousing architecture.

Data warehouses often adopt a three-tier architecture, as presented in Figure 3.12.

1. The bottom tier is a warehouse database server that is almost always a relational database system. Back-end tools and utilities are used to feed data into the bottom tier from operational databases or other external sources (such as customer profile information provided by external consultants). These tools and utilities perform data extraction, cleaning, and transformation (e.g., to merge similar data from different sources into a unified format), as well as load and refresh functions to update the data warehouse (Section 3.3.3). The data are extracted using application program interfaces known as gateways. A gateway is supported by the underlying DBMS and allows client programs to generate SQL code to be executed at a server. Examples of gateways include ODBC (Open Database Connection) and OLEDB (Open Linking and Embedding for Databases) by Microsoft and JDBC (Java Database Connection). This tier also contains a metadata repository, which stores information about the data warehouse and its contents. The metadata repository is further described in Section 3.3.4.
2. The middle tier is an OLAP server that is typically implemented using either (1) a relational OLAP (ROLAP) model, that is, an extended relational DBMS that maps

operations on multidimensional data to standard relational operations; or (2) a multidimensional OLAP (MOLAP) model, that is, a special-purpose server that directly implements multidimensional data and operations. OLAP servers are discussed in Section 3.3.5.

3. The top tier is a front-end client layer, which contains query and reporting tools, analysis tools, and/or data mining tools (e.g., trend analysis, prediction, and so on).

9) Explain the different data warehouse models from the architecture point of view. (4)

From the architecture point of view, there are three data warehouse models: the enterprise warehouse, the data mart, and the virtual warehouse.

Enterprise warehouse: An enterprise warehouse collects all of the information about subjects spanning the entire organization. It provides corporate-wide data integration, usually from one or more operational systems or external information providers, and is cross-functional in scope. It typically contains detailed data as well as summarized data, and can range in size from a few gigabytes to hundreds of gigabytes, terabytes, or beyond. An enterprise data warehouse may be implemented on traditional mainframes, computer superservers, or parallel architecture platforms. It requires extensive business modeling and may take years to design and build.

Data mart: A data mart contains a subset of corporate-wide data that is of value to a specific group of users. The scope is confined to specific selected subjects. For example, a marketing data mart may confine its subjects to customer, item, and sales. The data contained in data marts tend to be summarized. Data marts are usually implemented on low-cost departmental servers that are UNIX/LINUX- or Windows-based. The implementation cycle of a data mart is more likely to be measured in weeks rather than months or years. However, it may involve complex integration in the long run if its design and planning were not enterprise-wide. Depending on the source of data, data marts can be categorized as independent or dependent. Independent data marts are sourced from data captured from one or more operational systems or external information providers, or from data generated locally within a particular department or geographic area. Dependent data marts are sourced directly from enterprise data warehouses.

Virtual warehouse: A virtual warehouse is a set of views over operational databases. For efficient query processing, only some of the possible summary views may be materialized. A virtual warehouse is easy to build but requires excess capacity on operational database servers

10) Explain the different back-end tools and utilities included in data warehouse. (4)

Data warehouses are complex systems that store, manage, and analyze large volumes of data from various sources. The back-end tools and utilities used in data warehouses play critical roles in data extraction, transformation, loading, storage, and maintenance. Here are some of the key back-end tools and utilities included in data warehouses:

1. ETL (Extract, Transform, Load) Tools:

- Extraction: Tools like Apache Nifi, Talend, and Informatica PowerCenter are used to extract data from various sources, such as databases, flat files, and web services.

- Transformation: Once extracted, data needs to be cleaned, formatted, and transformed to fit the structure and requirements of the data warehouse. Tools like Apache Spark, Talend, and IBM DataStage handle these tasks.
- Loading: The final step is loading the transformed data into the data warehouse. Tools like Apache Sqoop, Talend, and Informatica PowerCenter are commonly used.

2. Data Integration Tools:
- These tools integrate data from disparate sources to provide a unified view. Examples include Microsoft SQL Server Integration Services (SSIS), Informatica, and MuleSoft.

3. Data Cleansing Tools:
- Data quality is crucial for accurate analysis. Tools such as Trifacta, OpenRefine, and IBM QualityStage help in cleaning and standardizing data to ensure its accuracy and consistency.

4. Metadata Management Tools:
- Metadata describes the data stored in the data warehouse, helping in its organization and management. Tools like Apache Atlas, Collibra, and Alation are used for metadata management.

5. Database Management Systems (DBMS):
- These systems manage the storage and retrieval of data. Common DBMS for data warehouses include Oracle, Microsoft SQL Server, IBM Db2, and cloud-based systems like Amazon Redshift, Google BigQuery, and Snowflake.

6. Data Modeling Tools:
- Data modeling tools like ER/Studio, ERwin, and Oracle SQL Developer Data Modeler help design the schema of the data warehouse, ensuring it is optimized for query performance and data integrity.

7. Data Warehouse Automation Tools:
- These tools automate various aspects of data warehouse development and management, reducing the manual effort and potential for errors. Examples include WhereScape and TimeXtender.

8. Data Governance Tools:
- Ensuring data security, privacy, and compliance with regulations is critical. Tools like Collibra, Talend, and Informatica offer features for data governance and compliance management.

9. OLAP (Online Analytical Processing) Tools:
- OLAP tools such as Microsoft SQL Server Analysis Services (SSAS), Oracle OLAP, and IBM Cognos are used for multidimensional analysis of data, allowing for complex queries and reporting.

10. Performance Monitoring and Optimization Tools:
- To ensure the data warehouse performs efficiently, tools like SolarWinds Database Performance Analyzer, Quest Foglight, and AWS CloudWatch are used to monitor and optimize performance.

11. Backup and Recovery Tools:
- Data warehouses need robust backup and recovery solutions to prevent data loss. Tools like Veritas NetBackup, IBM Spectrum Protect, and Veeam are commonly used.

Each of these tools and utilities plays a vital role in the overall functionality of a data warehouse, ensuring it is efficient, reliable, and capable of handling large volumes of data from various sources.

11) What is metadata? Explain metadata repository. (6)

Metadata is essentially data about data. It provides information that helps to understand, organize, and manage the data within a system. In the context of data warehouses, metadata describes various aspects of the data stored within the warehouse, such as its source, structure, transformations, relationships, and usage. Metadata can be categorized into several types:
1. Business Metadata: Information about the data that is understandable to business users, such as definitions, business rules, and data quality metrics.
2. Technical Metadata: Details about the data's structure and format, such as data types, lengths, constraints, and storage locations.
3. Operational Metadata: Information about the processes and operations that affect the data, such as ETL processes, job schedules, and performance statistics.

Metadata Repository

A metadata repository is a specialized database that stores and manages metadata. It acts as a central hub for all metadata related to the data warehouse, providing a unified and consistent view of the data assets. Here are some key functions and benefits of a metadata repository:
1. Centralized Storage: It consolidates metadata from various sources, making it easier to manage and access.
2. Improved Data Quality and Consistency: By maintaining detailed metadata, the repository helps ensure that data is accurate, consistent, and conforms to defined standards.
3. Enhanced Data Governance and Compliance: It supports data governance initiatives by providing transparency and traceability of data, ensuring compliance with regulatory requirements.
4. Facilitates Data Integration: By providing detailed information about data sources and transformations, the repository helps in the integration of data from disparate systems.
5. Supports Data Lineage and Impact Analysis: It tracks the lineage of data, showing its origins, transformations, and movement across systems. This helps in understanding the impact of changes to data structures or processes.
6. Improves Collaboration: It enables better collaboration among different stakeholders, such as data engineers, data scientists, and business analysts, by providing a shared understanding of the data.

Key Components of a Metadata Repository

1. Metadata Definitions: The actual descriptions of data elements, their meanings, and their relationships.
2. Metadata Management Tools: Tools and interfaces that allow users to input, update, and manage metadata.
3. Metadata Access and Querying: Mechanisms that allow users to search, retrieve, and analyze metadata.
4. Integration with Other Systems: APIs and connectors that allow the metadata repository to interact with other systems and tools within the data warehouse ecosystem.

Examples of Metadata Repositories

- Apache Atlas: An open-source metadata and governance framework that provides a scalable set of governance services.
- Informatica Metadata Manager: Part of Informatica's data management suite, it provides comprehensive metadata management and visibility.
- Collibra Data Catalog: A data intelligence platform that includes metadata management, data cataloging, and governance.

By utilizing a metadata repository, organizations can significantly enhance their ability to manage and leverage their data assets effectively, leading to better decision-making and operational efficiency.

12) Explain different types of OLAP servers. (4/6)

Online Analytical Processing (OLAP) servers are specialized databases designed to enable quick and efficient querying and reporting of multidimensional data. They support complex analytical and ad-hoc queries with a rapid execution time. There are several types of OLAP servers, each with distinct architectures and use cases:

1. MOLAP (Multidimensional OLAP)

MOLAP servers store data in multidimensional cubes rather than relational databases. This approach involves pre-computing and storing aggregated data, which allows for fast query performance.

- Advantages:
  - High query performance due to pre-aggregated data.
  - Efficient data storage through compression techniques.
  - Fast data retrieval for complex queries and computations.
- Disadvantages:
  - Can be less scalable with very large datasets due to the need for pre-computation and storage of aggregations.
  - Limited flexibility in handling ad-hoc queries that are not pre-defined.
- Examples: IBM Cognos TM1, Microsoft SQL Server Analysis Services (SSAS) in MOLAP mode.

2. ROLAP (Relational OLAP)

ROLAP servers store data in relational databases and use SQL queries to perform the necessary computations. Unlike MOLAP, ROLAP does not require pre-aggregation of data.

- Advantages:
  - Can handle large volumes of data, leveraging the scalability of relational databases.
  - More flexible for ad-hoc querying since data does not need to be pre-aggregated.
  - Utilizes existing RDBMS infrastructure, reducing costs and complexity.

- Disadvantages:
  - Slower query performance compared to MOLAP due to on-the-fly calculations.
  - Performance heavily depends on the efficiency of SQL queries and indexing.
- Examples: Oracle OLAP, MicroStrategy, SAP BusinessObjects.

3. HOLAP (Hybrid OLAP)

HOLAP servers combine the strengths of both MOLAP and ROLAP by storing frequently accessed data in a multidimensional format and less frequently accessed data in a relational format.

- Advantages:
  - Balances performance and scalability, offering fast access to commonly queried data while accommodating larger datasets.
  - Provides flexibility for ad-hoc analysis without compromising on performance for standard queries.
- Disadvantages:

- More complex architecture and implementation compared to purely MOLAP or ROLAP solutions.
- Management of data synchronization between the multidimensional and relational storage can be challenging.
- Examples: Microsoft SQL Server Analysis Services (SSAS) in HOLAP mode.

4. DOLAP (Desktop OLAP)

DOLAP servers, also known as Desktop OLAP, are designed to run on individual desktops or laptops. They download a subset of the data from the central OLAP server and allow users to perform analysis offline.

- Advantages:
  - Enables offline analysis, which is useful for users without constant access to a network.
  - Reduces the load on central OLAP servers by distributing processing tasks to individual desktops.
- Disadvantages:
  - Limited by the processing power and storage capacity of the desktop machine.
  - Data subsets may become outdated if not regularly synchronized with the central server.

- Examples: Local instances of IBM Cognos TM1, Excel PivotTables connected to OLAP data sources.

5. WOLAP (Web OLAP)

WOLAP servers are web-based OLAP solutions that provide access to OLAP data through a web browser, enabling remote and widespread access.

- Advantages:
  - Accessible from anywhere with an internet connection, facilitating remote and mobile work.
  - Simplifies deployment and updates since the application is hosted centrally.
- Disadvantages:
  - Performance can be affected by network latency and bandwidth.
  - Security and data privacy concerns need to be addressed with web access.
- Examples: SAS Web OLAP Viewer, Oracle OLAP Web Services.

Each type of OLAP server is designed to meet specific needs, and the choice of which to use depends on factors like data volume, query complexity, performance requirements, and the existing IT infrastructure.

13) Explain the different forms of data preprocessing. (4)

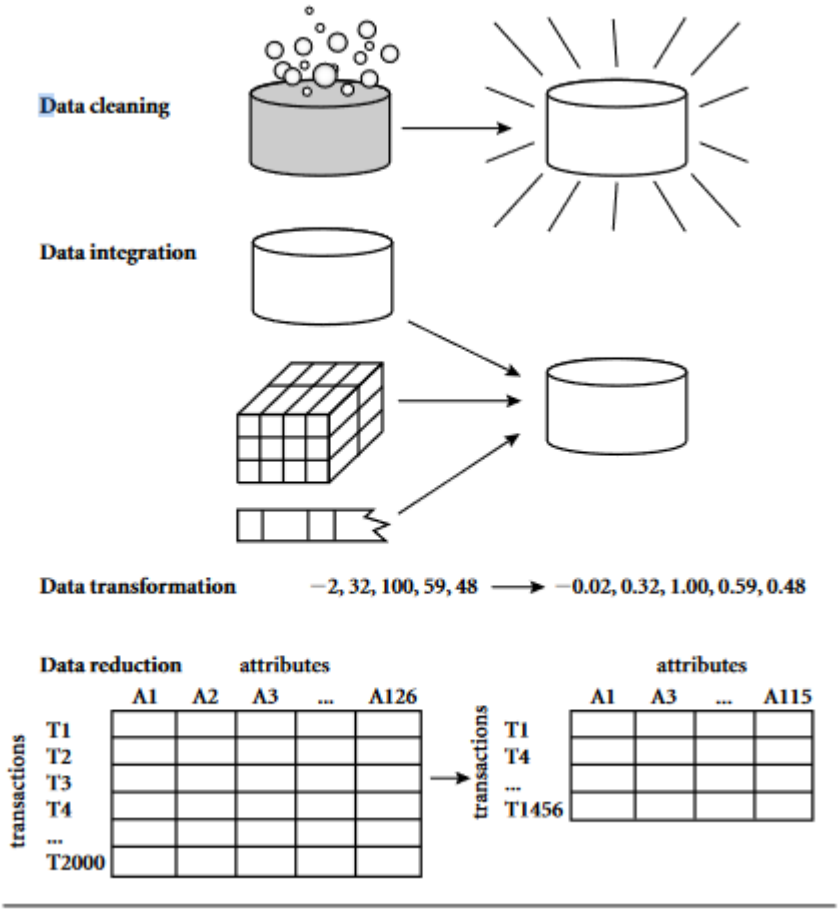


Figure 2.1 Forms of data preprocessing.

Duplicate tuples also require data cleaning.

Data cleaning routines work to "clean" the data by filling in missing values, smoothing noisy data, identifying or removing outliers, and resolving inconsistencies. If users believe the data are dirty, they are unlikely to trust the results of any data mining that has been applied to it. Furthermore, dirty data can cause confusion for the mining procedure, resulting in unreliable output. Although most mining routines have some procedures for dealing with incomplete or noisy data, they are not always robust. Instead, they may concentrate on avoiding overfitting the data to the function being modeled. Therefore, a useful preprocessing step is to run your data through some data cleaning routines. Section 2.3 discusses methods for cleaning up your data.

Getting back to your task at *AllElectronics*, suppose that you would like to include data from multiple sources in your analysis. This would involve integrating multiple

databases, data cubes, or files, that is, **data integration**. Yet some attributes representing a given concept may have different names in different databases, causing inconsistencies and redundancies. For example, the attribute for customer identification may be referred to as *customer_id* in one data store and *cust_id* in another. Naming inconsistencies may also occur for attribute values. For example, the same first name could be registered as "Bill" in one database, but "William" in another, and "B." in the third. Furthermore, you suspect that some attributes may be inferred from others (e.g., annual revenue). Having a large amount of redundant data may slow down or confuse the knowledge discovery process. Clearly, in addition to data cleaning, steps must be taken to help avoid redundancies during data integration. Typically, data cleaning and data integration are performed as a preprocessing step when preparing the data for a data warehouse. Additional data cleaning can be performed to detect and remove redundancies that may have resulted from data integration.

Getting back to your data, you have decided, say, that you would like to use a distance-based mining algorithm for your analysis, such as neural networks, nearest-neighbor classifiers, or clustering.[1] Such methods provide better results if the data to be analyzed have been *normalized*, that is, scaled to a specific range such as [0.0, 1.0]. Your customer data, for example, contain the attributes *age* and *annual salary*. The *annual salary* attribute usually takes much larger values than *age*. Therefore, if the attributes are

left unnormalized, the distance measurements taken on *annual salary* will generally outweigh distance measurements taken on *age*. Furthermore, it would be useful for your analysis to obtain aggregate information as to the sales per customer region—something that is not part of any precomputed data cube in your data warehouse. You soon realize that **data transformation** operations, such as normalization and aggregation, are additional data preprocessing procedures that would contribute toward the success of the mining process. Data integration and data transformation are discussed in Section 2.4.

"*Hmmm,*" you wonder, as you consider your data even further. "*The data set I have selected for analysis is HUGE, which is sure to slow down the mining process. Is there any way I can reduce the size of my data set, without jeopardizing the data mining results?*" **Data reduction** obtains a reduced representation of the data set that is much smaller in volume, yet produces the same (or almost the same) analytical results. There are a number of strategies for data reduction. These include *data aggregation* (e.g., building a data cube), *attribute subset selection* (e.g., removing irrelevant attributes through correlation analysis), *dimensionality reduction* (e.g., using encoding schemes such as minimum length encoding or wavelets), and *numerosity reduction* (e.g., "replacing" the data by alternative, smaller representations such as clusters or parametric models). Data reduction is the topic of Section 2.5. Data can also be "reduced" by *generalization* with the use of concept hierarchies, where low-level concepts, such as *city* for customer location, are replaced with higher-level concepts, such as *region* or *province_or_state*. A concept hierarchy organizes the concepts into varying levels of abstraction. *Data discretization* is a form of data reduction that is very useful for the automatic generation of concept hierarchies from numerical data. This is described in Section 2.6, along with the automatic generation of concept hierarchies for categorical data.

Figure 2.1 summarizes the data preprocessing steps described here. Note that the above categorization is not mutually exclusive. For example, the removal of redundant data may be seen as a form of data cleaning, as well as data reduction.

In summary, real-world data tend to be dirty, incomplete, and inconsistent. Data preprocessing techniques can improve the quality of the data, thereby helping to improve the accuracy and efficiency of the subsequent mining process. Data preprocessing is an

important step in the knowledge discovery process, because quality decisions must be based on quality data. Detecting data anomalies, rectifying them early, and reducing the data to be analyzed can lead to huge payoffs for decision making.

14) Explain (any four) the different ways of handling missing values. (4/6)

1. **Ignore the tuple:** This is usually done when the class label is missing (assuming the mining task involves classification). This method is not very effective, unless the tuple contains several attributes with missing values. It is especially poor when the percentage of missing values per attribute varies considerably.

2. **Fill in the missing value manually:** In general, this approach is time-consuming and may not be feasible given a large data set with many missing values.
3. **Use a global constant to fill in the missing value:** Replace all missing attribute values by the same constant, such as a label like "*Unknown*" or $-\infty$. If missing values are replaced by, say, "*Unknown,*" then the mining program may mistakenly think that they form an interesting concept, since they all have a value in common—that of "*Unknown.*" Hence, although this method is simple, it is not foolproof.
4. **Use the attribute mean to fill in the missing value:** For example, suppose that the average income of *AllElectronics* customers is $56,000. Use this value to replace the missing value for *income*.
5. **Use the attribute mean for all samples belonging to the same class as the given tuple:** For example, if classifying customers according to *credit_risk*, replace the missing value with the average *income* value for customers in the same credit risk category as that of the given tuple.
6. **Use the most probable value to fill in the missing value:** This may be determined with regression, inference-based tools using a Bayesian formalism, or decision tree induction. For example, using the other customer attributes in your data set, you may construct a decision tree to predict the missing values for *income*. Decision trees, regression, and Bayesian inference are described in detail in Chapter 6.

15) Define noise and explain different data smoothing techniques. (6)

"*What is noise?*" **Noise** is a random error or variance in a measured variable. Given a numerical attribute such as, say, *price*, how can we "smooth" out the data to remove the noise? Let's look at the following data smoothing techniques:

Sorted data for *price* (in dollars): 4, 8, 15, 21, 21, 24, 25, 28, 34

Partition into (equal-frequency) bins:

Bin 1: 4, 8, 15
Bin 2: 21, 21, 24
Bin 3: 25, 28, 34

Smoothing by bin means:

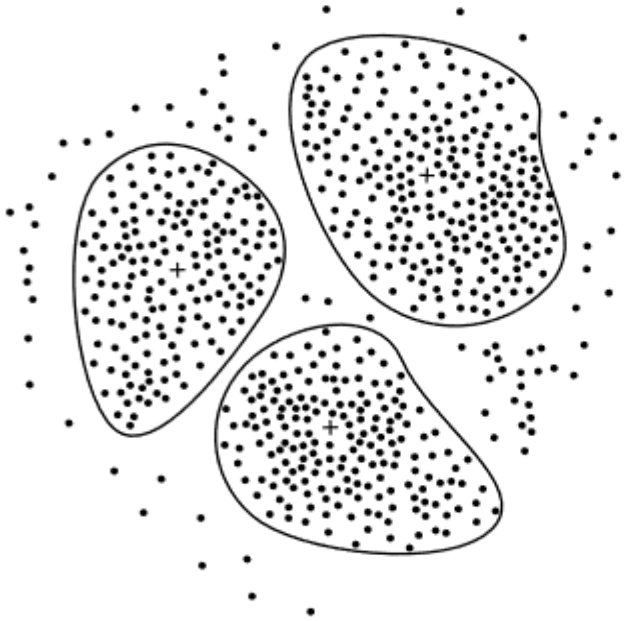
Bin 1: 9, 9, 9
Bin 2: 22, 22, 22
Bin 3: 29, 29, 29

Smoothing by bin boundaries:

Bin 1: 4, 4, 15
Bin 2: 21, 21, 24
Bin 3: 25, 25, 34

Figure 2.11 Binning methods for data smoothing.

1. **Binning:** Binning methods smooth a sorted data value by consulting its "neighborhood," that is, the values around it. The sorted values are distributed into a number of "buckets," or *bins*. Because binning methods consult the neighborhood of values, they perform *local* smoothing. Figure 2.11 illustrates some binning techniques. In this example, the data for *price* are first sorted and then partitioned into *equal-frequency* bins of size 3 (i.e., each bin contains three values). In **smoothing by bin means**, each value in a bin is replaced by the mean value of the bin. For example, the mean of the values 4, 8, and 15 in Bin 1 is 9. Therefore, each original value in this bin is replaced by the value 9. Similarly, **smoothing by bin medians** can be employed, in which each bin value is replaced by the bin median. In **smoothing by bin boundaries**, the minimum and maximum values in a given bin are identified as the *bin boundaries*. Each bin value is then replaced by the closest boundary value. In general, the larger the width, the greater the effect of the smoothing. Alternatively, bins may be *equal-width*, where the interval range of values in each bin is constant. Binning is also used as a discretization technique and is further discussed in Section 2.6.
2. **Regression:** Data can be smoothed by fitting the data to a function, such as with regression. *Linear regression* involves finding the "best" line to fit two attributes (or variables), so that one attribute can be used to predict the other. *Multiple linear regression* is an extension of linear regression, where more than two attributes are involved and the data are fit to a multidimensional surface. Regression is further described in Section 2.5.4, as well as in Chapter 6.



gure 2.12 A 2-D plot of customer data with respect to customer locations in a city, showing three data clusters. Each cluster centroid is marked with a "+", representing the average point in space for that cluster. Outliers may be detected as values that fall outside of the sets of clusters.

3. **Clustering:** Outliers may be detected by clustering, where similar values are organized into groups, or "clusters." Intuitively, values that fall outside of the set of clusters may be considered outliers (Figure 2.12). Chapter 7 is dedicated to the topic of clustering and outlier analysis.

16) Explain the different steps involved in data transformation. (4)

- **Smoothing**, which works to remove noise from the data. Such techniques include binning, regression, and clustering.
- **Aggregation**, where summary or aggregation operations are applied to the data. For example, the daily sales data may be aggregated so as to compute monthly and annual total amounts. This step is typically used in constructing a data cube for analysis of the data at multiple granularities.
- **Generalization** of the data, where low-level or "primitive" (raw) data are replaced by higher-level concepts through the use of concept hierarchies. For example, categorical

attributes, like *street*, can be generalized to higher-level concepts, like *city* or *country*. Similarly, values for numerical attributes, like *age*, may be mapped to higher-level concepts, like *youth*, *middle-aged*, and *senior*.

- **Normalization**, where the attribute data are scaled so as to fall within a small specified range, such as −1.0 to 1.0, or 0.0 to 1.0.
- **Attribute construction** (or *feature construction*), where new attributes are constructed and added from the given set of attributes to help the mining process.

17) What is data reduction? Explain the strategies for data reduction. (6)

Data reduction can reduce the data size by aggregating , eliminating redundant features, or clustering ,for instance. These techniques are not mutually exclusive ; they may work together.

Data reduction techniques can be applied to obtain a reduced representation of the data set that is much smaller in volume, yet closely maintains the integrity of the original data. That is, mining on the reduced data set should be more efficient yet produce the same (or almost the same) analytical results.

Strategies for data reduction include the following:

1. **Data cube aggregation**, where aggregation operations are applied to the data in the construction of a data cube.

   Imagine that you have collected the data for your analysis. These data consist of the *AllElectronics* sales per quarter, for the years 2002 to 2004. You are, however, interested in the annual sales (total per year), rather than the total per quarter. Thus the data can be *aggregated* so that the resulting data summarize the total sales per year instead of per quarter. This aggregation is illustrated in Figure 2.13. The resulting data set is smaller in volume, without loss of information necessary for the analysis task.

2. **Attribute subset selection**, where irrelevant, weakly relevant, or redundant attributes or dimensions may be detected and removed.

   Attribute subset selection[1] reduces the data set size by removing irrelevant or redundant attributes (or dimensions). The goal of attribute subset selection is to find a minimum set of attributes such that the resulting probability distribution of the data classes is as close as possible to the original distribution obtained using all attributes. Mining on a reduced set of attributes has an additional benefit. It reduces the number of attributes appearing in the discovered patterns, helping to make the patterns easier to understand.

3. **Dimensionality reduction**, where encoding mechanisms are used to reduce the data set size.

   In *dimensionality reduction*, data encoding or transformations are applied so as to obtain a reduced or "compressed" representation of the original data. If the original data can be *reconstructed* from the compressed data without any loss of information, the data reduction is called **lossless**. If, instead, we can reconstruct only an approximation of the original data, then the data reduction is called **lossy**. There are several well-tuned algorithms for string compression. Although they are typically lossless, they allow only limited manipulation of the data.

4. **Numerosity reduction**, where the data are replaced or estimated by alternative ,smaller data representations such as parametric models (which need store only the

[1] In machine learning, attribute subset selection is known as *feature subset selection*.

model parameters instead of the actual data) or nonparametric methods such as clustering, sampling, and the use of histograms.

5. **Discretization and concept hierarchy generation**, where raw data values for attributes are replaced by ranges or higher conceptual levels. Data discretization is a form of numerosity reduction that is very useful for the automatic generation of concept hierarchies. Discretization and concept hierarchy generation are powerful tools for data mining, in that they allow the mining of data at multiple levels of abstraction. We therefore defer the discussion of discretization and concept hierarchy generation to Section 2.6, which is devoted entirely to this topic.

18) Write a note on Wavelet Transform and Principal Component Analysis. (6)

Wavelet Transforms (DWT) is a linear signal processing technique that transforms a data vector XX into a vector $X0X0$ of wavelet coefficients. Though XX and $X0X0$ are of the same length, the usefulness of DWT lies in its ability to compress data by truncating wavelet coefficients. Only a small fraction of the strongest coefficients are retained, and the rest are set to zero, resulting in a sparse data representation. This sparsity facilitates faster computational operations and effective noise removal, making DWT a powerful tool for data reduction and cleaning.

- X: This is the original data vector. It is a sequence of numerical values representing the measurements or attributes of the data. For example, if you have a time-series data set, XX would be the vector containing the values at different time points: $X=(x1,x2,...,xn)X=(x1,x2,...,xn)$.

- $X0X0$: This is the transformed data vector, consisting of wavelet coefficients obtained after applying the Discrete Wavelet Transform (DWT) to XX. The length of $X0X0$ is the same as XX, but the values in $X0X0$ represent wavelet coefficients, which capture both the low-frequency (approximation) and high-frequency (detail) components of the original data.

Key Points:

Data Compression: DWT achieves better lossy compression compared to Discrete Fourier Transform (DFT). For the same number of retained coefficients, DWT provides a more accurate approximation of the original data.
Localized in Space: Wavelets are localized in space, preserving local detail and contributing to better data approximation.
Wavelet Families: Different DWTs like Haar, Daubechies-4, and Daubechies-6 are defined by their vanishing moments, which influence the number of coefficients.

Hierarchical Algorithm: DWT uses a hierarchical pyramid algorithm that applies smoothing and differencing to data pairs recursively until a minimal length is reached.
Orthogonal Matrix: The transformation matrix for DWT is orthonormal, allowing the inverse transformation to reconstruct the original data accurately.
Applications:

Multidimensional Data: DWT can be applied to data cubes by transforming each dimension sequentially.
Compression: Better than JPEG for certain applications.
Real-World Uses: Includes fingerprint image compression, computer vision, time-series analysis, and data cleaning.

Principal Components Analysis (PCA) is a method for dimensionality reduction that projects data onto a smaller set of orthogonal vectors (principal components) that best represent the data's variance. This results in a reduced-dimensionality dataset while preserving the most significant patterns and relationships.

Key Points:
Normalization: The input data are normalized to ensure that no single attribute dominates due to its scale.
Orthogonal Vectors: PCA computes orthonormal vectors that serve as the principal components. These vectors are perpendicular to each other and represent the directions of maximum variance.
Variance and Significance: Principal components are ordered by the amount of variance they explain. The first principal component accounts for the most variance, the second for the next most, and so on.
Dimensionality Reduction: By retaining only the most significant principal components (those with high variance), the size of the data can be effectively reduced.
Reconstruction: A good approximation of the original data can be reconstructed using the strongest principal components.
Applications:

Pattern Identification: PCA reveals hidden relationships and patterns within the data that might not be evident otherwise.
Handling Sparse and Skewed Data: PCA works well with both sparse and skewed data.
Multidimensional Data: The technique can reduce multidimensional data to two dimensions for easier analysis.
Integration with Other Techniques: Principal components can be used as inputs for regression and cluster analysis.

19) Explain any two numerosity reduction techniques. (4)

Histograms

Histograms use binning to approximate data distributions and are a popular form of data reduction. Histograms were introduced in Section 2.2.3. A **histogram** for

an attribute, *A*, partitions the data distribution of *A* into disjoint subsets, or *buckets*. If each bucket represents only a single attribute-value/frequency pair, the buckets are called *singleton buckets*. Often, buckets instead represent continuous ranges for the given attribute.

Example 2.5 Histograms. The following data are a list of prices of commonly sold items at *AllElectronics* (rounded to the nearest dollar). The numbers have been sorted: 1, 1, 5, 5, 5, 5, 5, 8, 8, 10, 10, 10, 10, 12, 14, 14, 14, 15, 15, 15, 15, 15, 15, 18, 18, 18, 18, 18, 18, 18, 18, 20, 20, 20, 20, 20, 20, 20, 21, 21, 21, 21, 25, 25, 25, 25, 25, 28, 28, 30, 30, 30.

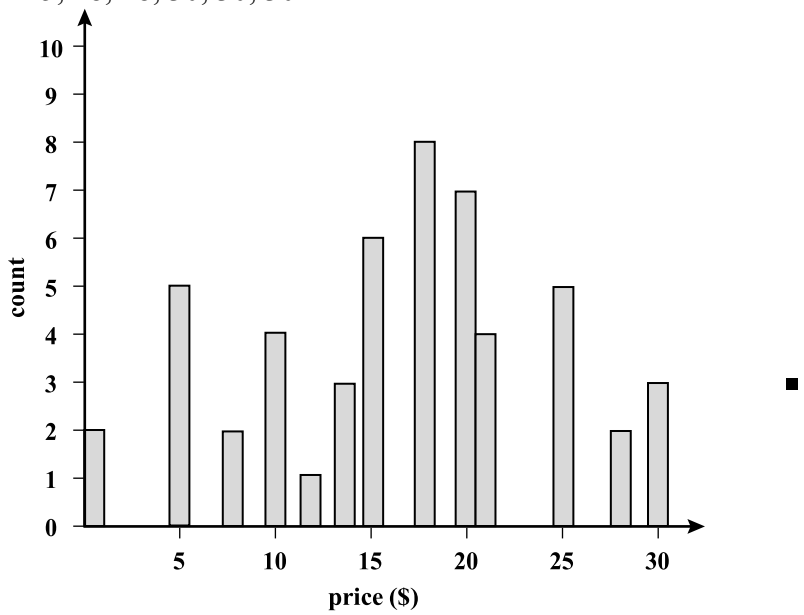


Figure 2.18 A histogram for *price* using singleton buckets—each bucket represents one price-value/ frequency pair.

Figure 2.18 shows a histogram for the data using singleton buckets. To further reduce the data, it is common to have each bucket denote a continuous range of values for the given attribute. In Figure 2.19, each bucket represents a different $10 range for *price*.

*"Howarethebucketsdeterminedandtheattributevaluespartitioned?"*Therearesseveral partitioning rules, including the following:

- **Equal-width**: In an equal-width histogram, the width of each bucket range is uniform (such as the width of $10 for the buckets in Figure 2.19).
- **Equal-frequency** (or equidepth): In an equal-frequency histogram, the buckets are created so that, roughly, the frequency of each bucket is constant (that is, each bucket contains roughly the same number of contiguous data samples).
- **V-Optimal**:Ifweconsiderallofthepossiblehistogramsforagivennumberofbuckets, the V-Optimal histogram is the one with the least variance. Histogram variance

is a weighted sum of the original values that each bucket represents, where bucket weight is equal to the number of values in the bucket.

- **MaxDiff**: In a MaxDiff histogram, we consider the difference between each pair of adjacent values. A bucket boundary is established between each pair for pairs having the $\beta-1$ largest differences, where β is the user-specified number of buckets.

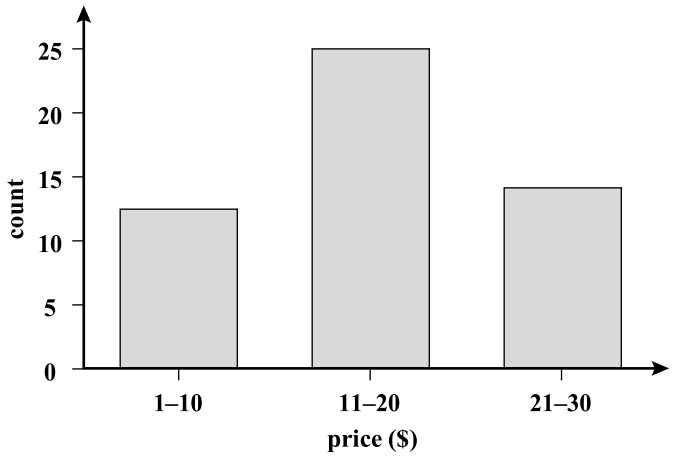


Figure 2.19 An equal-width histogram for *price*, where values are aggregated so that each bucket has a uniform width of $10.

V-Optimal and MaxDiff histograms tend to be the most accurate and practical. Histogramsarehighlyeffectiveatapproximatingbothsparseanddensedata,aswellashighly skewed and uniform data. The histograms described above for single attributes can be extended for multiple attributes. *Multidimensional histograms* can capture dependencies between attributes. Such histograms have been found effective in approximating data with up to five attributes. More studies are needed regarding the effectiveness of multidimensional histograms for very high dimensions. Singleton buckets are useful for storing outliers with high frequency.

Clustering

Clustering techniques consider data tuples as objects. They partition the objects into groups or *clusters*, so that objects within a cluster are "similar" to one another and "dissimilar" to objects in other clusters. Similarity is commonly defined in terms of how "close" the objects are in space, based on a distance function. The "quality" of a cluster may be represented by its *diameter*, the maximum distance between any two objects in the cluster. *Centroid distance* is an alternative measure of cluster quality and is defined as the average distance of each cluster object from the cluster centroid (denoting the "average object," or average point in space for the cluster). Figure 2.12 of Section 2.3.2 shows a 2-D plot of customer data with respect to customer locations in a city, where the centroid of each cluster is shown with a "+". Three data clusters are visible.

In data reduction, the cluster representations of the data are used to replace the actual data. The effectiveness of this technique depends on the nature of the data. It

is much more effective for data that can be organized into distinct clusters than for smeared data.

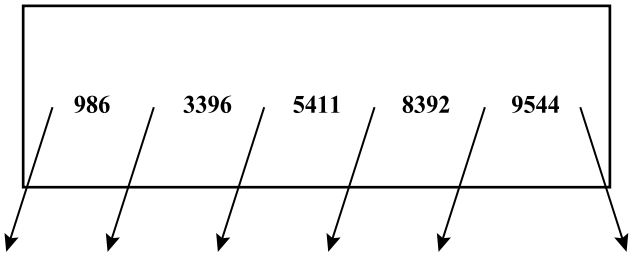


Figure 2.20 The root of a B+-tree for a given set of data.

In database systems, **multidimensional index trees** are primarily used for providing fast data access. They can also be used for hierarchical data reduction, providing a multiresolution clustering of the data. This can be used to provide approximate answers to queries. An index tree recursively partitions the multidimensional space for a given set of data objects, with the root node representing the entire space. Such trees are typically balanced, consisting of internal and leaf nodes. Each parent node contains keys and pointers to child nodes that, collectively, represent the space represented by the parent node. Each leaf node contains pointers to the data tuples they represent (or to the actual tuples). An index tree can therefore store aggregate and detail data at varying levels of resolution or abstraction. It provides a hierarchy of clusterings of the data set, where each cluster has a label that holds for the data contained in the cluster. If we consider each child of a parent node as a bucket, then an index tree can be considered as a *hierarchical histogram*. For example, consider the root of a B+-tree as shown in Figure 2.20, with pointers to the data keys 986, 3396, 5411, 8392, and 9544. Suppose that the tree contains 10,000 tuples with keys ranging from 1 to 9999. The data in the tree can be approximated by an equal-frequency histogram of six buckets for the key ranges 1 to 985, 986 to 3395, 3396 to 5410, 5411 to 8391, 8392 to 9543, and 9544 to 9999. Each bucket contains roughly 10,000/6 items. Similarly, each bucket is subdivided into smaller buckets, allowing for aggregate data at a finer-detailed level. The use of multidimensional index trees as a form of data reduction relies on an ordering of the attribute values in each dimension. Two-dimensional or multidimensional index trees include R-trees, quad-trees, and their variations. They are well suited for handling both sparse and skewed data.

20) Explain any three methods for data discretization. (6)

Binning

Binning is a top-down splitting technique based on a specified number of bins. Section 2.3.2 discussed binning methods for data smoothing. These methods are also used as discretization methods for numerosity reduction and concept hierarchy generation. For example, attribute values can be discretized by applying equal-width or equal-frequency binning, and then replacing each bin value by the bin mean or median, as in *smoothing by bin means* or *smoothing by bin medians*, respectively. These techniques can be applied recursively to the resulting partitions in order to generate concept hierarchies. Binning does not use class information and is therefore an unsupervised discretization

technique. It is sensitive to the user-specified number of bins, as well as the presence of outliers.

Cluster Analysis

Cluster analysis is a popular data discretization method. A clustering algorithm can be applied to discretize a numerical attribute, A, by partitioning the values of A into clusters or groups. Clustering takes the distribution of A into consideration, as well as the closeness of data points, and therefore is able to produce high-quality discretization results. Clustering can be used to generate a concept hierarchy for A by following either a topdown splitting strategy or a bottom-up merging strategy, where each cluster forms a node of the concept hierarchy. In the former, each initial cluster or partition may be further decomposed into several subclusters, forming a lower level of the hierarchy. In the latter, clusters are formed by repeatedly grouping neighboring clusters in order to form higher-level concepts

Entropy-Based Discretization

Entropy is one of the most commonly used discretization measures. It was first introduced by Claude Shannon in pioneering work on information theory and the concept of information gain. Entropy-based discretization is a supervised, top-down splitting technique. It explores class distribution information in its calculation and determination of split-points (data values for partitioning an attribute range). To discretize a numerical attribute, A, the method selects the value of A that has the minimum entropy as a split-point, and recursively partitions the resulting intervals to arrive at a hierarchical discretization. Such discretization forms a concept hierarchy for A.

Let D consist of data tuples defined by a set of attributes and a class-label attribute. The class-label attribute provides the class information per tuple. The basic method for entropy-based discretization of an attribute A within the set is as follows:

1. Each value of A can be considered as a potential interval boundary or split-point (denoted *split point*) to partition the range of A. That is, a split-point for A can partition the tuples in D into two subsets satisfying the conditions $A \leq split\_point$ and $A > split\_point$, respectively, thereby creating a binary discretization.
2. Entropy-based discretization, as mentioned above, uses information regarding the class label of tuples. To explain the intuition behind entropy-based discretization, we must take a glimpse at classification. Suppose we want to classify the tuples in D by partitioning on attribute A and some split-point. Ideally, we would like this partitioning to result in an exact classification of the tuples. For example, if we had two classes, we would hope that all of the tuples of, say, class C_1 will fall into one partition, and all of the tuples of class C_2 will fall into the other partition. However, this is unlikely. For example, the first partition may contain many tuples of C_1, but also some of C_2. How much more information would we still need for a perfect classification, after this partitioning? This amount is called the *expected information requirement* for classifying a tuple in D based on partitioning by A. It is given by

$$Info_A(D) = \frac{|D_1|}{|D|} Entropy(D^1) + \frac{|D_2|}{|D|} Entropy(D_2), \qquad (2.15)$$

where D_1 and D_2 correspond to the tuples in D satisfying the conditions $A \leq split\_point$ and $A > split\_point$, respectively; $|D|$ is the number of tuples in D, and so on. The entropy function for a given set is calculated based on the class distribution of the tuples in the set. For example, given m classes, $C_1, C_2, ..., C_m$, the entropy of D_1 is

$$Entropy(D_1) = -\sum_{i=1}^{m} p_i \log_2(p_i), \qquad (2.16)$$

where p_i is the probability of class C_i in D_1, determined by dividing the number of tuples of class C_i in D_1 by $|D_1|$, the total number of tuples in D_1. Therefore, when selecting a split-point for attribute A, we want to pick the attribute value that gives the minimumexpectedinformationrequirement(i.e.,min($Info_A(D)$)).Thiswouldresult in the minimum amount of expected information (still) required to perfectly classify thetuplesafterpartitioningby $A \leq split\_point$ and$A > split_point$.Thisisequivalent to the attribute-value pair with the maximum information gain (the further details of which are given in Chapter 6 on classification.) Note that the value of $Entropy(D_2)$ can be computed similarly as in Equation (2.16).

"But our task is discretization, not classification!", you may exclaim. This is true. We use the split-point to partition the range of A into two intervals, corresponding to $A \leq split\_point$ and $A > split\_point$.

3. The process of determining a split-point is recursively applied to each partition obtained, until some stopping criterion is met, such as when the minimum information requirement on all candidate split-points is less than a small threshold, ε, or when the number of intervals is greater than a threshold, *max interval*.

Entropy-based discretization can reduce data size. Unlike the other methods mentioned here so far, entropy-based discretization uses class information. This makes it more likely that the interval boundaries (split-points) are defined to occur in places that may help improve classification accuracy. The entropy and information gain measures described here are also used for decision tree induction

21) Write a note on different methods used for the generation of concept hierarchies forcategorical data. (4/6)

Categorical data are discrete data. Categorical attributes have a finite (but possibly large) number of distinct values, with no ordering among the values. Examples include *geographic location*, *job category*, and *item type*. There are several methods for the generation of concept hierarchies for categorical data.

Specification of a partial ordering of attributes explicitly at the schema level by users or experts: Concept hierarchies for categorical attributes or dimensions typically involve a group of attributes. A user or expert can easily define a concept hierarchy by specifying a partial or total ordering of the attributes at the schema level. For example, a relational database or a dimension *location* of a data warehouse may contain the following group of attributes: *street, city, province_or_state*, and *country*. A hierarchy can be defined by specifying the total ordering among these attributes at the schema level, such as $street < city < province\_or\_state < country$.

Specification of a portion of a hierarchy by explicit data grouping: This is essentially the manual definition of a portion of a concept hierarchy. In a large database, it

is unrealistic to define an entire concept hierarchy by explicit value enumeration. On the contrary, we can easily specify explicit groupings for a small portion of intermediate-level data. For example, after specifying that *province* and *country* form a hierarchy at the schema level, a user could define some intermediate levels manually, such as "{*Alberta, Saskatchewan, Manitoba*} $\subset$ *prairies_Canada*" and "{*British Columbia, prairies_Canada*} $\subset$ *Western_Canada*".

Specification of a *set of attributes*, but not of their partial ordering: A user may specify a set of attributes forming a concept hierarchy, but omit to explicitly state their partial ordering. The system can then try to automatically generate the attribute ordering so as to construct a meaningful concept hierarchy. "*Without knowledge of data semantics, how can a hierarchical ordering for an arbitrary set of categorical attributes be found?*" Consider the following observation that since higher-level concepts generally cover several subordinate lower-level concepts, an attribute defining a high concept level (e.g., *country*) will usually contain a smaller number of distinct values than an attribute defining a lower concept level (e.g., *street*). Based on this observation, a concept hierarchy can be automatically generated based on the number of distinct values per attribute in the given attribute set. The attribute with the most distinct values is placed at the lowest level of the hierarchy. The lower the number of distinct values an attribute has, the higher it is in the generated concept hierarchy. This heuristic rule works well in many cases. Some local-level swapping or adjustments may be applied by users or experts, when necessary, after examination of the generated hierarchy.

Let's examine an example of this method.

Example 2.7 **Concept hierarchy generation based on the number of distinct values per attribute.** Suppose a user selects a set of location-oriented attributes, *street, country, province_or_state*, and *city*, from the *AllElectronics* database, but does not specify the hierarchical ordering among the attributes.

A concept hierarchy for *location* can be generated automatically, as illustrated in Figure 2.24. First, sort the attributes in ascending order based on the number of distinct values in each attribute. This results in the following (where the number of distinct values per attribute is shown in parentheses): *country* (15), *province_or_state* (365), *city* (3567), and *street* (674,339). Second, generate the hierarchy from the top down according to the sorted order, with the first attribute at the top level and the last attribute at the bottom level. Finally, the user can examine the generated hierarchy, and when necessary, modify it to reflect desired semantic relationships among the attributes. In this example, it is obvious that there is no need to modify the generated hierarchy. ■

Note that this heuristic rule is not foolproof. For example, a time dimension in a database may contain 20 distinct years, 12 distinct months, and 7 distinct days of the week. However, this does not suggest that the time hierarchy should be "*year* $<$ *month* $<$ *days_of_the_week*", with *days_of_the_week* at the top of the hierarchy.

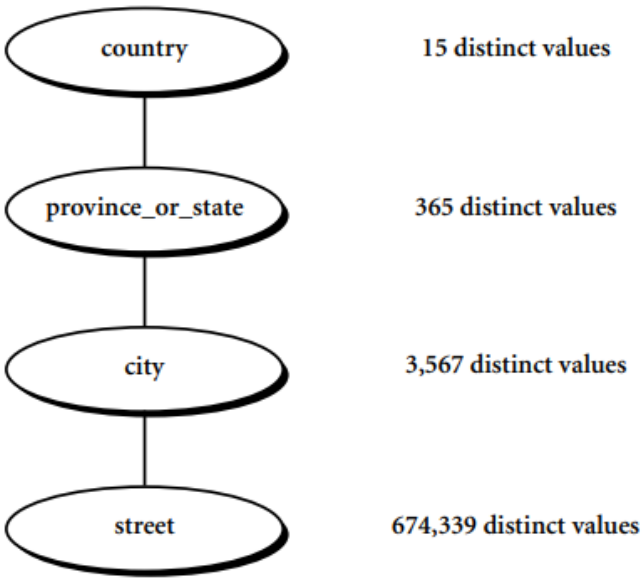


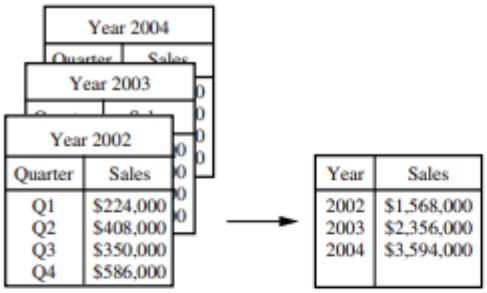
Figure 2.24 Automatic generation of a schema concept hierarchy based on the number of distinct attribute values.

Specification of only a partial set of attributes: Sometimes a user can be sloppy when defining a hierarchy, or have only a vague idea about what should be included in a hierarchy. Consequently, the user may have included only a small subset of the relevant attributes in the hierarchy specification. For example, instead of including all of the hierarchically relevant attributes for *location*, the user may have specified only *street* and *city*. To handle such partially specified hierarchies, it is important to embed data semantics in the database schema so that attributes with tight semantic connections can be pinned together. In this way, the specification of one attribute may trigger a whole group of semantically tightly linked attributes to be "dragged in" to form a complete hierarchy. Users, however, should have the option to override this feature, as necessary.

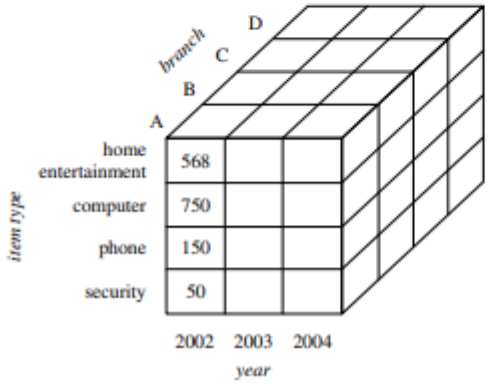
22) Explain Data cube aggregation? (4)

Imagine that you have collected the data for your analysis. These data consist of the *AllElectronics* sales per quarter, for the years 2002 to 2004. You are, however, interested in the annual sales (total per year), rather than the total per quarter. Thus the data can be *aggregated* so that the resulting data summarize the total sales per year instead of per quarter. This aggregation is illustrated in Figure 2.13. The resulting data set is smaller in volume, without loss of information necessary for the analysis task.

Data cubes are discussed in detail in Chapter 3 on data warehousing. We briefly introduce some concepts here. Data cubes store multidimensional aggregated information. For example, Figure 2.14 shows a data cube for multidimensional analysis of sales data with respect to annual sales per item type for each *AllElectronics* branch. Each cell holds an aggregate data value, corresponding to the data point in multidimensional space. (For readability, only some cell values are shown.) Concept



igure 2.13 Sales data for a given branch of *AllElectronics* for the years 2002 to 2004. On the left, the sales are shown per quarter. On the right, the data are aggregated to provide the annual sales.



igure 2.14 A data cube for sales at *AllElectronics*.

hierarchies may exist for each attribute, allowing the analysis of data at multiple levels of abstraction. For example, a hierarchy for *branch* could allow branches to be grouped into regions, based on their address. Data cubes provide fast access to precomputed, summarized data, thereby benefiting on-line analytical processing as well as data mining.

The cube created at the lowest level of abstraction is referred to as the *base cuboid*. The base cuboid should correspond to an individual entity of interest, such as *sales* or *customer*. In other words, the lowest level should be usable, or useful for the analysis. A cube at the highest level of abstraction is the *apex cuboid*. For the sales data of Figure 2.14, the apex cuboid would give one total—the total *sales*

for all three years, for all item types, and for all branches. Data cubes created fo varying levels of abstraction are often referred to as *cuboids*, so that a data cube may instead refer to a *lattice of cuboids*. Each higher level of abstraction further reduces the resulting data size. When replying to data mining requests, the *smallest* available cuboid relevant to the given task should be used. This issue is also addressed in Chapter 3.

23) Explain heuristic methods of attribute subset selection with an example and explain itstechniques. (6)

Data sets for analysis may contain hundreds of attributes, many of which may be irrelevant to the mining task or redundant. For example, if the task is to classify customers as to whether or not they are likely to purchase a popular new CD at *AllElectronics* when notified of a sale, attributes such as the customer's telephone number are likely to be irrelevant, unlike attributes such as *age* or *music_taste*. Although it may be possible for a domain expert to pick out some of the useful attributes, this can be a difficult and time-consuming task, especially when the behavior of the data is not well known (hence, a reason behind its analysis!). Leaving out relevant attributes or keeping irrelevant attributes may be detrimental, causing confusion for the mining algorithm employed. This can result in discovered patterns of poor quality. In addition, the added volume of irrelevant or redundant attributes can slow down the mining process.

Attribute subset selection[6] reduces the data set size by removing irrelevant or redundant attributes (or dimensions). The goal of attribute subset selection is to find a minimum set of attributes such that the resulting probability distribution of the data classes is as close as possible to the original distribution obtained using all attributes. Mining on a reduced set of attributes has an additional benefit. It reduces the number of attributes appearing in the discovered patterns, helping to make the patterns easier to understand.

"*How can we find a 'good' subset of the original attributes?*" For n attributes, there are 2^n possible subsets. An exhaustive search for the optimal subset of attributes can be prohibitively expensive, especially as n and the number of data classes increase. Therefore, heuristic methods that explore a reduced search space are commonly used for attribute subset selection. These methods are typically **greedy** in that, while searching through attribute space, they always make what looks to be the best choice at the time. Their strategy is to make a locally optimal choice in the hope that this will lead to a globally optimal solution. Such greedy methods are effective in practice and may come close to estimating an optimal solution.

The "best" (and "worst") attributes are typically determined using tests of statistical significance, which assume that the attributes are independent of one another. Many

[6]In machine learning, attribute subset selection is known as *feature subset selection*.

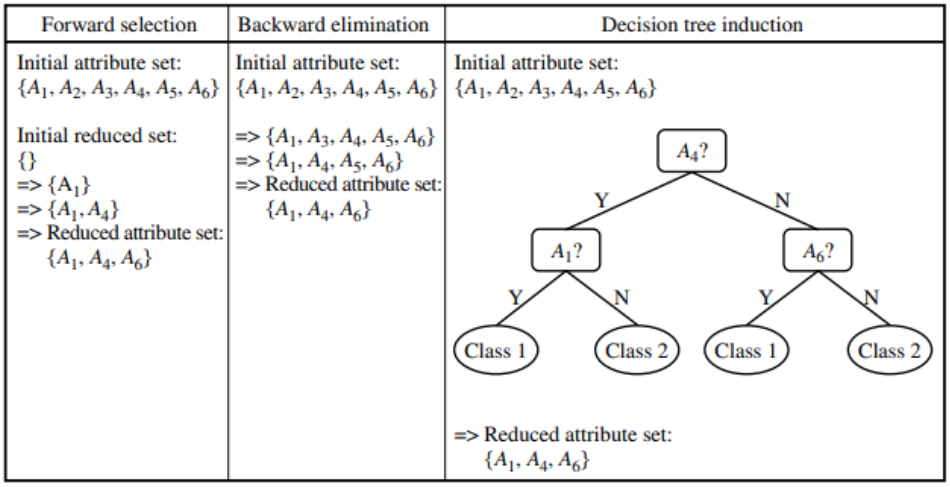


Figure 2.15 Greedy (heuristic) methods for attribute subset selection.

other attribute evaluation measures can be used, such as the *information gain* measure used in building decision trees for classification.[7]

Basic heuristic methods of attribute subset selection include the following techniques, some of which are illustrated in Figure 2.15.

1. **Stepwise forward selection**: The procedure starts with an empty set of attributes as the reduced set. The best of the original attributes is determined and added to the reduced set. At each subsequent iteration or step, the best of the remaining original attributes is added to the set.
2. **Stepwise backward elimination**: The procedure starts with the full set of attributes. At each step, it removes the worst attribute remaining in the set.
3. **Combination of forward selection and backward elimination**: The stepwise forward selection and backward elimination methods can be combined so that, at each step, the procedure selects the best attribute and removes the worst from among the remaining attributes.
4. **Decision tree induction**: Decision tree algorithms, such as ID3, C4.5, and CART, were originally intended for classification. Decision tree induction constructs a flowchart-like structure where each internal (nonleaf) node denotes a test on an attribute, each branch corresponds to an outcome of the test, and each external (leaf) node denotes a

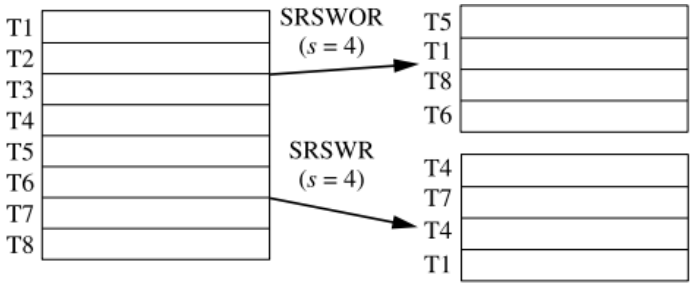
class prediction. At each node, the algorithm chooses the "best" attribute to partition the data into individual classes.

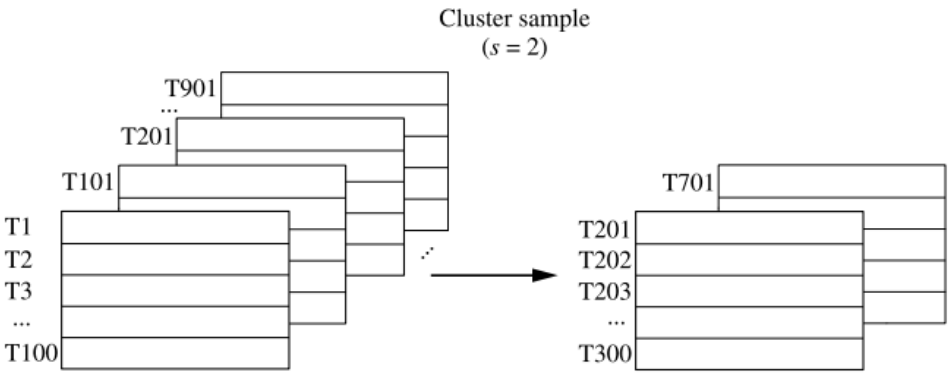
When decision tree induction is used for attribute subset selection, a tree is constructed from the given data. All attributes that do not appear in the tree are assumed to be irrelevant. The set of attributes appearing in the tree form the reduced subset of attributes.

The stopping criteria for the methods may vary. The procedure may employ a threshold on the measure used to determine when to stop the attribute selection process.

24) What is sampling?Explain the ways used to sample for data reduction?

Sampling can be used as a data reduction technique because it allows a large data set to be represented by a much smaller random sample (or subset) of the data. Suppose that a large data set, D, contains N tuples. Let's look at the most common ways that we could sample D for data reduction, as illustrated in Figure 2.21.





Stratified sample (according to *age*)

T38	youth
T256	youth
T307	youth
T391	youth
T96	middle_aged
T117	middle_aged
T138	middle_aged
T263	middle_aged
T290	middle_aged
T308	middle_aged
T326	middle_aged
T387	middle_aged
T69	senior
T284	senior

T38	youth
T391	youth
T117	middle_aged
T138	middle_aged
T290	middle_aged
T326	middle_aged
T69	senior

- **Simple random sample without replacement (SRSWOR) of size** s: This is created by drawing s of the N tuples from D ($s < N$), where the probability of drawing any tuple in D is $1/N$, that is, all tuples are equally likely to be sampled.
- **Simple random sample with replacement (SRSWR) of size** s: This is similar to SRSWOR, except that each time a tuple is drawn from D, it is recorded and then *replaced*. That is, after a tuple is drawn, it is placed back in D so that it may be drawn again.
- **Cluster sample**: If the tuples in D are grouped into M mutually disjoint "clusters," then an SRS of s clusters can be obtained, where $s < M$. For example, tuples in a database are usually retrieved a page at a time, so that each page can be considered a cluster. A reduced data representation can be obtained by applying, say, SRSWOR to the pages, resulting in a cluster sample of the tuples. Other clustering criteria conveying rich semantics can also be explored. For example, in a spatial database, we may choose to define clusters geographically based on how closely different areas are located.
- **Stratified sample**: If D is divided into mutually disjoint parts called *strata*, a stratified sample of D is generated by obtaining an SRS at each stratum. This helps ensure a representative sample, especially when the data are skewed. For example, a stratified sample may be obtained from customer data, where a stratum is created for each customer age group. In this way, the age group having the smallest number of customers will be sure to be represented.

An advantage of sampling for data reduction is that the cost of obtaining a sample *is proportional to the size of the sample*, s, as opposed to N, the data set size. Hence, sampling complexity is potentially *sublinear* to the size of the data. Other data reduction techniques can require at least one complete pass through D. For a fixed sample size, sampling complexity increases only linearly as the number of data dimensions, n, increases, whereas techniques using histograms, for example, increase exponentially in n.

When applied to data reduction, sampling is most commonly used to estimate the answer to an aggregate query. It is possible (using the central limit theorem) to determine a sufficient sample size for estimating a given function within a specified degree of error. This sample size, s, may be extremely small in comparison to N. Sampling is a natural choice for the progressive refinement of a reduced data set. Such a set can be further refined by simply increasing the sample size.

Unit-3

2 Marks :

1) What are frequent patterns? What is the use of frequent pattern mining?

Frequent pattern refer to sets of items,subsequence,or substructures that appear frequently together in a dataset.

Use of frequent pattern mining: Identifying products that are often purchased together to optimize inventory, improve sales strategies and design better promotion campaigns.

2) What is market basket analysis?

Market basket analysis is a data mining technique used by retailers to increase sales by better understanding customer purchasing patterns. It involves analyzing large data sets, such as purchase history, to reveal product groupings, as well as products that are likely to be purchased together.

3) What is the meaning of the following association rule:

Computer ⇒ Antivirus_Software (Support=2%, Confidence=60%)

The given association rule can be interpreted as follows:

Computer → Antivirus_Software: This indicates that the purchase or presence of a computer is associated with the purchase or presence of antivirus software.

Support = 2%: This means that 2% of all transactions in the dataset include both a computer and antivirus software. Support indicates how often the items in the rule occur together in the dataset.

Confidence = 60%: This means that among all the transactions that include a computer, 60% also include antivirus software. Confidence measures the strength of the implication, showing how often the rule is true.

In summary, this rule suggests that computers and antivirus software are purchased together in 2% of all transactions, and when a computer is purchased, there is a 60% chance that antivirus software is also purchased.

4) What is relative support and absolute support?

Relative support is the proportion or percentage of transactions in the dataset that contain a particular itemset.

Absolute support refers to the raw count of transactions in the dataset that contain a particular itemset.

5) What is frequent itemset?

A frequent itemset refers to a set of items that frequently appear together in a transactional dataset.

6) What is closed itemset?

A closed itemset is a type of frequent itemset with a specific property,it has no immediate superset that has the same support count.

7) Give the two step process of association rule mining.

1. Support: It means how often X and Y occur together as a percentage of the total trnsactions.
2. Confidence: It measures how much a particular item is dependent on another.

8) What is the purpose of sequential pattern mining and structured pattern mining.

1. Sequential pattern: A frequently occurring subsequence, such as the pattern that customers tend to purchase first a PC, followed by a digital camera, and then a memory card, is a (*frequent*) Sequential pattern.

2. Structured pattern: A substructure can refer to different structural forms, such as graphs, trees, or lattices, which may be combined with itemsets and subsequences. If a substructure occurs frequently, it is called a (*frequent*) Structured pattern.

9) What is the purpose of Apriori property? Mention it.

Apriori property: All nonempty subsets of a frequent itemset must also be frequent.

The Apriori property is based on the following observation. If an itemset *I*does notsatisfy the minimum support therehold, *min_sup*, then *I*is not frequent; that is, $P(I) < min\_sup$. If an item A is added to the itemset *I*, then the resulting itemset(i.e. *I*UA) cannot occur more frequently than *I* . Therefore, *I*UA is not frequent either; that is, $P(I\text{UA}) < min\_sup$.

This property belongs to a special category of properties called antimonotone in the sense that if a set cannot pass a test, all of of its subsets will fail the same test as well. It is called antimonotone because, the property is monotonic in the cintext of failing a test.

10) Mention the two step process followed in Apriori algorithm.

1. **Candidate Generation (Join Step):**

   In this initial step, the algorithm generates candidate itemsets of size kk (denoted as $CkCk$) by joining frequent itemsets of size $k-1k-1$ (denoted as $Lk-1Lk-1$). This is achieved by combining itemsets from $Lk-1Lk-1$ that share $k-2k-2$ items in common, thereby forming larger itemsets. For instance, to generate candidate itemsets of size 3, the algorithm will join pairs of frequent itemsets of size 2 that have two items in common.

2. **Frequent Itemset Generation (Prune Step):**

   After generating the candidate itemsets $CkCk$, the algorithm scans the transaction database to count the occurrences of each candidate. It then compares these counts against a user-defined minimum support threshold. Itemsets that meet or exceed this threshold are considered frequent and form the set $LkLk$. The algorithm then prunes candidate itemsets that do not meet the minimum support, as well as those that contain subsets not present in $Lk-1Lk-1$, leveraging the Apriori principle (which states that all non-empty subsets of a frequent itemset must also be frequent).

11) List any 2 techniques used to improve the efficiency of Apriori based mining.

1. **Hash-Based Technique:**

   - **Hashing Itemsets:** During the candidate generation process, a hash-based technique can be used to reduce the number of candidateitemsets. By hashing item pairs into buckets and counting the bucket frequencies, infrequent buckets can be eliminated early on, thus reducing the number of candidate 2-itemsets. This technique helps in quickly identifying and pruning non-promising candidates, thereby improving the efficiency of the algorithm.

2. **Transaction Reduction:**

   - **Reducing the Number of Transactions:** After each iteration of generating frequent itemsets, transactions that do not contain any frequent itemsets can be

removed from subsequent scans. Since these transactions do not contribute to finding larger frequent itemsets, removing them reduces the size of the dataset that needs to be scanned in the next iteration. This reduces the computational overhead and speeds up the mining process.

12) What is the strategy of frequent pattern growth?

The strategy of Frequent Pattern Growth (FP-Growth) in the context of Fundamentals of Data Science involves a more efficient approach to finding frequent itemsets compared to the Apriori algorithm. FP-Growth eliminates the need for candidate generation and repeatedly scanning the entire database.

13) What is horizontal data format and vertical data format?

Both the Apriori and FP-growth methods mine frequent patterns from a set of transactions in TID-itemset format (that is, {T ID : itemset}), where TID is a transaction-id and itemset is the set of items bought in transaction TID. This data format is known as **horizontal data format.**

Alternatively, data can also be presented in item-TID set format (that is, {item : T ID set}), where item is an item name, and TID set is the set of transaction identifiers containing the item. This format is known as **vertical data format**

14) Expand ECLAT and ARCS.

ECLAT -Equivalence CLASS Transformation.

ARCS - Association Rule Clustering System.

15) Name any two pruning strategies in mining closed frequent itemset.

- Item merging
- Sub-itemset pruning
- Item skipping:

16) Write the difference between multilevel association rule and multidimensional association rule?

Aspect	**Multilevel Association Rule**	**Multidimensional Association Rule**
Definition	Involves rules derived from data at different levels of abstraction or hierarchy.	Involves rules that consider multiple dimensions or attributes of the data.
Example	"If a customer buys electronics, they are likely to buy a computer" (different levels: electronics and computer).	"If a young customer buys a laptop, they are likely to buy a gaming mouse" (dimensions: age and product).
Hierarchical Relationships	Focuses on hierarchical relationships within a single dimension (e.g., item categories and subcategories).	Focuses on relationships across different dimensions or attributes (e.g., time, location, customer type).
Application	Useful for understanding detailed patterns within a	Useful for analyzing complex interactions between different attributes in the dataset.

	hierarchical structure of a single attribute.	
Focus	Hierarchical relationships within a single dimension (e.g., product hierarchy).	Relationships across multiple attributes or dimensions (e.g., demographic and transactional data).

17) What are categorical attributes and quantitative attributes?

Categorical attributes:

- Finite number of values with no ordering among the values.
- E.g. Name, Branch, Section, Dept etc.

Quantitative attributes:

- These are numeric and have implicit ordering among the values.
- E.g.: Rank, Mark, Age etc.

18) Name any two common binning strategies with respect to mining quantitative association rule?

Equal-width binning:

- Interval size of each bin is the same.

Equal frequency binning:

- Each bin has approximately the same number of tuples assigned to it.

Clustering base binning:

- Clustering is performed to group the neighbouring points into the same bin.

19) Name any two constraints included in constraint-based mining.

Constraints included in constraint-based mining are:

1. Knowledge type constraints: Specifies the type of knowledge (correlation/association) to be mined.
2. Data constraints: Specifies the set of task relevant data.
3. Dimension/level constraints: These specifies the desired dimensions of the data or level of concept hierarchies to be used in mining.
4. Interestingness constraint: These specifies the thresholds on statistical measures of rule interestingness (support, confidence, lift, chi-square, cosineetc.)
5. Rule constraints: These specifies the form of rules to be mined.

20) How are metarules useful?

1. Meta rules allows the users to specify the syntactic form of rules that they are interested in mining.

2. A metarule forms a hypothesis regarding the relationships that the user is interested in probing.
3. The datamining technique can then search the rule that match the given metarule.
4. These rule forms can be used as constraints to help improve the efficiency of mining process.
5. Metarules are based on the analyst experience, expectation or intuition regarding the data or may be automatically generated based on the database schema.

21) Define frequent set and border set.

Frequent Sets

A frequent set (or frequent itemset) is a set of items that appear together in a dataset with a frequency greater than or equal to a user-specified threshold called the minimum support threshold.

Border Sets

The concept of border sets is related to the efficient computation and pruning of candidate itemsets in frequent itemset mining.

22) What is FP growth?

FP growth iswhich adopts a divide-and-conquer strategy as follows. First, it compresses the database representing frequent items into a frequent-pattern tree, or FP-tree, which retains the itemset association information. It then divides the compressed database into a set of conditional databases, each associated with one frequent item or "pattern fragment" and mines each such database separately.

23) Define Support count or Frequency.

The occurrence frequency of an itemset is the number of transactions that contain the itemset. This is also known as the frequency or support count or count of the itemset.

24) Define Maximal Frequent itemset.

An itemset X is a maximal frequent itemset (or max-itemset) in set S if X is frequent, and there exists no super-itemset Y such that $X \subset Y$ and Y is frequent in S.

Long answer questions:

1)Explain Market Basket Analysis with an Example. (4/6)

Frequent itemset mining leads to the discovery of associations and correlations among items in large transactional or relational data sets. With massive amounts of data continuously being collected and stored, many industries are becoming interested in mining such patterns from their databases. The discovery of interesting correlation relationships among huge amounts of business transaction records can help in many business decision-making processes, such as catalog design, cross-marketing, and customer shopping behavior analysis. A typical example of frequent itemset mining is market basket analysis. This process analyzes customer buying habits by finding associations between the different items that customers place in their "shopping baskets". The discovery of such associations can help retailers develop marketing strategies by gaining insight into which items are frequently purchased together by customers. For instance, if customers are buying milk, how likely are they to also

buy bread (and what kind of bread) on the same trip to the supermarket? Such information can lead to increased sales by helping retailers do selective marketing and plan their shelf space.

2) Write a note on frequent item set and closed item set. (4)

A set of items is referred to as an itemset. An itemset that contains k items is a k-itemset. The set {computer, antivirus software} is a 2-itemset. The occurrence frequency of an itemset is the number of transactions that contain the itemset. This is also known, simply, as the frequency, support count, or count of the itemset. Note that the itemset support defined in Equation (5.2) is sometimes referred to as relative support, whereas the occurrence frequency is called the absolute support. If the relative support of an itemset I satisfies a prespecified minimum support threshold (i.e., the absolute support of I satisfies the corresponding minimum support count threshold), then I is a frequent itemset.3 The set of frequent k-itemset is commonly denoted by Lk.

The set of closed frequent itemsets contains complete information regarding the frequent itemsets. For example, from C, we can derive, say, (1) {a2, a45 : 2} since {a2, a45} is a sub-itemset of the itemset {a1, a2,..., a50 : 2}; and (2) {a8, a55 : 1} since {a8, a55} is not a sub-itemset of the previous itemset but of the itemset {a1, a2,..., a100 : 1}.

3) Explain the classification of Frequent pattern Minning. (6)

Based on the completeness of patterns to be mined: As we discussed in the previous subsection, we can mine the complete set of frequent itemsets , the closed frequent itemset, and the maximal frequent itemsets, given a minimum support threshold. We can also mine constrained frequent itemsets (i.e., those that satisfy a set of user-defined constraints), approximate frequent itemsets(i.e., those that derive only approximate support counts for the mined frequent itemsets), near-match frequent itemsets (i.e., those that tally the support count of the near or almost matching itemsets), top-k frequent itemsets (i.e., the k most frequent itemsets for a user-specified value, k), and so on.

Based on the levels of abstraction involved in the rule set: Some methods for association rule mining can find rules at differing levels of abstraction. For example, suppose that a set of association rules mined includes the following rules where X is a variable representing a customer:

buys(X, “computer”) ⇒ buys(X, “HP printer”)

4) Explain the two-step process followed in Apriori algorithm.

- The join step:
  - To find Lk, a set of candidate k-itemsets is generated by joining Lk−1 with itself.
  - This set of candidates is denoted Ck.
  - Let l1 and l2 be itemsets in Lk−1.
  - The notation li [j] refers to the jth item in li (e.g., l1[k−2] refers to the second to the last item in l1).
  - By convention, Apriori assumes that items within a transaction or itemset are sorted in lexicographic order.
  - For the (k −1)-itemset, li , this means that the items are sorted such that li [1] < li [2] < ... < li [k − 1].
  - The join, Lk−1 on Lk−1, is performed, where members of Lk−1 are joinable if their first (k − 2) items are in common.

- That is, members l1 and l2 of Lk−1 are joined if (l1[1] = l2[1]) ∧ (l1[2] = l2[2]) ∧...∧(l1[k−2] = l2[k−2]) ∧(l1[k−1] < l2[k−1]).
- The condition l1[k−1] < l2[k−1]simply ensures that no duplicates are generated.
- The resulting itemset formed by joining l1 and l2 is l1[1], l1[2],..., l1[k −2], l1[k −1], l2[k −1]. 2.

- The prune step:
  - Ck is a superset of Lk, that is, its members may or may not be frequent, but all of the frequent k-itemsets are included in Ck.
  - A scan of the database to determine the count of each candidate in Ck would result in the determination of Lk (i.e., all candidates having a count no less than the minimum support count are frequent by definition, and therefore belong to Lk).
  - Ck however can be huge, and so this could involve heavy computation.
  - To reduce the size of Ck, the Apriori property is used as follows.
    - Any (k − 1)-itemset that is not frequent cannot be a subset of a frequent k-itemset.
    - Hence, if any (k − 1)-subset of a candidate k-itemset is not in Lk−1, then the candidate cannot be frequent either and so can be removed from Ck.
    - This subset testing can be done quickly by maintaining a hash tree of all frequent itemsets.

5) Explain Apriori algorithm.

- Apriori is a seminal algorithm proposed by R. Agrawal and R. Srikant in 1994 for mining frequent itemsets for Boolean association rules.
- The name of the algorithm is based on the fact that the algorithm uses prior knowledge of frequent itemset properties.
- Apriori employs an iterative approach known as a level-wise search, where k-itemsets are used to explore (k+1)-itemsets.
- First, the set of frequent 1-itemsets is found by scanning the database to accumulate the count for each item, and collecting those items that satisfy minimum support.
- The resulting set is denoted L1.
- Next, L1 is used to find L2, the set of frequent 2-itemsets, which is used to find L3, and so on, until no more frequent k-itemsets can be found.
- The finding of each Lk requires one full scan of the database.
- To improve the efficiency of the level-wise generation of frequent itemsets, an important property called the Apriori property is used to reduce the search space.
- Apriori property: All nonempty subsets of a frequent itemset must also be frequent.
  - By definition, if an itemset I does not satisfy the minimum support threshold, min sup, then I is not frequent; that is, P(I) < min sup.
  - If an item A is added to the itemset I, then the resulting itemset (i.e., I ∪A) cannot occur more frequently than I.
  - Therefore, I ∪A is not frequent either; that is, P(I ∪A) < min sup.

- This property belongs to a special category of properties called antimonotone in the sense that if a set cannot pass a test, all of its supersets will fail the same test as well.
- It is called antimonotone because the property is monotonic in the context of failing a test.

6) Write a note on generating association rules from frequent itemsets.

- Once the frequent itemsets from transactions in a database D have been found, it is straightforward to generate strong association rules from them (where strong association rules satisfy both minimum support and minimum confidence).
- This can be done using Equation for confidence, which we show again here for completeness:

$$\text{confidence}(A \Rightarrow B) = P(B|A) = \frac{\text{support count}(A \cup B)}{\text{support count}(A)}$$

- The conditional probability is expressed in terms of itemset support count, where support count(A∪B) is the number of transactions containing the itemsets A∪B, and support count(A) is the number of transactions containing the itemset A.
- Based on this equation, association rules can be generated as follows: For each frequent itemset l, generate all nonempty subsets of l.
- For every nonempty subset s of l, output the rule "s ⇒ (l − s)" if $\frac{\text{support count}(l)}{\text{support count}(s)} \geq$ min_conf, where min_conf is the minimum confidence threshold.
- Because the rules are generated from frequent itemsets, each one automatically satisfies minimum support.
- Frequent itemsets can be stored ahead of time in hash tables along with their counts so that they can be accessed quickly.

7) Explain any two variations to improve the efficiency of Apriori based mining. (4)

- **Transaction reduction** (reducing the number of transactions scanned in future iterations): A transaction that does not contain any frequent k-itemsets cannot contain any frequent (k + 1)-itemsets. Therefore, such a transaction can be marked or removed from further consideration because subsequent scans of the database for j-itemsets, where j > k, will not require it.
- **Dynamic itemset counting** (adding candidate itemsets at different points during a scan): A dynamic itemset counting technique was proposed in which the database is partitioned into blocks marked by start points. In this variation, new candidate itemsets can be added at any start point, unlike in Apriori, which determines new candidate itemsets only immediately before each complete database scan. The technique is dynamic in that it estimates the support of all of the itemsets that have been counted so far, adding new candidate itemsets if all of their subsets are estimated to be frequent. The resulting algorithm requires fewer database scans than Apriori.

8) Explain any three variations to improve the efficiency of Apriori based mining. (6)

- **Transaction reduction** (reducing the number of transactions scanned in future iterations): A transaction that does not contain any frequent k-itemsets cannot contain

any frequent (k + 1)-itemsets. Therefore, such a transaction can be marked or removed from further consideration because subsequent scans of the database for j-itemsets, where j > k, will not require it.

- **Dynamic itemset counting** (adding candidate itemsets at different points during a scan): A dynamic itemset counting technique was proposed in which the database is partitioned into blocks marked by start points. In this variation, new candidate itemsets can be added at any start point, unlike in Apriori, which determines new candidate itemsets only immediately before each complete database scan. The technique is dynamic in that it estimates the support of all of the itemsets that have been counted so far, adding new candidate itemsets if all of their subsets are estimated to be frequent. The resulting algorithm requires fewer database scans than Apriori.
- **Hash-based technique** (hashing itemsets into corresponding buckets): A hash-based technique can be used to reduce the size of the candidate k-itemsets, Ck, for k > 1. For example, when scanning each transaction in the database to generate the frequent 1-itemsets, L1, from the candidate 1-itemsets in C1, we can generate all of the 2-itemsets for each transaction, hash (i.e., map) them into the different buckets of a hash table structure, and increase the corresponding bucket counts . A 2-itemset whose corresponding bucket count in the hash table is below the support threshold cannot be frequent and thus should be removed from the candidate set. Such a hash-based technique may substantially reduce the number of the candidate k-itemsets examined (especially when k = 2).

9) With diagram, explain mining by partitioning data. (4)

Partitioning (partitioning the data to find candidate itemsets): A partitioning technique can be used that requires just two database scans to mine the frequent itemsets (Figure 5.6). It consists of two phases. In Phase I, the algorithm subdivides the transactions of D into n nonoverlapping partitions. If the minimum support threshold for transactions in D is min sup, then the minimum support count for a partition is min sup × the number of transactions in that partition. For each partition, all frequent itemsets within the partition are found. These are referred to as local frequent itemsets. The procedure employs a special data structure that, for each itemset, records the TIDs of the transactions containing the items in the itemset. This allows it to find all of the local frequent k-itemsets, for k = 1, 2,..., in just one scan of the database. A local frequent itemset may or may not be frequent with respect to the entire database, D. Any itemset that is potentially frequent with respect to D must occur as a frequent itemset in at least one of the partitions. Therefore, all local frequent itemsets are candidate itemsets with respect to D. The collection of frequent itemsets from all partitions forms the global candidate itemsets with respect to D. In Phase II, a second scan of D is conducted in which the actual support of each candidate is assessed in order to determine the global frequent itemsets. Partition size and the number of partitions are set so that each partition can fit into main memory and therefore be read only once in each phase.

10) Explain any two pruning strategies of mining closed frequent itemsets. (4)

- **Item merging**:If every transaction containing a frequent itemset X also contains an itemset Y but not any proper superset of Y, then X ∪Y forms a frequent closed itemset

and there is no need to search for any itemset containing X but no Y. For example, in Table 5.2 of Example 5.5, the projected conditional database for prefix itemset {I5:2} is {{I2, I1},{I2, I1, I3}}, from which we can see that each of its transactions contains itemset {I2, I1} but no proper superset of {I2, I1}. Itemset {I2, I1} can be merged with {I5} to form the closed itemset, {I5, I2, I1: 2}, and we do not need to mine for closed itemsets that contain I5 but not {I2, I1}.

- **Sub-itemset pruning:**If a frequent itemset X is a proper subset of an already found frequent closed itemset Y and support count(X) = support count(Y), then X and all of X's descendants in the set enumeration tree cannot be frequent closed itemsets and thus can be pruned. Similar to Example 5.2, suppose a transaction database has only two transactions: {ha1, a2,..., a100i, ha1, a2,..., a50i}, and the minimum support count is min sup = 2. The projection on the first item, a1, derives the frequent itemset, {a1, a2,..., a50 : 2}, based on the itemset merging optimization. Because support({a2}) = support ({a1, a2,..., a50}) = 2, and {a2} is a proper subset of {a1, a2,..., a50}, there is no need to examine a2 and its projected database. Similar pruning can be done for a3,..., a50 as well. Thus the mining of closed frequent itemsets in this data set terminates after mining a1's projected database.

11) Explain the three pruning strategies of mining closed frequent itemsets. (6)

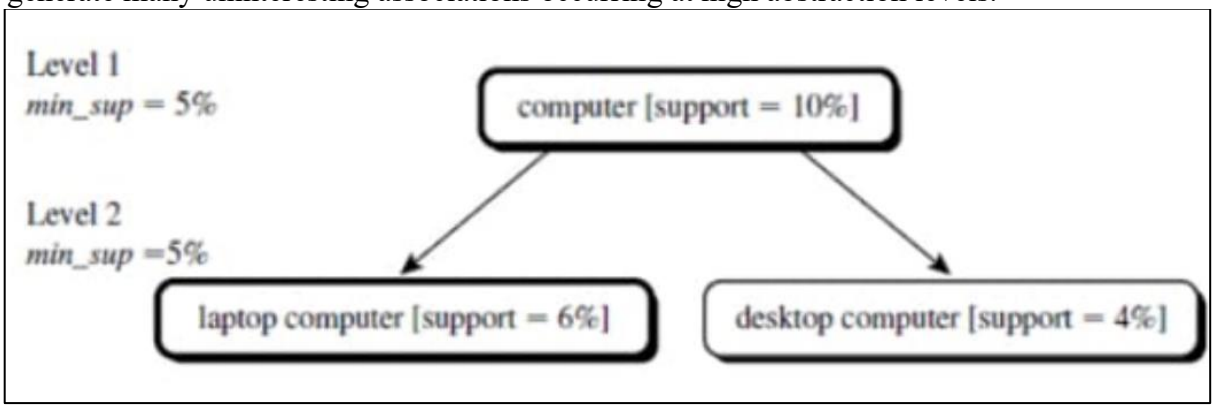
- **Item merging:** If every transaction containing a frequent itemset X also contains an itemset Y but not any proper superset of Y, then XUY forms a frequent closed itemset and there is no need to search for any itemset containing X but no Y.
  For example : in Table 5.2 of Example 5.5, the projected conditional database for prefix itemset {I5:2} is {{I2,I1}, { I2,I1,I3}}, from which we can see that each of its transactions contains itemset {I2,I1} but no proper superset of {I2,I1}. Itemset {I2,I1} can be merged with {I5} to form the closed itemset, {I5, I2, I1: 2}, and we do not need to mine for closed itemsets that contain I5 but not {I2, I1}.
- **Sub-itemset pruning:** If a frequent itemset X is a proper subset of an already found frequent closed itemset Y and support count(X) = support count(Y), then X and all of X's descendants in the set enumeration tree cannot be frequent closed itemsets and thus can be pruned.
  Similar to Example 5.2, suppose a transaction database has only two transactions: {(a1, a2,…, a100), (a1, a2,…, a50)}, and the minimum support count is min_sup = 2. The projection on the first item, a1, derives the frequent itemset, {a1, a2,…, a50 : 2}, based on the itemset merging optimization. Because support({a2}) = support ({a1, a2,…, a50}) = 2, and {a2} is a proper subset of {a1, a2,…., a50}, there is no need to examine a2 and its projected database. Similar pruning can be done for a3,…, a50 as well. Thus the mining of closed frequent itemsets in this data set terminates after mining a1's projected database.
- **Item skipping:** In the depth-first mining of closed itemsets, at each level, there will be a prefix itemset X associated with a header table and a projected database. If a local fre-quent item p has the same support in several header tables at different levels, we can safely prune p from the header tables at higher levels.
  Consider, for example, the transaction database above having only two transactions: {(a1, a2,…, a100), (a1, a2,…, a50)}, where min_sup=2. Because a2 in a1's projected

database has the same support as a2 in the global header table, a2 can be pruned from the global header table. Similar pruning can be done for a3,…, a50. There is no need to mine anything more after mining a1's projected database

12) Write a note on:
i) Mining Multilevel association rule with uniform minimum support
ii) Mining Multilevel association with reduced minimum support. (6)

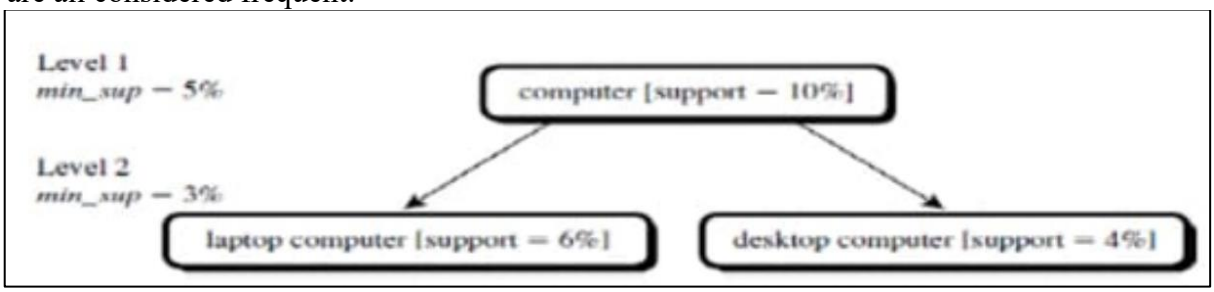
1) uniform minimum support

The same minimum support threshold is used when mining at each level of abstraction. When a uniform minimum support threshold is used, the search procedure is simplified. The method is also simple in that users are required to specify only one minimum support threshold.
The uniform support approach, however, has some difficulties. It is unlikely that items at lower levels of abstraction will occur as frequently as those at higher levels of abstraction. If the minimum support threshold is set too high, it could miss some meaningful associations occurring at low abstraction levels. If the threshold is set too low, it may generate many uninteresting associations occurring at high abstraction levels.



2) Reduced Minimum Support:

· Each level of abstraction has its own minimum support threshold.
· The deeper the level of abstraction, the smaller the corresponding threshold is.
· For example, the minimum support thresholds for levels 1 and 2 are 5% and 3%, respectively. In this way, —computer, ‖ —laptop computer, ‖ and —desktop computer‖ are all considered frequent.



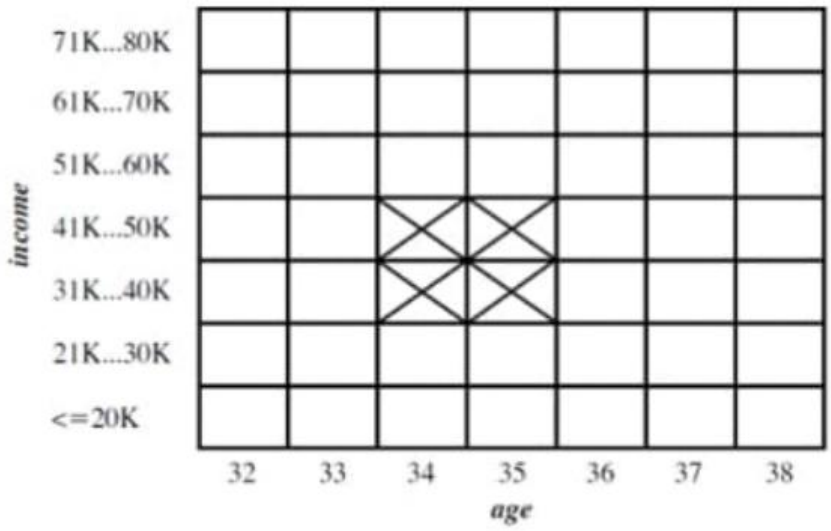
13) Briefly explain the steps involved in ARCS. (6)

1. Binning:

· Quantitative attributes, like age and income, can have a wide range of values, making it impractical to plot them directly on a 2-D grid.

· Instead, these attributes are divided into intervals or "bins" to create manageable grid sizes. There are three common binning strategies:
· Equal-width binning: Each bin has the same interval size.
· Equal-frequency binning: Each bin has approximately the same number of data points.
· Clustering-based binning: Neighboring data points are grouped together into the same bin using clustering techniques.
· ARCS uses equal-width binning, where the user specifies the size of each bin for each quantitative attribute.-
2. 2-D Array Creation:
· For each possible combination of bins involving both quantitative attributes, a 2-D array is created. · Each cell in the array holds the count distribution for each possible class of the categorical attribute (the "rule right-hand side").

3. Data Structure for Rule Generation:
· By creating this data structure, the relevant data only needs to be scanned once.
· The same 2-D array can be used to generate association rules for any value of the categorical attribute, based on the same two quantitative attributes.
4. Finding Frequent Predicate Sets:
· The 2-D array is scanned to find frequent predicate sets that satisfy minimum support and confidence thresholds.
· Strong association rules are then generated from these predicate sets using a rule generation algorithm.
5.Clustering the association rules:
The strong association rules obtained in the previous step are then mapped to a 2-D grid. Figure below shows a 2-D grid for 2-D quantitative association rules predicting the condition buys(X, “HDTV”) on the rule right-hand side, given the quantitative attributes age and income.
The four Xs correspond to the rules
· age(X, 34)^income(X, “31K..40K”))buys(X, “HDTV”) (5.16)
· age(X, 35)^income(X, “31K…40K”))buys(X, “HDTV”) (5.17)
· age(X, 34)^income(X, “41K…50K”))buys(X, “HDTV”) (5.18)
· age(X, 35)^income(X, “41K…50K”))buys(X, “HDTV”). (5.19)
the four rules can be combined or “clustered” together to form the following simpler rule, which subsumes and replaces the above four rules:



5.14 A 2-D grid for tuples representing customers who purchase high-definition TVs.

$$age(X, \text{“34...35”}) \wedge income(X, \text{“31}K\text{...50}K\text{”}) \Rightarrow buys(X, \text{“}HDTV\text{”}) \quad (5$$

ARCS employs a clustering algorithm for this purpose. The algorithm scans the grid, searching for rectangular clusters of rules. In this way, bins of the quantitative attributes occurring within a rule cluster may be further combined, and hence further dynamic discretization of the quantitative attributes occurs.

14) What is constrain based mining? Explain the constraints included in constrain based mining. (6)

Constraint-Based MiningA data mining process may uncover thousands of rules from a given set of data, most of which end up being unrelated or uninteresting to the users. Often, users have a good sense of which “direction” of mining may lead to interesting patterns and the “form” of the patterns or rules they would like to find. Thus, a good heuristic is to have the users specify such intuition or expectations as constraints to confine the search space. This strategy is known as constraint-based mining.

The constraints can include the following:

Knowledge type constraints: These specify the type of knowledge to be mined, suchas association or correlation.Data constraints: These specify the set of task-relevant data.Dimension/level constraints: These specify the desired dimensions (or attributes) of the data, or levels of the concept hierarchies, to be used in mining.Interestingness constraints: These specify thresholds on statistical measures of rule interestingness, such as support, confidence, and correlation.Rule constraints: These specify the form of rules to be mined. Such constraints may be expressed as metarules (rule templates), as the maximum or minimum number of predicates that can occur in the rule antecedent or consequent, or as relationships among attributes, attribute values, and/or aggregates.

15) Explain association rules with example.

Generating Association Rules from Frequent Itemsets: Once the frequent itemsets from transactions in a database D have been found, it is straightforward to generate strong association rules from them.

$$confidence(A \Rightarrow B) = P(B|A) = \frac{support\_count(A \cup B)}{support\_count(A)}.$$

The conditional probability is expressed in terms of itemset support count, where $support\_count(A \cup B)$ is the number of transactions containing the itemsets $A \cup B$, and $support\_count(A)$ is the number of transactions containing the itemset A. Based on this equation, association rules can be generated as follows:

- For each frequent itemset l, generate all nonempty subsets of l.
- For every nonempty subset s of l, output the rule "$s \Rightarrow (l - s)$" if $\frac{support\_count(l)}{support\_count(s)} \geq min\_conf$, where min_conf is the minimum confidence threshold.

Example:

Generating association rules. Let's try an example based on the transactional data for *AllElectronics* shown in Table 5.1. Suppose the data contain the frequent itemset $l = \{I1, I2, I5\}$. What are the association rules that can be generated from l? The nonempty subsets of l are $\{I1, I2\}$, $\{I1, I5\}$, $\{I2, I5\}$, $\{I1\}$, $\{I2\}$, and $\{I5\}$. The resulting association rules are as shown below, each listed with its confidence:

$I1 \wedge I2 \Rightarrow I5$, $confidence = 2/4 = 50\%$
$I1 \wedge I5 \Rightarrow I2$, $confidence = 2/2 = 100\%$
$I2 \wedge I5 \Rightarrow I1$, $confidence = 2/2 = 100\%$
$I1 \Rightarrow I2 \wedge I5$, $confidence = 2/6 = 33\%$
$I2 \Rightarrow I1 \wedge I5$, $confidence = 2/7 = 29\%$
$I5 \Rightarrow I1 \wedge I2$, $confidence = 2/2 = 100\%$

16) Explain Apriori algorithm for finding frequent item sets with an example.

The Apriori Algorithm: Finding Frequent Itemsets Using Candidate GenerationApriori is a seminal algorithm proposed by R. Agrawal and R. Srikant in 1994 for mining frequent itemsets for Boolean association rules. The name of the algorithm is based on the fact that the algorithm uses prior knowledge of frequent itemset properties, as we shall see following. Apriori employs an iterative approach known as a level-wise search, where k-itemsets are used to explore (k+1)-itemsets. First, the setof frequent 1-itemsets is found by scanning the

database to accumulate the count for each item, and collecting those items that satisfy minimum support. The resulting set is denoted L1.Next, L1 is used to find L2, the set of frequent 2-itemsets, which is used to find L3, and so on, until no more frequent k-itemsets can be found. The finding of each Lk requires one full scan of the database.

Transactional data for an *AllElectronics* branch.

TID	*List of item_IDs*
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Algorithm: Apriori. Find frequent itemsets using an iterative level-wise approach based on candidate generation.

Input:

- D, a database of transactions;
- min_sup, the minimum support count threshold.

Output: L, frequent itemsets in D.

Method:

```
(1)   L1 = find_frequent_1-itemsets(D);
(2)   for (k = 2; Lk−1 ≠ ϕ; k++) {
(3)       Ck = apriori_gen(Lk−1);
(4)       for each transaction t ∈ D { // scan D for counts
(5)           Ct = subset(Ck, t); // get the subsets of t that are candidates
(6)           for each candidate c ∈ Ct
(7)               c.count++;
(8)       }
(9)       Lk = {c ∈ Ck | c.count ≥ min_sup}
(10)  }
(11)  return L = ∪k Lk;

procedure apriori_gen(Lk−1:frequent (k − 1)-itemsets)
(1)   for each itemset l1 ∈ Lk−1
(2)       for each itemset l2 ∈ Lk−1
(3)           if (l1[1] = l2[1]) ∧ (l1[2] = l2[2]) ∧ ... ∧ (l1[k − 2] = l2[k − 2]) ∧ (l1[k − 1] < l2[k − 1]) then {
(4)               c = l1 ⋈ l2; // join step: generate candidates
(5)               if has_infrequent_subset(c, Lk−1) then
(6)                   delete c; // prune step: remove unfruitful candidate
(7)               else add c to Ck;
(8)           }
(9)   return Ck;

procedure has_infrequent_subset(c: candidate k-itemset;
          Lk−1: frequent (k − 1)-itemsets); // use prior knowledge
(1)   for each (k − 1)-subset s of c
(2)       if s ∉ Lk−1 then
(3)           return TRUE;
(4)   return FALSE;
```

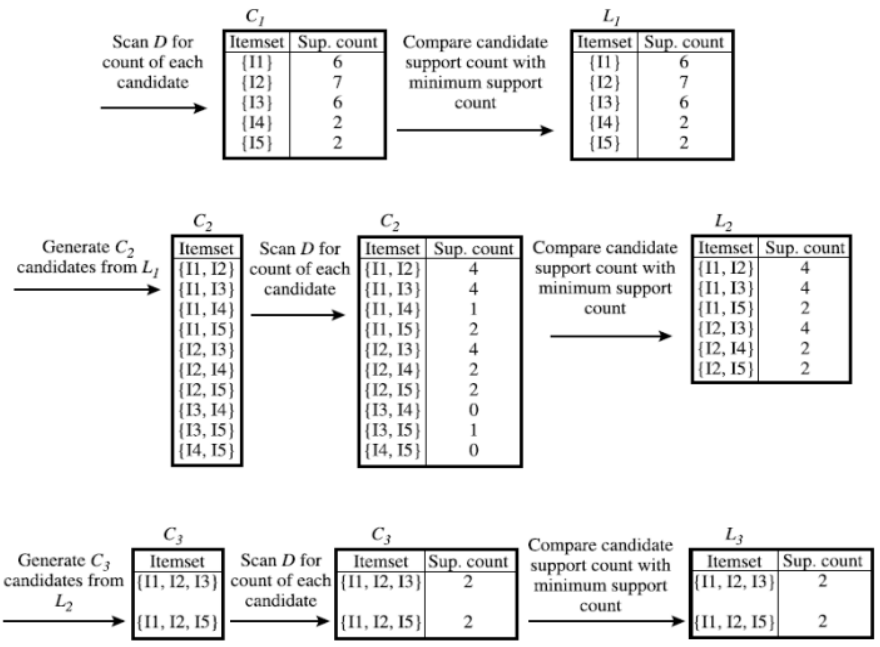
5.4 The Apriori algorithm for discovering frequent itemsets for mining Boolean association rules.

There are nine transactions in this database, that is, |D| = 9.

Steps:

1. In the first iteration of the algorithm, each item is a member of the set of candidate1-itemsets, C1. The algorithm simply scans all of the transactions in order to count the number of occurrences of each item.
2. Suppose that the minimum support count required is 2, that is, min sup = 2. The set of frequent 1- itemsets, L1, can then be determined. It consists of the candidate 1-itemsets satisfying minimum support.In our example, all of the candidates in C1 satisfy minimum support.
3. To discover the set of frequent 2-itemsets, L2, the algorithm uses the join L1 on L1 to generate a candidate set of 2-itemsets, C2.No candidates are removed from C2 during the prune step because each subset of the candidates is also frequent.

4. Next, the transactions in D are scanned and the support count of each candidate itemset In C2 is accumulated.
5. The set of frequent 2-itemsets, L2, is then determined, consisting of those candidate2- itemsets in C2 having minimum support.
6. The generation of the set of candidate 3-itemsets,C3, From the join step, we first get C3 =L2x L2 = ({I1, I2, I3}, {I1, I2, I5}, {I1, I3, I5}, {I2, I3, I4},{I2, I3, I5}, {I2, I4, I5}. Based on the Apriori property that all subsets of a frequent itemset must also be frequent, we can determine that the four latter candidates cannot possibly be frequent.
7. The transactions in D are scanned in order to determine L3, consisting of those candidate 3-itemsets in C3 having minimum support.
8. The algorithm uses L3x L3 to generate a candidate set of 4-itemsets,



gure 5.2 Generation of candidate itemsets and frequent itemsets, where the minimum support count is 2.

(a) Join: $C_3 = L_2 \bowtie L_2 = \{\{I1, I2\}, \{I1, I3\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}\} \bowtie$
$\{\{I1, I2\}, \{I1, I3\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}\}$
$= \{\{I1, I2, I3\}, \{I1, I2, I5\}, \{I1, I3, I5\}, \{I2, I3, I4\}, \{I2, I3, I5\}, \{I2, I4, I5\}\}.$

(b) Prune using the Apriori property: All nonempty subsets of a frequent itemset must also be frequent. Do any of the candidates have a subset that is not frequent?

- The 2-item subsets of $\{I1, I2, I3\}$ are $\{I1, I2\}$, $\{I1, I3\}$, and $\{I2, I3\}$. All 2-item subsets of $\{I1, I2, I3\}$ are members of L_2. Therefore, keep $\{I1, I2, I3\}$ in C_3.
- The 2-item subsets of $\{I1, I2, I5\}$ are $\{I1, I2\}$, $\{I1, I5\}$, and $\{I2, I5\}$. All 2-item subsets of $\{I1, I2, I5\}$ are members of L_2. Therefore, keep $\{I1, I2, I5\}$ in C_3.
- The 2-item subsets of $\{I1, I3, I5\}$ are $\{I1, I3\}$, $\{I1, I5\}$, and $\{I3, I5\}$. $\{I3, I5\}$ is not a member of L_2, and so it is not frequent. Therefore, remove $\{I1, I3, I5\}$ from C_3.
- The 2-item subsets of $\{I2, I3, I4\}$ are $\{I2, I3\}$, $\{I2, I4\}$, and $\{I3, I4\}$. $\{I3, I4\}$ is not a member of L_2, and so it is not frequent. Therefore, remove $\{I2, I3, I4\}$ from C_3.
- The 2-item subsets of $\{I2, I3, I5\}$ are $\{I2, I3\}$, $\{I2, I5\}$, and $\{I3, I5\}$. $\{I3, I5\}$ is not a member of L_2, and so it is not frequent. Therefore, remove $\{I2, I3, I5\}$ from C_3.
- The 2-item subsets of $\{I2, I4, I5\}$ are $\{I2, I4\}$, $\{I2, I5\}$, and $\{I4, I5\}$. $\{I4, I5\}$ is not a member of L_2, and so it is not frequent. Therefore, remove $\{I2, I4, I5\}$ from C_3.

(c) Therefore, $C_3 = \{\{I1, I2, I3\}, \{I1, I2, I5\}\}$ after pruning.

.3 Generation and pruning of candidate 3-itemsets, C_3, from L_2 using the Apriori property.

17) Explain support, confidence and lift measure with respect to association rule mining

1. Support

Support is a measure of how frequently the items appear in the dataset. It helps to identify the most common items or itemsets in the dataset.

- **Definition**: The support of an itemset is the proportion of transactions in the dataset that contain the itemset.
  - Formula:

$$\text{Support}(A) = \frac{\text{Number of transactions containing } A}{\text{Total number of transactions}}$$

  - **Example**: If we have 100 transactions and item A appears in 20 of them, the support for item A is $\frac{20}{100} = 0.2$ or 20%.

2. Confidence

Confidence is a measure of the reliability of the inference made by a rule. It quantifies the likelihood of finding item B in transactions under the condition that the transaction already contains item A.

- **Definition**: The confidence of a rule $A \rightarrow B$ is the proportion of transactions that contain A which also contain B.

- **Formula**:

$$\text{Confidence}(A \rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$$

- **Example**: If item A appears in 20 transactions, and item B appears in 10 of those 20 transactions, the confidence of the rule $A \rightarrow B$ is $\frac{10}{20} = 0.5$ or 50%.

3. Lift

Lift measures the strength of a rule over the random co-occurrence of the itemset, providing a metric to understand how much more likely item B is to be bought when item A is bought compared to if B was bought independently.

- **Definition**: The lift of a rule $A \rightarrow B$ is the ratio of the observed support to the expected support if A and B were independent.

- **Formula**:

$$\text{Lift}(A \rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A) \times \text{Support}(B)}$$

- **Interpretation**:
  - Lift > 1: Positive association (items A and B are more likely to occur together than if they were independent).
  - Lift $= 1$: No association (items A and B are independent).
  - Lift < 1: Negative association (items A and B are less likely to occur together).
- **Example**: Suppose item A has a support of 20%, item B has a support of 30%, and both items A and B together have a support of 10%. The lift of the rule $A \rightarrow B$ is:

$$\text{Lift}(A \rightarrow B) = \frac{0.10}{0.20 \times 0.30} = \frac{0.10}{0.06} = 1.67$$

This means that item B is 1.67 times more likely to be bought when item A is bought compared to buying item B independently.

18) Explain the basic concept of frequent itemset mining

Basic Concept of Frequent Itemset Mining

1. **Transaction Database**: A collection of transactions, where each transaction is a set of items. For example, in a retail context, each transaction might represent a single purchase and the items are the products bought.

2. **Itemset**: A collection of one or more items. For instance, {bread, butter} is an itemset containing two items.

3. **Support Count (Frequency)**: The number of transactions in which a particular itemset appears. For example, if the itemset {bread, butter} appears in 30 transactions out of 100, its support count is 30.

4. **Support**: The proportion of transactions in which an itemset appears. It is calculated as the support count divided by the total number of transactions. Using the previous example, the support for {bread, butter} is 30100=0.310030=0.3 or 30%.

5. **Frequent Itemset**: An itemset is considered frequent if its support is greater than or equal to a user-specified minimum support threshold. This threshold is a parameter set by the user to define what is considered "frequent."

Steps in Frequent Itemset Mining

1. **Set Minimum Support Threshold**: Define the minimum support threshold to identify frequent itemsets. This value is typically chosen based on domain knowledge and the specific goals of the analysis.

2. **Generate Candidate Itemsets**: Create potential itemsets that could be frequent. Initially, this might be all individual items (1-itemsets).

3. **Count Support for Candidate Itemsets**: Scan the transaction database to count the support for each candidate itemset.

4. **Prune Infrequent Itemsets**: Eliminate itemsets whose support is below the minimum support threshold. The remaining itemsets are considered frequent.

5. **Generate Larger Itemsets**: Use the frequent itemsets to generate candidate itemsets of larger sizes (e.g., from 1-itemsets to 2-itemsets, from 2-itemsets to 3-itemsets, and so on). This step typically uses algorithms like the Apriori principle, which states that any subset of a frequent itemset must also be frequent.

6. **Repeat**: Continue the process of generating candidate itemsets, counting their support, and pruning infrequent itemsets until no more frequent itemsets can be found.

Algorithms for Frequent Itemset Mining

Several algorithms have been developed for efficient frequent itemset mining:

1. **Apriori Algorithm**: Uses the Apriori principle to reduce the number of candidate itemsets by pruning infrequent subsets. It iteratively generates candidate itemsets of increasing length and counts their support.

2. **FP-Growth Algorithm (Frequent Pattern Growth)**: Uses a compressed representation of the transaction database called an FP-tree (Frequent Pattern Tree) to

avoid multiple database scans. It generates frequent itemsets by recursively projecting the database into smaller subsets.

19) Explain multilevel association rules and multidimensional association rules for mining data

Multilevel Association Rules

Multilevel association rules involve finding relationships between items at different levels of abstraction in a hierarchical structure. For instance, in a retail scenario, products can be organized into categories such as "Electronics" and "Home Appliances," which can be further divided into subcategories like "Mobile Phones" and "Refrigerators."

1. **Hierarchy of Items**: Items are organized in a hierarchical manner. For example:
    - Level 1: Electronics, Home Appliances
    - Level 2: Mobile Phones, Laptops (under Electronics), Refrigerators, Washing Machines (under Home Appliances)
    - Level 3: Specific brands or models of mobile phones, laptops, etc.
2. **Support and Confidence**: At different levels, the support (frequency of itemsets) and confidence (reliability of the association) are calculated. Higher-level associations might be more general but less specific, while lower-level associations might be more detailed.
3. **Rule Generation**: Rules can be generated at multiple levels. For instance:
    - High-level rule: If a customer buys Electronics, they are likely to buy Home Appliances.
    - Lower-level rule: If a customer buys Mobile Phones, they are likely to buy Laptops.
4. **Applications**: This is useful in scenarios where data is naturally hierarchical, such as product categorizations, web page structures, and organizational hierarchies.

Multidimensional Association Rules

Multidimensional association rules involve relationships between items across multiple dimensions or attributes, rather than a single dimension of items. This approach allows for more complex patterns that consider various aspects of transactions.

1. **Multiple Dimensions**: Each itemset can have multiple dimensions. For example, a retail transaction database might include dimensions such as time, location, customer demographics, and product categories.
2. **Rule Structure**: A typical rule might look like:

- If (Gender = Male) and (Age = 20-30) and (Product Category = Electronics), then (Product Category = Gaming Consoles).
- This rule considers multiple dimensions: customer gender, age, and product category.

3. **Mining Process**: The process involves:

- **Preprocessing**: Transforming data into a format where each transaction includes multiple dimensions.
- **Pattern Discovery**: Identifying frequent itemsets across these dimensions.
- **Rule Generation**: Creating rules that show the relationships across dimensions.

4. **Applications**: Useful in marketing for targeted promotions, healthcare for understanding patient profiles and treatment patterns, and finance for analyzing transaction patterns across different demographics.

Example:

TID	T100	T200	T300	T400	T500	T600	T700	T800	T900
ITEM IDS	I1,I2,I5	I2,I4	I2,I3	I1,I2,I4	I1,I3	I2,I3	I1,I3	I1,I2,I3,I5	I1,I2,I3

Generate the list of frequent item-set ordered by their corresponding suffixes, where the minimum support count is 2 and minimum confidence is 60%.

20)Make use of the database which has five transactions. Let minimum support = 60% and minimum confidence = 80%.

Transaction	Items
T10	M, O, N, K, E, Y
T20	D, O, N, K, E, Y
T30	M, A, K, E
T40	M, U, C, K, Y
T50	C, O, O, K, I, E

Find all frequent item sets using Apriori and FP-growth, respectively

To find all frequent item sets using Apriori and FP-Growth algorithms, let's start by understanding the given transactions and the criteria.
Transactions:
T10: M, O, N, K, E, Y
T20: D, O, N, K, E, Y
T30: M, A, K, E
T40: M, U, C, K, Y
T50: C, O, O, K, I, E
Parameters:
Minimum Support: 60%

Minimum Confidence: 80%

Step 1: Calculate the minimum support count
Minimum support = 60% of total transactions
Total transactions = 5
Minimum support count = 0.6 * 5 = 3
So, an itemset must appear in at least 3 transactions to be considered frequent.
Apriori Algorithm:

1. Generate candidate 1-itemsets (C1):
C1 = {M, O, N, K, E, Y, D, A, U, C, I}
2. Calculate support for each candidate in C1:
M: 3
O: 3
N: 2
K: 5
E: 4
Y: 3
D: 1
A: 1
U: 1
C: 2
I: 1
3. Select frequent 1-itemsets (L1) with minimum support count ≥ 3:
L1 = {M, O, K, E, Y}
4.Generate candidate 2-itemsets (C2) from L1:
C2 = {MO, MK, ME, MY, OK, OE, OY, KE, KY, EY}
5. Calculate support for each candidate in C2:

MO: 1
MK: 3
ME: 2
MY: 2
OK: 2
OE: 2
OY: 2
KE: 3
KY: 3
EY: 3
6. Select frequent 2-itemsets (L2) with minimum support count ≥ 3:
L2 = {MK, KE, KY, EY}
7.Generate candidate 3-itemsets (C3) from L2:
C3 = {MKE, MKY, KEY}
8. Calculate support for each candidate in C3:

MKE: 1

MKY: 2
KEY: 2
9. Select frequent 3-itemsets (L3) with minimum support count ≥ 3:
L3 = {}
Conclusion:
Frequent itemsets using Apriori: {M, O, K, E, Y, MK, KE, KY, EY}
FP-Growth Algorithm:
FP-Growth algorithm uses a different approach by building a prefix tree (FP-tree) to represent the dataset and then extracting the frequent itemsets from this tree.

1.. Construct FP-tree:
- Scan the dataset to determine the frequency of each item.
- Order items in each transaction by their frequency in descending order.
- Build the tree with these ordered transactions.

2.FP-tree Construction:
- Transactions ordered by frequency (descending):
  T10: K, E, M, O, N, Y
  T20: K, E, O, N, D, Y
  T30: K, E, M, A
  T40: K, Y, M, U, C
  T50: O, O, K, E, C, I

3. Extract frequent itemsets from the FP-tree:
- Starting from the least frequent item, create conditional FP-trees and find frequent patterns.

Steps:
- Create conditional trees for each item and extract patterns:

K-conditional FP-tree:
- Frequent patterns: {K, E, M, Y}, {K, E, O, Y}, {K, E}, {K, Y}, {K, O}, {K, E, M}, {K, E, Y}, {K, E, O}, {K, Y, M}

E-conditional FP-tree:
- Frequent patterns: {E, K, M}, {E, K, Y}, {E, K, O}, {E, K}, {E, M}, {E, Y}

O-conditional FP-tree:
- Frequent patterns: {O, K, E}, {O, K, Y}

Y-conditional FP-tree:
- Frequent patterns: {Y, K}, {Y, K, E}
Conclusion:
Frequent itemsets using FP-Growth: {K, E, Y}, {K, E}, {K, Y}, {K, O}, {K, M}, {E, M}, {E, Y}, {O, Y}
By comparing results from both algorithms:
Apriori Frequent Item sets: {M, O, K, E, Y, MK, KE, KY, EY}

FP-Growth Frequent Item sets: {K, E, Y}, {K, E}, {K, Y}, {K, O}, {K, M}, {E, M}, {E, Y}, {O, Y}

The frequent item sets found with both methods largely overlap, indicating consistency in the results.

21)Explain about Constraint based Association mining

Constraint-Based Association Mining

A data mining process may uncover thousands of rules from a given set of data, most of which end up being unrelated or uninteresting to the users. Often ,users have a good sense of which "direction" of mining may lead to interesting patterns and the "form" of the patterns or rules they would like to find. Thus, a good heuristic is to have the users specify such intuition or expectations as constraints to confine the search space. This strategy is known as constraint-based mining. The constraints can include the following:

- Knowledge type constraints: These specify the type of knowledge to be mined, such as association or correlation.
- Data constraints: These specify the set of task-relevant data. Dimension/level constraints: These specify the desired dimensions (or attributes) of the data, or levels of the concept hierarchies, to be used in mining.
- Interestingness constraints: These specify thresholds on statistical measures of rule interestingness, such as support, confidence, and correlation.
- Rule constraints: These specify the form of rules to be mined. Such constraints may be expressed as metarules (rule templates), as the maximum or minimum number of predicates that can occur in the rule antecedent or consequent, or as relationships among attributes, attribute values, and/or aggregates.

The above constraints can be specified using a high-level declarative data mining query language and user interface

22)Describe the steps involved in improving the efficiency of the Apriori Algorithm.

"How can we further improve the efficiency of Apriori-based mining?" Many variations of the Apriori Algorithm have been proposed that focus on improving the efficiency of the original algorithm. Several of these variations are summarized as follows:

Hash-based technique (hashing item sets into corresponding buckets): A hash-based technique can be used to reduce the size of the candidate k-item sets, Ck, for k > 1. For example, when scanning each transaction in the database to generate the frequent 1-itemsets, L1, from the candidate 1-itemsets in C1, we can generate all of the 2-itemsets for each transaction, hash (i.e., map) them into the different buckets of a hash table structure, and increase the corresponding bucket counts (Figure 5.5). A2-itemset whose corresponding bucket count in the hash table is below the support

H_2

Create hash table H_2 using hash function $h(x, y) = ((order\ of\ x) \times 10 + (order\ of\ y)) \bmod 7$

bucket address	0	1	2	3	4	5	6
bucket count	2	2	4	2	2	4	4
bucket contents	{I1, I4} {I3, I5}	{I1, I5} {I1, I5}	{I2, I3} {I2, I3} {I2, I3} {I2, I3}	{I2, I4} {I2, I4}	{I2, I5} {I2, I5}	{I1, I2} {I1, I2} {I1, I2} {I1, I2}	{I1, I3} {I1, I3} {I1, I3} {I1, I3}

Hashtable,H2,forcandidate2-itemsets:Thishashtablewasgeneratedbyscanningthetrans actions of Table 5.1 while determining L1 from C1. If the minimum support count is, say, 3, then the item sets in buckets 0, 1, 3, and 4 cannot be frequent and so they should not be included in C2. threshold cannot be frequent and thus should be removed from the candidate set. Such a hash-based technique may substantially reduce the number of the candidate k-item sets examined (especially when k = 2).

Transaction reduction (reducing the number of transactions scanned in future iterations): Atransactionthatdoesnotcontainanyfrequentk-itemsetscannotcontainany frequent (k +1)-item sets. Therefore, such a transaction can be marked or removed from further consideration because subsequent scans of the database for j-item sets, where j >k, will not require it.

Partitioning (partitioning the data to find candidate item sets): A partitioning technique can be used that requires just two database scans to mine the frequent item sets (Figure 5.6). It consists of two phases. In Phase I, the algorithm subdivides the trans actions of D into n nonoverlapping partitions. If the minimum support threshold for transactions in D is min sup, then the minimum support count for a partition is min sup the number of transactions in that partition. For each partition, all frequent item sets within the partition are found. These are referred to as local frequent item sets. The procedure employs a special data structure that, for each itemset, records the TIDs of the transactions containing the items in the itemset. This allows it to find all of the local frequent k-item sets, for k = 1, 2,:::, in just one scan of the database. A local frequent itemset may or may not be frequent with respect to the entire database, D. Any itemset that is potentially frequent with respect to D must occur as a frequent itemset in at least one of the partitions. Therefore, all local frequent item sets are candidate item sets with respect to D. The collection of frequent item sets from all partitions forms the global candidate item sets with respect to D. In Phase II, a second scan of D is conducted in which the actual support of each candidate is assessed in order to determine the global frequent item sets. Partition size and the number of partitions are set so that each partition can fit into main memory and therefore be read only once in each phase.

Sampling (mining on a subset of the given data): The basic idea of the sampling approach is to pick a random sample S of the given data D, and then search for frequent item sets in S instead of D. In this way, we trade off some degree of accuracy

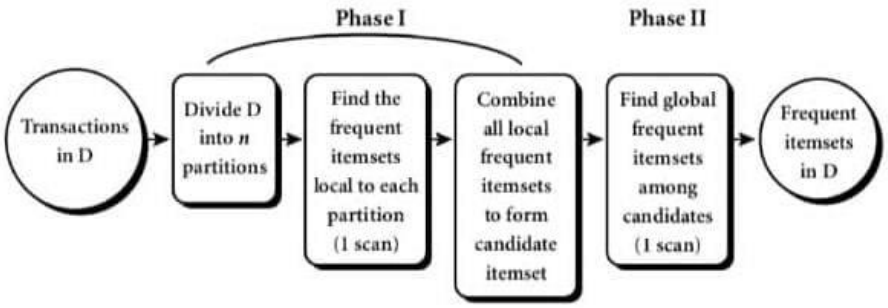


Figure 5.6 Mining by partitioning the data.

against efficiency. The sample size of S is such that the search for frequent item sets in S can be done in main memory, and so only one scan of the transactions in S is required overall. Because we are searching for frequent item sets in S rather than in D, it is possible that we

will miss some of the global frequent item sets. To lessen this possibility, we use a lower support threshold than minimum support to find the frequent item sets local to S (denoted LS). The rest of the database is then used to compute the actual frequencies of each itemset in LS. A mechanism is used to determine whether all of the global frequent item sets are included in LS. If LS actually contains all of the frequent item sets in D, then only one scan of D is required. Otherwise, a second pass can be done in order to find the frequent item sets that were missed in the first pass. The sampling approach is especially beneficial when efficiency is of utmost importance, such as in computationally intensive applications that must be run frequently.

Dynamic itemset counting(adding candidate itemsets at different points during scan): A dynamic itemset counting technique was proposed in which the database is partitioned into blocks marked by start points. In this variation, new candidate item sets can be added at any start point, unlike in Apriori which determines new candidate item sets only immediately before each complete database scan. The technique is dynamic in that it estimates the support of all of the item sets that have been counted so far, adding new candidate item sets if all of their subsets are estimated to be frequent The resulting algorithm requires fewer database scans than Apriori.

Unit IV

2 marks questions

1. What is Classification?

Classification is a supervised learning technique where the goal is to assign a label to a given input based on its features. It involves predicting the categorical label of a new instance, given a set of instances with known labels.

2. What is Prediction?

Prediction refers to the process of forecasting or estimating the outcome of an event based on past and present data. In data science, it often involves building models to predict future values from existing data.

3. What is Regression Analysis?

Regression analysis is a statistical method used to model the relationship between a dependent variable and one or more independent variables. It helps in predicting a continuous outcome variable based on the values of the predictor variables.

4. Write steps involved in data classification:

- Data Preprocessing
- Feature Selection
- Model Selection
- Training the Model
- Evaluating the Model
- Making Predictions

5. What is Supervised Learning?

Supervised learning is a type of machine learning where the algorithm is trained on labeled data. It uses input-output pairs to learn a mapping from inputs to outputs, allowing it to predict the output for new inputs.

6. What is Unsupervised Learning?

Unsupervised learning is a type of machine learning where the algorithm is trained on data without labeled responses. It seeks to find hidden patterns or intrinsic structures in the input data.

7. What is Correlation Analysis?

Correlation analysis is a statistical technique used to determine the degree and direction of the relationship between two or more variables. It measures how changes in one variable are associated with changes in another.

8. What is Decision Tree Induction?

Decision tree induction is the process of creating a decision tree from a dataset. It involves selecting the best attribute to split the data at each node and recursively partitioning the data until the leaves are pure or meet a stopping criterion.

9. What is Decision Tree?

A decision tree is a tree-like model used for classification and regression. Each internal node represents a test on an attribute, each branch represents an outcome of the test, and each leaf node represents a class label or a continuous value.

10. What is Tree Pruning?

Tree pruning is the process of removing parts of the tree that are not necessary for classifying instances. It helps in reducing the complexity of the model and improving its generalization ability by removing overfitting.

11. What is the use of Tree Selection Measures?

Tree selection measures, such as information gain, gain ratio, and Gini index, are used to select the best attribute to split the data at each node in a decision tree. They help in choosing the attribute that best separates the data into distinct classes.

12. Expand CART, ID3:

- CART: Classification and Regression Trees
- ID3: Iterative Dichotomiser 3

13. What is Splitting Criterion? List distinct splitting criteria's values:

Splitting criterion is a rule used to decide which feature should be used to split the data at each node in a decision tree. Common splitting criteria include:

- Information Gain
- Gain Ratio
- Gini Index

14. What is Attribute Selection Measure? List popular attribute measures:

An attribute selection measure is a metric used to select the best attribute to split the data at each node in a decision tree. Popular measures include:

- Information Gain
- Gain Ratio
- Gini Index
- Chi-Square
- Reduction in Variance

15. What is Information Gain? Write the formula to find expected information needed to classify a tuple:

Information gain is a measure of the effectiveness of an attribute in classifying the data. It is calculated as the reduction in entropy after a dataset is split on an attribute.

$$\text{Information Gain}(D, A) = \text{Entropy}(D) - \sum_{v \in \text{Values}(A)} \frac{|D_v|}{|D|} \text{Entropy}(D_v)$$

where D is the dataset, AA is the attribute, and Dv is the subset of D for which attribute A has value v.

16. What is Entropy?

Entropy is a measure of the uncertainty or impurity in a dataset. It is used in decision trees to quantify the amount of disorder or randomness in the data.

$$\text{Entropy}(D) = -\sum_{i=1}^{n} p_i \log_2 p_i$$

where $pipi$ is the proportion of instances belonging to class ii.

17. Define Gain Ratio. Write formula:

Gain ratio is a modification of information gain that reduces its bias towards attributes with a large number of values. It is defined as:

$$\text{Gain Ratio}(A) = \frac{\text{Information Gain}(A)}{\text{Split Information}(A)}$$

where Split Information is the entropy of the attribute values.

18. Define Gini Index. Write formula:

Gini index is a measure of impurity or purity used in the CART algorithm to create decision trees. It is calculated as:

$$\text{Gini}(D) = 1 - \sum_{i=1} (p_i)^2$$

where pi is the proportion of instances in class i

19. List the use of MDL:

The Minimum Description Length (MDL) principle is used for model selection. It favors models that best compress the data, balancing model complexity against the goodness of fit.

20. Write two approaches to tree pruning:

- Pre-pruning (Early Stopping): Halting the tree growth early by setting conditions to stop splitting.
- Post-pruning: Removing branches from a fully grown tree that are not necessary for classification.

21. What is Pessimistic Pruning?

Pessimistic pruning is a post-pruning method where branches are pruned if their presence does not statistically improve the accuracy of the tree on a separate validation set.

22. Expand and briefly explain BOAT:

BOAT: Bootstrapped Optimistic Algorithm for Tree construction. It is an efficient algorithm for constructing decision trees using bootstrap sampling and optimistic error estimates.

23. What are Eager Learners? Give examples:

Eager learners are models that create a generalized model from the training data before receiving queries. Examples include decision trees, neural networks, and support vector machines.

24. What are Instance-based Learners?

Instance-based learners, also known as lazy learners, store the training instances and delay the processing until a query is received. They classify new instances based on the similarity to stored instances. Examples include k-nearest neighbors (k-NN).

25. What is Clustering? Write the advantages:

Clustering is an unsupervised learning technique used to group similar instances into clusters. Advantages include:

- Discovering inherent groupings in data
- Summarizing large datasets
- Improving data compression and reduction

26. What is the purpose of Partitioning Algorithm?

Partitioning algorithms divide a dataset into a set of disjoint clusters, each cluster representing a group of data points that are more similar to each other than to those in other clusters.

27. Expand CLARA and ROCK:

- CLARA: Clustering LARge Applications
- ROCK: Robust Clustering using Links

28. Expand BIRCH and DBSCAN:

- BIRCH: Balanced Iterative Reducing and Clustering using Hierarchies
- DBSCAN: Density-Based Spatial Clustering of Applications with Noise

29. Expand OPTICS and DENCLUE:

- OPTICS: Ordering Points To Identify the Clustering Structure
- DENCLUE: DENsity-based CLUstEring

30. What is STING Clustering?

STING (Statistical Information Grid) clustering is a grid-based method that partitions the data space into a hierarchical structure of rectangular cells and performs clustering based on the statistical information stored in these cells.

31. What is Grid-Based Clustering? Give examples:

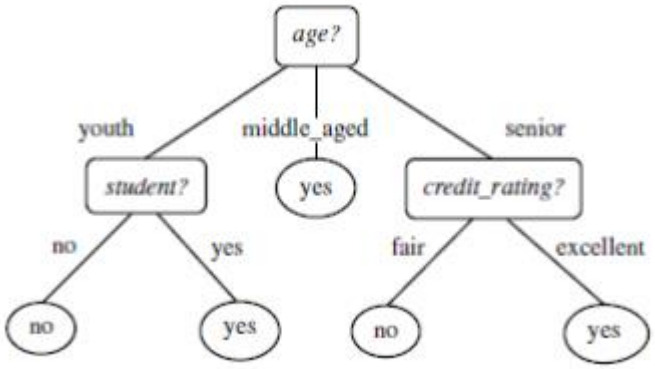
Grid-based clustering methods divide the data space into a finite number of cells that form a grid structure. Clustering operations are performed on these grids. Examples include STING, CLIQUE (CLustering In QUEst), and WaveCluster.

II. Long Questions (4 to 6 marks)

1. Explain Decision Tree Induction Algorithm

Decision tree induction is the learning of decision trees from class-labeled training tuples.
A decision tree is a flowchart-like tree structure,where

- Each internal node denotes a test on an attribute.
- Each branch represents an outcome of the test.
- Each leaf node holds a class label.
- The topmost node in a tree is the root node



- The construction of decision treeclassifiers does not require any domain knowledge or
- parameter setting, and therefore I appropriate for exploratory knowledge discovery.
- Decision trees can handle high dimensionaldata.
- Their representation of acquired knowledge in tree formis intuitive and generallyeasy to
- assimilate by humans.
- The learning and classification steps of decision treeinduction are simple and fast.
- In general, decision tree classifiers have good accuracy.
- Decision tree induction algorithmshave been used for classification in many application
- areas, such as medicine,manufacturing and production, financial analysis, astronomy, and
- molecular biology

Algorithm: Generate_decision_tree. Generate a decision tree from the training tuples of data partition *D*.

Input:

- Data partition, *D*, which is a set of training tuples and their associated class labels;
- *attribute_list*, the set of candidate attributes;
- *Attribute_selection_method*, a procedure to determine the splitting criterion that "best" partitions the data tuples into individual classes. This criterion consists of a *splitting_attribute* and, possibly, either a *split point* or *splitting subset*.

Output: A decision tree.

Method:

```
(1)  create a node N;
(2)  if tuples in D are all of the same class, C then
(3)      return N as a leaf node labeled with the class C;
(4)  if attribute_list is empty then
(5)      return N as a leaf node labeled with the majority class in D; // majority voting
(6)  apply Attribute_selection_method(D, attribute_list) to find the "best" splitting_criterion;
(7)  label node N with splitting_criterion;
(8)  if splitting_attribute is discrete-valued and
         multiway splits allowed then // not restricted to binary trees
(9)      attribute_list ← attribute_list − splitting_attribute; // remove splitting_attribute
(10) for each outcome j of splitting_criterion
     // partition the tuples and grow subtrees for each partition
(11)     let Dj be the set of data tuples in D satisfying outcome j; // a partition
(12)     if Dj is empty then
(13)         attach a leaf labeled with the majority class in D to node N;
(14)     else attach the node returned by Generate_decision_tree(Dj, attribute_list) to node N;
     endfor
(15) return N;
```

The algorithm is called with three parameters:
Data partition
Attribute list
Attribute selection method

2. Explain the Criteria Used for Comparing Classification and Prediction Methods

When comparing classification and prediction methods, several criteria are considered:

1. Accuracy: The proportion of correctly classified instances out of the total instances.
   - Example: If a model correctly predicts 90 out of 100 instances, its accuracy is 90%.
2. Speed: The time taken to build the model (training time) and the time taken to make predictions (prediction time).
   - Example: A model that takes hours to train may not be practical for real-time applications.
3. Robustness: The model's ability to handle noise and outliers.
   - Example: A robust model maintains high accuracy even if the dataset contains some erroneous or extreme values.
4. Scalability: The ability of the model to handle large datasets efficiently.
   - Example: A scalable model performs well on datasets with millions of records without significant performance degradation.

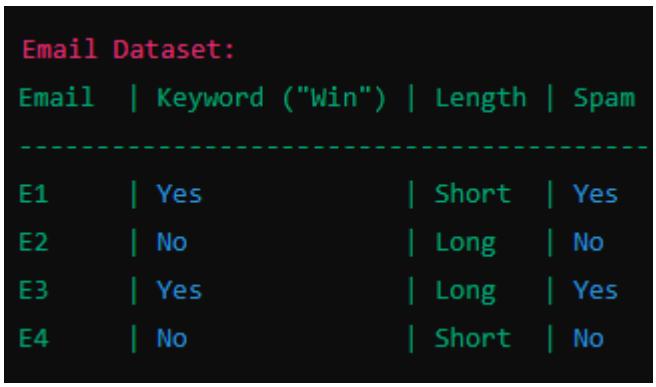
5. Interpretability: How easily the results and the decision-making process of the model can be understood.
   - Example: Decision trees are often considered more interpretable than neural networks.

3. List and Explain the Issues in Classification and Prediction Methods

1.Preparing the Data for Classification and Prediction:
The following preprocessing steps may be applied to the data to help improve the accuracy, efficiency, and scalability of the classification or prediction process.
(i)Data cleaning:
This refers to the preprocessing of data in order to remove or reduce noise (by applying smoothing techniques) and the treatment of missingvalues (e.g., by replacing a missing value with the most commonly occurring value for that attribute, or with the most probable value based on statistics).
Although most classification algorithms have some mechanisms for handling noisy or missing data, this step can help reduce confusion during learning.
(ii)Relevance analysis:
Many of the attributes in the data may be redundant.
Correlation analysis can be used to identify whether any two given attributes are statisticallyrelated.
For example, a strong correlation between attributes A1 and A2 would suggest that one of the two could be removed from further analysis.
A database may also contain irrelevant attributes. Attribute subset selection can be used in these cases to find a reduced set of attributes such that the resulting probability distribution of the data classes is as close as possible to the original distribution obtained using all attributes.
Hence, relevance analysis, in the form of correlation analysis and attribute subset selection, can be used to detect attributes that do not contribute to the classification or prediction task.
Such analysis can help improve classification efficiency and scalability.
(iii)Data Transformation And Reduction
The data may be transformed by normalization, particularly when neural networks or methods involving distance measurements are used in the learning step.
Normalization involves scaling all values for a given attribute so that they fall within a small specified range, such as -1 to +1 or 0 to 1.
The data can also be transformed by generalizing it to higher-level concepts. Concept hierarchies may be used for this purpose. This is particularly useful for continuous valued attributes
For example, numeric values for the attribute income can be generalized to discrete ranges, such as low, medium, and high. Similarly, categorical attributes, like street, can be generalized to higher-level concepts, like city.
Data can also be reduced by applying many other methods, ranging from wavelet transformation and principle components analysis to discretization techniques, such as binning, histogram analysis, and clustering.

4. Explain Data Classification with Example

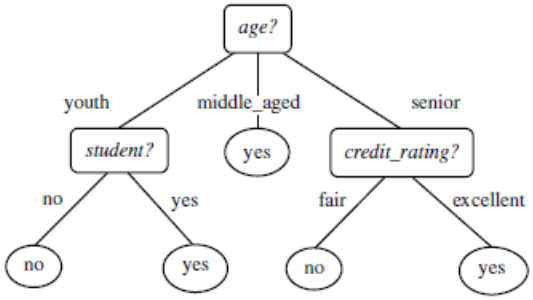
Data classification involves assigning predefined categories (classes) to new observations based on their attributes. It is a type of supervised learning where the model is trained on labeled data.
Steps in Data Classification:
1. Data Collection: Gather a labeled dataset for training.
2. Preprocessing: Clean and prepare the data (e.g., handling missing values, normalizing features).
3. Model Training: Choose a classification algorithm (e.g., decision tree, SVM) and train the model on the training data.
4. Model Evaluation: Evaluate the model using a separate validation set and performance metrics (e.g., accuracy, precision, recall).
5. Prediction: Use the trained model to classify new, unseen data.
Example:
Suppose we want to classify emails as "Spam" or "Not Spam". We collect a dataset of emails with attributes such as the presence of certain keywords, sender address, and email length.
1. Data Collection: A dataset of emails labeled as "Spam" or "Not Spam".
2. Preprocessing: Convert emails into numerical features (e.g., presence of words).
3. Model Training: Train a Naive Bayes classifier on the training data.
4. Model Evaluation: Test the model on a validation set, achieving 95% accuracy.
5. Prediction: Use the model to classify new emails as "Spam" or "Not Spam".



The model learns that emails with the keyword "Win" and short length are more likely to be spam. When a new email E5 with "Yes" for "Win" and "Short" for "Length" is fed into the model, it classifies E5 as "Spam".
These explanations provide detailed insights into each concept, along with examples to illustrate how they work in practice.

5. Explain the classification by Decision tree induction with an example?

Decision tree induction is the learning of decision trees from class-labeled training tuples.
A decision tree is a flowchart-like tree structure, where each internal node (non-leaf node) denotes a test on an attribute, each branch represents an outcome of the test, and each leaf node (or *terminal node*) holds a class label. The topmost node in a tree is the root node.



A typical decision tree is shown in Figure 6.2. It represents the concept *buys computer*, that is, it predicts whether a customer at *All Electronics* is likely to purchase a computer. Internal nodes are denoted by rectangles, and leaf nodes are denoted by ovals. Some decision tree algorithms produce only *binary* trees (where each internal node branches to exactly two other nodes), whereas others can produce nonbinary trees.

"How are decision trees used for classification?" Given a tuple, ***X***, for which the associated class label is unknown, the attribute values of the tuple are tested against the decision tree. A path is traced from the root to a leaf node, which holds the class prediction for that tuple. Decision trees can easily be converted to classification rules.

"Why are decision tree classifiers so popular?" The construction of decision tree classifiers does not require any domain knowledge or parameter setting, and therefore is appropriate for exploratory knowledge discovery. Decision trees can handle high dimensional data. Their representation of acquired knowledge in tree form is intuitive and generally easy to assimilate by humans. The learning and classification steps of decision tree induction are simple and fast. In general, decision tree classifiers have good accuracy. However, successful use may depend on the data at hand. Decision tree induction algorithms have been used for classification in many application areas, such as medicine, manufacturing and production, financial analysis, astronomy, and molecular biology. Decision trees are the basis of several commercial rule induction systems.

During tree construction, *attribute selection measures* are used to select the attribute that best partitions the tuples into distinct classes. When decision trees are built, many of the branches may reflect

noise or outliers in the training data. *Tree pruning* attempts to identify and remove such branches, with the goal of improving classification accuracy on unseen data.

6.Explain distinct values of splitting attribute based on training data in decision tree induction algorithm?

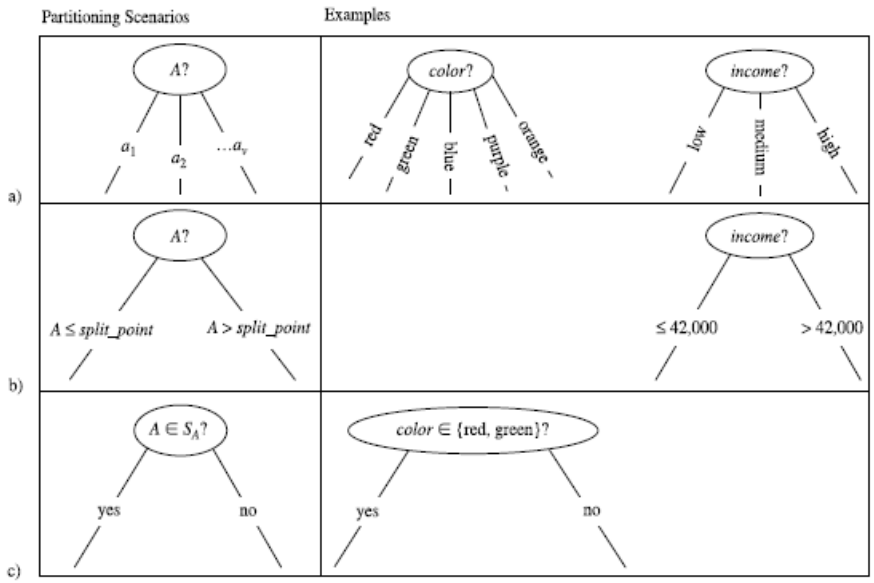
The node *N* is labelled with the splitting criterion, which serves as a test at the node (step 7). A branch is grown from node *N* for each of the outcomes of the splitting criterion. The tuples in *D* are partitioned accordingly (steps 10 to 11). There are three possible scenarios, as illustrated in Figure 6.4. Let *A* be the splitting attribute. *A* has *v* distinct values, f*a*1, *a*2, : : : , *av*g, based on the training data.

1. *A is discrete-valued*: In this case, the outcomes of the test at node *N* correspond directly to the known values of *A*. A branch is created for each known value, *aj*, of *A* and labelled with that value (Figure 6.4(a)). Partition *Dj* is the subset of class-labelled tuples in *D* having value *aj* of *A*. Because all of the tuples in a given partition have the same value for *A*, then *A* need not be considered in any future partitioning of the tuples. Therefore, it is removed from *attribute list*

2. *A is continuous-valued*: In this case, the test at node *N* has two possible outcomes,

corresponding to the conditions *A* _ *split point* and *A* > *split point*, respectively, where *split point* is the split-point returned by *Attribute selection method* as part of the splitting criterion. (In practice, the split-point, *a*, is often taken as the midpoint of two known adjacent values of *A* and therefore may not actually be a pre-existing value of *A* from the training data.) Two branches are grown from *N* and labeled according to the above outcomes (Figure 6.4(b)). The tuples are partitioned such that*D*1 holds the subset of class-labeled tuples in*D*forwhich *A*_*split point*,while *D*2 holds the rest.

3. *A is discrete-valued* and a *binary tree*must be produced (as dictated by the attribute selection measure or algorithm being used): The test at node *N* is of the form "*A* 2 *SA*?". *SA* is the splitting subset for *A*, returned by *Attribute selection method* as part of the splitting criterion. It is a subset of the known values of *A*. If a given tuple has value *aj* of *A* and if *aj* 2 *SA*, then the test at node *N* is satisfied. Two branches are grown from *N*. By convention, the left branch out of *N* is labeled *yes* so that *D*1 corresponds to the subset of class-labeled tuples in *D* that satisfy the test. The right branch out of *N* is labeled *no* so that *D*2 corresponds to the subset of class-labeled tuples from *D* that do not satisfy the test.



7.Explain Bayes Theorem?

Bayes' theorem is named after Thomas Bayes, a nonconformist English clergyman who did early work in probability and decision theory during the 18th century. Let ***X*** be a data tuple. In Bayesian terms, ***X*** is considered "evidence." As usual, it is described by measurements made on a set of *n* attributes. Let *H* be some hypothesis, such as that the data tuple ***X*** belongs to a specified class *C*. For classification problems, we want to determine *P*(*H*j***X***), the probability that the hypothesis *H* holds given the "evidence" or observed data tuple ***X***. In other words, we are looking for the probability that tuple ***X*** belongs to class *C*, given that we know the attribute description of ***X***.

P(*H*j***X***) is the posterior probability, or *a posteriori probability*, of *H* conditioned on ***X***. For example, suppose our world of data tuples is confined to customers described by the attributes *age* and *income*, respectively, and that ***X*** is a 35-year-old customer with an income of $40,000. Suppose that *H* is the hypothesis that our customer will buy a

computer. Then $P(H|X)$ reflects the probability that customer X will buy a computer given that we know the customer's age and income.
In contrast, $P(H)$ is the prior probability, or *a priori probability,* of H. For our example, this is the probability that any given customer will buy a computer, regardless of age, income, or any other information, for that matter. The posterior probability, $P(H|X)$, is based on more information (e.g., customer information) than the prior probability, $P(H)$, which is independent of X.
Similarly, $P(X|H)$ is the posterior probability of X conditioned on H. That is, it is the probability that a customer, X, is 35 years old and earns $40,000, given that we know the customer will buy a computer.
$P(X)$ is the prior probability of$X$.Using our example, it is the probability that a person from our set of customers is 35 years old and earns $40,000.
"How are these probabilities estimated?" $P(H)$, $P(X|H)$, and $P(X)$ may be estimated from the given data, as we shall see below. Bayes' theorem is useful in that it provides a way of calculating the posterior probability, $P(H|X)$, from $P(H)$, $P(X|H)$, and $P(X)$. Bayes' theorem is

$$P(H|X) = \frac{P(X|H)P(H)}{P(X)} : (6.10)$$

8.Explain Naïve Bayesian Classification?

The naïve Bayesian classifier, or simple Bayesian classifier, works as follows:
1. Let D be a training set of tuples and their associated class labels. As usual, each tuple is represented by an n-dimensional attribute vector, $X = (x1, x2, : : : , xn)$, depicting n measurements made on the tuple from n attributes, respectively, $A1, A2, : : : , An$.
2. Suppose that there are m classes, $C1, C2, : : : , Cm$. Given a tuple, X, the classifier will predict that X belongs to the class having the highest posterior probability, conditioned on X. That is, the naïve Bayesian classifier predicts that tuple X belongs to the class Ci if and only if
$P(Ci|X) > P(Cj|X)$ for $1 \_ j \_ m; j 6= i$:
Thus we maximize $P(Ci|X)$. The class Ci for which $P(Ci|X)$ is maximized is called the *maximum posteriori hypothesis*. By Bayes' theorem

$$P(Ci|X) = \frac{P(X|Ci)P(Ci)}{P(X)} :$$

3. As $P(X)$ is constant for all classes, only $P(X|Ci)P(Ci)$ need be maximized. If the class prior probabilities are not known, then it is commonly assumed that the classes are equally likely, that is, $P(C1) = P(C2) = \_ \_ \_ = P(Cm)$, and we would therefore maximize $P(X|Ci)$. Otherwise, we maximize $P(X|Ci)P(Ci)$. Note that the class prior probabilities may be estimated by $P(Ci)=|Ci,D|/|D|$,where $|Ci,D|$ is the number of training tuples of class Ci in D.
4. Given data sets with many attributes, it would be extremely computationally expensive to compute $P(X|Ci)$. In order to reduce computation in evaluating $P(X|Ci)$, the naive assumption of class conditional independence is made. This presumes that the values of the attributes are conditionally independent of one another, given the class label of the tuple (i.e., that there are no dependence relationships among the attributes). Thus,

$$P(X|C_i) = \prod_{k=1}^{n} P(x_k|C_i)$$
$$= P(x_1|C_i) \times P(x_2|C_i) \times \cdots \times P(x_n|C_i).$$

We can easily estimate the probabilities $P(x1jCi)$, $P(x2jCi)$, : : : , $P(xnjCi)$ fromthe training tuples. Recall that here xk refers to the value of attribute Ak for tuple $\boldsymbol{X}$. For each attribute, we look at whether the attribute is categorical or continuous-valued. For instance, to compute $P(\boldsymbol{X}jCi)$, we consider the following:

(a) If Ak is categorical, then $P(xkjCi)$ is the number of tuples of class Ci in D having the value xk for Ak, divided by jCi,Dj, the number of tuples of class Ci in D.

(b) If Ak is continuous-valued, then we need to do a bit more work, but the calculation is pretty straightforward. A continuous-valued attribute is typically assumed to have a Gaussian distribution with a mean μ and standard deviation □, defined by

$$g(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}},$$

$$P(x_k|C_i) = g(x_k, \mu_{C_i}, \sigma_{C_i}).$$

so that

$P(xkjCi) = g(xk, \mu Ci , □Ci)$.

These equations may appear daunting, but hold on!We need to compute μCi and □Ci , which are the mean (i.e., average) and standard deviation, respectively, of the values of attribute Ak for training tuples of class Ci. We then plug these two quantities into Equation (6.13), together with xk, in order to estimate $P(xkjCi)$.

For example, let $\boldsymbol{X}$ = (35, $40,000), where $A1$ and $A2$ are the attributes *age* and *income*, respectively. Let the class label attribute be *buys computer*. The associated class label for $\boldsymbol{X}$ is *yes* (i.e., *buys computer* = *yes*). Let's suppose that *age* has not been discretized and therefore exists as a continuous-valued attribute. Suppose that from the training set, we find that customers in D who buy a computer are 38_12 years of age. In other words, for attribute *age* and this class, we have μ = 38 years and □□= 12. We can plug these quantities, along with $x1$ = 35 for our tuple $\boldsymbol{X}$ into Equation in order to estimate *P(age = 35jbuys computer = yes)*. For a quick review of mean and standard deviation calculations, please see Section 2.2.

5. In order to predict the class label of $\boldsymbol{X}$, $P(\boldsymbol{X}jCi)P(Ci)$ is evaluated for each class Ci. The classifier predicts that the class label of tuple $\boldsymbol{X}$ is the class Ci if and only if $P(\boldsymbol{X}jCi)P(Ci) > P(\boldsymbol{X}jCj)P(Cj)$ for $1 \_ j \_ m, j 6= i$.

In other words, the predicted class label is the class Ci for which $P(\boldsymbol{X}jCi)P(Ci)$ is the maximum.

"How effective are Bayesian classifiers?" Various empirical studies of this classifier in comparison to decision tree and neural network classifiers have found it to be comparable in some domains. In theory, Bayesian classifiers have the minimum error rate in comparison to all other classifiers. However, in practice this is not always the case, owing to inaccuracies in the assumptions made for its use, such as class conditional independence, and the lack of available probability data.

Bayesian classifiers are also useful in that they provide a theoretical justification for other classifiers that do not explicitly use Bayes' theorem. For example, under certain assumptions, it can be shown that many neural network and curve-fitting algorithms output the *maximum posteriori* hypothesis, as does the naïve Bayesian classifier.

9. Write a Note on Bayesian Belief Network

Bayesian Belief Network:

A Bayesian Belief Network (BBN), also known as a Bayesian Network or Belief Network, is a probabilistic graphical model that represents a set of variables and their conditional dependencies via a directed acyclic graph (DAG). Each node in the network represents a random variable, and the edges between the nodes represent conditional dependencies. BBNs are used to model uncertainty in complex systems and are particularly useful for reasoning under uncertainty and making predictions.

Components of BBN:

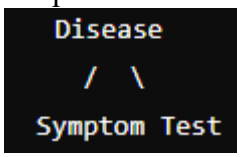
1. Nodes: Represent random variables which can be discrete or continuous.
2. Edges: Directed edges between nodes represent conditional dependencies. An edge from node A to node B indicates that B is conditionally dependent on A.
3. Conditional Probability Tables (CPTsEach node has an associated CPT that quantifies the effect of the parent nodes on the node. For a node XX with parents $P1,P2,...,PnP1,P2,...,Pn$, the CPT provides $P(X|P1,P2,...,Pn)P(X|P1,P2,...,Pn)$.

Example:

Consider a simple medical diagnosis scenario with three variables: "Disease" (D), "Symptom" (S), and "Test Result" (T). The relationships can be represented as follows:

- D → S
- D → T

The BBN indicates that both the symptom and the test result are conditionally dependent on the presence of the disease.



CPT Example:

For the "Symptom" node, the CPT might look like this:

Disease	Symptom (Yes)	Symptom (No)
Yes	0.9	0.1
No	0.2	0.8

This table indicates that if the disease is present, the probability of showing the symptom is 0.9, and if the disease is not present, the probability of showing the symptom is 0.2.

10. Write the Steps Involved in Training Bayesian Belief Network

Steps Involved in Training a Bayesian Belief Network:

1. Structure Learning:
   - Determine the network structure (i.e., the DAG that represents the conditional dependencies between variables). This can be done through:
     - Expert Knowledge: Incorporate domain expertise to define the structure.
     - Data-Driven Methods: Use algorithms like the Hill-Climbing algorithm, Greedy Search, or the PC algorithm to learn the structure from data.
2. Parameter Learning:

- Once the structure is defined, learn the parameters of the network (i.e., the CPTs for each node). Parameter learning can be done using:
  - Maximum Likelihood Estimation (MLE): Estimate parameters that maximize the likelihood of the observed data.
  - Bayesian Estimation: Use prior distributions over parameters and update them based on observed data.

3. Model Validation:
- Validate the learned network using techniques like cross-validation or hold-out validation to ensure the model's predictive accuracy and robustness.

4. Inference:
- Use the trained BBN to perform probabilistic inference. This involves computing the posterior probabilities of certain variables given evidence. Inference methods include:
  - Exact Inference: Algorithms like Variable Elimination or Junction Tree Algorithm.
  - Approximate Inference: Algorithms like Markov Chain Monte Carlo (MCMC) or Loopy Belief Propagation.

Example:
For our medical diagnosis example, the steps might involve:
- Structure Learning: Using patient data to determine the relationships between "Disease," "Symptom," and "Test Result."
- Parameter Learning: Estimating the probabilities in the CPTs from historical patient data.
- Model Validation: Checking the model against a separate validation set to ensure it correctly predicts the disease given symptoms and test results.
- Inference: Using the network to infer the probability of having the disease given a new patient's symptoms and test results.

11. Explain IF-THEN Rules for Classification

IF-THEN Rules for Classification:
IF-THEN rules are a common method used for classification in rule-based systems. These rules take the form of "IF condition THEN outcome," where the condition specifies the criteria based on feature values and the outcome specifies the class label.
Components of IF-THEN Rules:
1. Antecedent (IF part): Specifies the condition based on one or more attributes.
2. Consequent (THEN part): Specifies the class label assigned if the condition is met.

Example:
Consider a dataset for classifying whether a person is eligible for a loan based on income and credit score. An IF-THEN rule might be:
- Rule 1: IF Income > 50,000 AND Credit Score > 700 THEN Class = Eligible
- Rule 2: IF Income ≤ 50,000 AND Credit Score > 700 THEN Class = Review
- Rule 3: IF Credit Score ≤ 700 THEN Class = Not Eligible

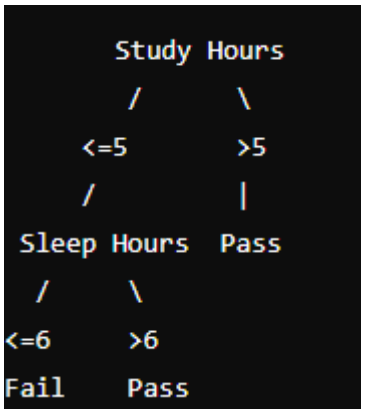
Advantages:
- Interpretability: The rules are easy to understand and interpret by humans.
- Simplicity: Each rule is a simple conditional statement, making the model easy to explain.

Example Use Case:
In a medical diagnosis system, an IF-THEN rule might be:
- Rule: IF Temperature > 100 AND Cough = Yes THEN Diagnosis = Flu

This rule indicates that if a patient has a temperature greater than 100 degrees and a cough, they are diagnosed with the flu.

12. Explain Rule Extraction from Decision Tree

Rule Extraction from Decision Tree:
Rule extraction from a decision tree involves translating the tree structure into a set of IF-THEN rules. Each path from the root node to a leaf node can be converted into a rule.
Steps for Rule Extraction:
1. Traverse the Tree: Start from the root node and traverse to each leaf node.
2. Formulate Conditions: For each path from the root to a leaf, formulate the conditions based on the decisions made at each node.
3. Create Rule: Combine the conditions into an IF-THEN rule, where the IF part contains the conditions and the THEN part contains the class label assigned at the leaf node.
Example:
Consider the decision tree for predicting if a student passes based on study hours and sleep hours:



Extracted Rules:
1. IF Study Hours > 5 THEN Class = Pass
2. IF Study Hours ≤ 5 AND Sleep Hours ≤ 6 THEN Class = Fail
3. IF Study Hours ≤ 5 AND Sleep Hours > 6 THEN Class = Pass
Advantages:
- Transparency: Rules derived from decision trees are easy to interpret.
- Ease of Use: Rules can be easily implemented in rule-based systems.
Example Use Case:
In customer segmentation, a decision tree might classify customers based on purchase history and demographic information. The rules extracted from the tree can help in targeting marketing strategies:
- Rule 1: IF Age > 30 AND Purchase History = High THEN Segment = Premium Customer
- Rule 2: IF Age ≤ 30 AND Purchase History = Low THEN Segment = Budget Customer
These rules provide clear guidelines for categorizing customers based on their attributes.

13. Explain Sequential covering algorithm.

IF-THEN rules can be extracted directly from the training data (i.e., without having to generate a decision tree first) using a **sequential covering algorithm**. The name comes from the notion that the rules are learned *sequentially* (one at a time), where each rule for a given class will ideally *cover* many of the tuples of that class (and hopefully none of the tuples of other classes). Sequential covering algorithms are the most widely used approach to mining disjunctive sets of classification rules, and form the topic of this

subsection. Note that in a newer alternative approach, classification rules can be generated using *associative classification algorithms*, which search for attribute-value pairs that occur frequently in the data. These pairs may form association rules, which can be analyzed and used in classification.

There are many sequential covering algorithms. Popular variations include AQ, CN2, and the more recent, RIPPER. The general strategy is as follows. Rules are learned one at a time. Each time a rule is learned, the tuples covered by the rule are removed, and the process repeats on the remaining tuples. This sequential learning of rules is in contrast to decision tree induction. Because the path to each leaf in a decision tree corresponds to a rule, we can consider decision tree induction as learning a set of rules *simultaneously*.

A basic sequential covering algorithm is shown in Figure 6.12. Here, rules are learned for one class at a time. Ideally, when learning a rule for a class, *Ci*, we would like the rule to cover all (or many) of the training tuples of class *C* and none (or few) of the tuples from other classes. In this way, the rules learned should be of high accuracy. The rules need not necessarily be of high coverage. This is because we can have more than one Algorithm: Sequential covering. Learn a set of IF-THEN rules for classification.
Input:
D, a data set class-labeled tuples;
Att vals, the set of all attributes and their possible values.
Output: A set of IF-THEN rules.
Method:
(1) *Rule set* = fg; // initial set of rules learned is empty
(2) for each class *c* do
(3) repeat
(4) Rule = Learn One Rule(*D*, *Att vals*, *c*);
(5) remove tuples covered by *Rule* from *D*;
(6) until terminating condition;
(7) *Rule set* = *Rule set* +*Rule*; // add new rule to rule set
(8) endfor
(9) return *Rule Set*;
Figure 6.12 Basic sequential covering algorithm.rule for a class, so that different rules may cover different tuples within the same class.
The process continues until the terminating condition is met, such as when there are no more training tuples or the quality of a rule returned is below a user-specified threshold. The *Learn One Rule* procedure finds the "best" rule for the current class, given the current set of training tuples.

"How are rules learned?" Typically, rules are grown in a *general-to-specific* manner (Figure 6.13). We can think of this as a beam search, where we start off with an empty rule and then gradually keep appending attribute tests to it. We append by adding the attribute test as a logical conjunct to the existing condition of the rule antecedent. Suppose our training set, *D*, consists of loan application data. Attributes regarding each applicant include their age, income, education level, residence, credit rating, and the term of the loan. The classifying attribute is *loan decision*, which indicates whether a loan is accepted (considered safe) or rejected (considered risky). To learn a rule for the

class “accept,” we start off with the most general rule possible, that is, the condition of the rule antecedent is empty. The rule is:
IF THEN *loan decision = accept*.
We then consider each possible attribute test that may be added to the rule. These can be derived from the parameter *Att vals*, which contains a list of attributes with their associated values. For example, for an attribute-value pair (*att*, *val*), we can consider

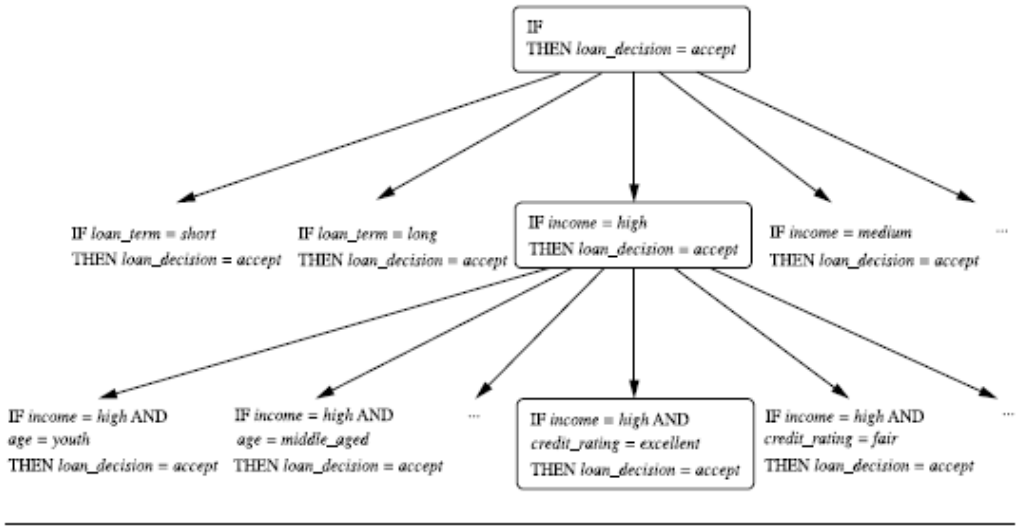


Figure 6.13 A general-to-specific search through rule space.

attribute tests such as *att = val*, *att _ val*, *att > val*, and so on. Typically, the training data will contain many attributes, each of which may have several possible values. Finding an optimal rule set becomes computationally explosive. Instead, *Learn One Rule* adopts a greedy depth-first strategy. Each time it is faced with adding a new attribute test (conjunct) to the current rule, it picks the one that most improves the rule quality, based on the training samples. We will say more about rule quality measures in a minute. For the moment, let’s say we use rule accuracy as our quality measure. Getting back to our example with Figure 6.13, suppose *Learn One Rule* finds that the attribute test *income = high* best improves the accuracy of our current (empty) rule. We append it to the condition, so that the current rule becomes
IF *income = high* THEN *loan decision = accept*.
Each time we add an attribute test to a rule, the resulting rule should cover more of the “accept” tuples. During the next iteration, we again consider the possible attribute tests and end up selecting *credit rating = excellent*. Our current rule grows to become
IF *income = high* AND *credit rating = excellent* THEN *loan decision = accept*.
The process repeats, where at each step, we continue to greedily grow rules until the resulting rule meets an acceptable quality level.
Greedy search does not allow for backtracking. At each step, we *heuristically* add what appears to be the best choice at the moment. What if we unknowingly made a poor choice along the way? To lessen the chance of this happening, instead of selecting the best attribute test to append to the current rule, we can select the best *k* attribute tests. In this way, we perform a beam search of width *k* wherein we maintain the *k* best candidates overall at each step, rather than a single best candidate.

14. Explain K-nearest neighbours classification.

The *k*-nearest-neighbor method was first described in the early 1950s. The method is labor intensive when given large training sets, and did not gain popularity until the 1960s when increased computing power became available. It has since been widely used in the area of pattern recognition.

Nearest-neighbor classifiers are based on learning by analogy, that is, by comparing a given test tuple with training tuples that are similar to it. The training tuples are described by *n* attributes. Each tuple represents a point in an *n*-dimensional space. In this way, all of the training tuples are stored in an *n*-dimensional pattern space. When given an unknown tuple, a ***k***-nearest-neighbor classifier searches the pattern space for the *k* training tuples that are closest to the unknown tuple. These *k* training tuples are the *k* "nearest neighbors" of the unknown tuple.

"Closeness" is defined in terms of a distance metric, such as Euclidean distance. The Euclidean distance between two points or tuples, say, $\boldsymbol{X}1 = (x11, x12, : : : , x1n)$ and $\boldsymbol{X}2 = (x21, x22, : : : , x2n)$, is

$$dist(\boldsymbol{X_1}, \boldsymbol{X_2}) = \sqrt{\sum_{i=1}^{n} (x_{1i} - x_{2i})^2}. \qquad (6.45)$$

In other words, for each numeric attribute, we take the difference between the corresponding values of that attribute in tuple ***X*1** and in tuple ***X*2**, square this difference, and accumulate it. The square root is taken of the total accumulated distance count. Typically, we normalize the values of each attribute before using Equation (6.45). This helps prevent attributes with initially large ranges (such as *income*) from outweighing attributes with initially smaller ranges (such as binary attributes). Min-max normalization, for example, can be used to transform a value *v* of a numeric attribute *A* to *v*0 in the range [0, 1] by computing

$$v' = \frac{v - min_A}{max_A - min_A}, \qquad (6.46)$$

where *minA* and *maxA* are the minimum and maximum values of attribute *A*. Chapter 2 describes other methods for data normalization as a form of data transformation.

For *k*-nearest-neighbor classification, the unknown tuple is assigned the most common class among its *k* nearest neighbors. When $k = 1$, the unknown tuple is assigned the class of the training tuple that is closest to it in pattern space. Nearest neighbor classifiers can also be used for prediction, that is, to return a real-valued prediction for a given unknown tuple. In this case, the classifier returns the average value of the real-valued labels associated with the *k* nearest neighbors of the unknown tuple.

"But how can distance be computed for attributes that not numeric, but categorical, such as color?" The above discussion assumes that the attributes used to describe the tuples are all numeric. For categorical attributes, a simple method is to compare the corresponding value of the attribute in tuple ***X*1** with that in tuple ***X*2**. If the two are identical (e.g., tuples ***X*1** and ***X*2** both have the color blue), then the difference between the two is taken as 0. If the two are different (e.g., tuple ***X*1** is blue but tuple ***X*2** is red), then the difference is considered to be 1. Other methods may incorporate more sophisticated schemes for differential grading (e.g., where a larger difference score is assigned, say, for blue and

white than for blue and black).
"What about missing values?" In general, if the value of a given attribute *A* is missing in tuple ***X*1** and/or in tuple ***X*2**, we assume the maximum possible difference. Suppose that each of the attributes have been mapped to the range [0, 1]. For categorical attributes, we take the difference value to be 1 if either one or both of the corresponding values of *A* are missing. If *A* is numeric and missing from both tuples ***X*1** and ***X*2**, then the difference is also taken to be 1. If only one value is missing and the other (which we'll call *v*0) is present and normalized, then we can take the difference to be either j1 □ *v*0j or j0 □ *v*0j (i.e., 1 □ *v*0 or *v*0), whichever is greater.
"How can I determine a good value for k, the number of neighbors?" This can be determined experimentally. Starting with $k = 1$, we use a test set to estimate the error rate of the classifier. This process can be repeated each time by incrementing *k* to allow for one more neighbor. The *k* value that gives the minimum error rate may be selected. In general, the larger the number of training tuples is, the larger the value of *k* will be (so that classification and prediction decisions can be based on a larger portion of the stored tuples). As the number of training tuples approaches infinity and $k = 1$, the error rate can be no worse then twice the Bayes error rate (the latter being the theoretical minimum). If *k* also approaches infinity, the error rate approaches the Bayes error rate.
Nearest-neighbor classifiers use distance-based comparisons that intrinsically assign equal weight to each attribute. They therefore can suffer from poor accuracy when given noisy or irrelevant attributes. The method, however, has been modified to incorporate attribute weighting and the pruning of noisy data tuples. The choice of a distance metric can be critical. The Manhattan (city block) distance (Section 7.2.1), or other distance measurements, may also be used.
Nearest- neighbor classifiers can be extremely slow when classifying test tuples. If *D* is a training database of j*D*j tuples and $k = 1$, then *O*(j*D*j) comparisons are required in order to classify a given test tuple. By pre sorting and arranging the stored tuples into search trees, the number of comparisons can be reduced to *O*(*log* (j*D*j). Parallel implementation can reduce the running time to a constant, that is *O*(1), which is independent of j*D*j. Other techniques to speed up classification time include the use of *partial distance* calculations and *editing* the stored tuples. In the partial distance method, we compute the distance based on a subset of the *n* attributes. If this distance exceeds a threshold, then further computation for the given stored tuple is halted, and the process moves on to the next stored tuple. The editing method removes training tuples that prove useless. This method is also referred to as pruning or condensing because it reduces the total number of tuples stored.

15. Explain linear regression.

Straight-line regression analysis involves a response variable, y, and a single predictor variable, x. It is the simplest form of regression, and models y as a linear function of x. That is,

$$y = b + wx, \tag{6.48}$$

where the variance of y is assumed to be constant, and b and w are **regression coefficients** specifying the Y-intercept and slope of the line, respectively. The regression coefficients, w and b, can also be thought of as weights, so that we can equivalently write,

$$y = w_0 + w_1 x. \tag{6.49}$$

These coefficients can be solved for by the **method of least squares**, which estimates the best-fitting straight line as the one that minimizes the error between the actual data and the estimate of the line. Let D be a training set consisting of values of predictor variable, x, for some population and their associated values for response variable, y. The training set contains $|D|$ data points of the form $(x_1, y_1), (x_2, y_2), \ldots, (x_{|D|}, y_{|D|})$.[12] The regression coefficients can be estimated using this method with the following equations:

$$w_1 = \frac{\sum_{i=1}^{|D|}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{|D|}(x_i - \bar{x})^2} \tag{6.50}$$

[12]Note that earlier, we had used the notation $(\boldsymbol{X}_i, y_i)$ to refer to training tuple i having associated class label y_i, where $\boldsymbol{X}_i$ was an attribute (or feature) *vector* (that is, $\boldsymbol{X}_i$ was described by more than one attribute). Here, however, we are dealing with just one predictor variable. Since the $\boldsymbol{X}_i$ here are one-dimensional, we use the notation x_i over $\boldsymbol{X}_i$ in this case.

$$w_0 = \bar{y} - w_1 \bar{x} \tag{6.51}$$

where $\bar{x}$ is the mean value of $x_1, x_2, \ldots, x_{|D|}$, and $\bar{y}$ is the mean value of $y_1, y_2, \ldots, y_{|D|}$. The coefficients w_0 and w_1 often provide good approximations to otherwise complicated regression equations.

Example 6.11 **Straight-line regression using the method of least squares.** Table 6.7 shows a set of paired data where x is the number of years of work experience of a college graduate and y is the

Table 6.7 Salary data.

x years experience	*y salary (in $1000s)*
3	30
8	57
9	64
13	72
3	36
6	43
11	59
21	90
1	20
16	83

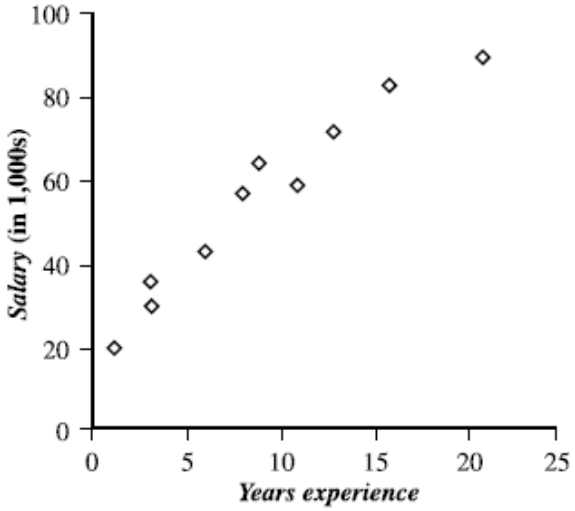


Figure 6.26 Plot of the data in Table 6.7 for Example 6.11. Although the points do not fall on a straight line, the overall pattern suggests a linear relationship between x (*years experience*) and y (*salary*).

corresponding salary of the graduate. The 2-D data can be graphed on a *scatter plot*, as in Figure 6.26. The plot suggests a linear relationship between the two variables, x and y. We model the relationship that salary may be related to the number of years of work experience with the equation $y = w_0 + w_1 x$.

Given the above data, we compute $\bar{x} = 9.1$ and $\bar{y} = 55.4$. Substituting these values into Equations (6.50) and (6.51), we get

$$w_1 = \frac{(3-9.1)(30-55.4)+(8-9.1)(57-55.4)+\cdots+(16-9.1)(83-55.4)}{(3-9.1)^2+(8-9.1)^2+\cdots+(16-9.1)^2} = 3.5$$

$$w_0 = 55.4-(3.5)(9.1) = 23.6$$

Thus, the equation of the least squares line is estimated by $y = 23.6 + 3.5x$. Using this equation, we can predict that the salary of a college graduate with, say, 10 years of experience is \$58,600. ■

Multiple linear regression is an extension of straight-line regression so as to involve more than one predictor variable. It allows response variable y to be modeled as a linear function of, say, n predictor variables or attributes, $A_1, A_2, \ldots, A_n$, describing a tuple, X. (That is, $X = (x_1, x_2, \ldots, x_n)$.) Our training data set, D, contains data of the form $(X_1, y_1), (X_2, y_2), \ldots, (X_{|D|}, y_{|D|})$, where the X_i are the n-dimensional training tuples with associated class labels, y_i. An example of a multiple linear regression model based on two predictor attributes or variables, A_1 and A_2, is

$$y = w_0 + w_1 x_1 + w_2 x_2, \tag{6.52}$$

where x_1 and x_2 are the values of attributes A_1 and A_2, respectively, in X. The method of least squares shown above can be extended to solve for w_0, w_1, and w_2. The equations, however, become long and are tedious to solve by hand. Multiple regression problems are instead commonly solved with the use of statistical software packages, such as SAS, SPSS, and S-Plus (see references above.)

16. Explain nonlinear regression.

"How can we model data that does not show a linear dependence? For example, what if a given response variable and predictor variable have a relationship that may be modeled by a polynomial function?" Think back to the straight-line linear regression case above where dependent response variable, y, is modeled as a linear function of a single independent predictor variable, x. What if we can get a more accurate model using a nonlinear model, such as a parabola or some other higher-order polynomial? **Polynomial regression** is often of interest when there is just one predictor variable. It can be modeled by adding polynomial terms to the basic linear model. By applying transformations to the variables, we can convert the nonlinear model into a linear one that can then be solved by the method of least squares.

Example 6.12 **Transformation of a polynomial regression model to a linear regression model.** Consider a cubic polynomial relationship given by

$$y = w_0 + w_1 x + w_2 x^2 + w_3 x^3. \tag{6.53}$$

To convert this equation to linear form, we define new variables:

$$x_1 = x \qquad x_2 = x^2 \qquad x_3 = x^3 \tag{6.54}$$

Equation (6.53) can then be converted to linear form by applying the above assignments, resulting in the equation $y = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3$, which is easily solved by the method of least squares using software for regression analysis. Note that polynomial regression is a special case of multiple regression. That is, the addition of high-order terms like x^2, x^3, and so on, which are simple functions of the single variable, x, can be considered equivalent to adding new independent variables. ■

In Exercise 15, you are asked to find the transformations required to convert a non-linear model involving a power function into a linear regression model.

Some models are intractably nonlinear (such as the sum of exponential terms, for example) and cannot be converted to a linear model. For such cases, it may be possible to obtain least square estimates through extensive calculations on more complex formulae.

Various statistical measures exist for determining how well the proposed model can predict y. These are described in Section 6.12.2. Obviously, the greater the number of predictor attributes is, the slower the performance is. Before applying regression analysis, it is common to perform attribute subset selection (Section 2.5.2) to eliminate attributes that are unlikely to be good predictors for y. In general, regression analysis is accurate for prediction, except when the data contain outliers. Outliers are data points that are highly inconsistent with the remaining data (e.g., they may be way out of the expected value range). Outlier detection is discussed in Chapter 7. Such techniques must be used with caution, however, so as not to remove data points that are valid, although they may vary greatly from the mean.

17. Explain the requirements of clustering in data mining.

Clustering in data mining involves organizing data points into groups, or clusters, such that points within the same cluster are more similar to each other than to points in other clusters. The main requirements for effective clustering are:

1. Scalability: The algorithm must handle large datasets efficiently.
   - Example: A social media company clustering billions of user interactions to identify patterns needs an algorithm that can process data at this scale without excessive computational cost.
2. Ability to deal with different types of attributes: The algorithm should manage various data types, such as numerical, categorical, and binary attributes.
   - Example: A retail company clustering customer data might have numerical attributes (age, income), categorical attributes (gender, location), and binary attributes (membership status).
3. Discovery of clusters with arbitrary shape: Clusters in real-world data can take on various shapes, not just spherical.
   - Example: In image processing, clustering pixels in an image can result in clusters with complex shapes corresponding to objects in the image.

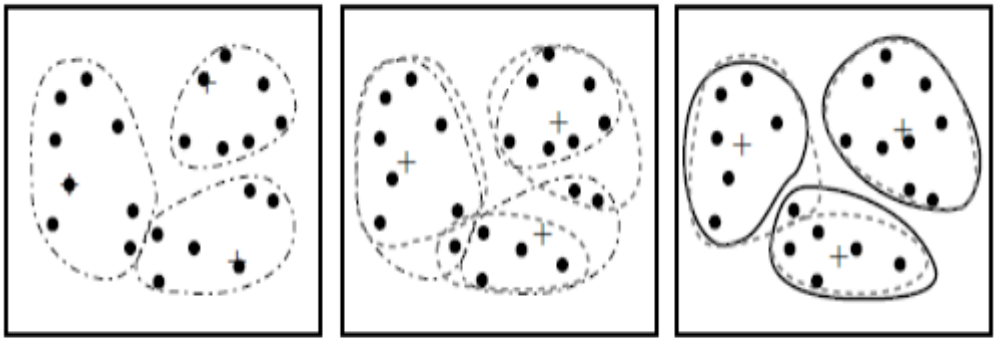
4. Minimal domain knowledge: The algorithm should require minimal input parameters and user intervention.
- Example: An unsupervised learning task, such as customer segmentation, where the exact number of segments is not known beforehand.
5. Handling noisy data: Real-world data often contains noise and outliers; the algorithm should be robust to such anomalies.
- Example: Sensor data in environmental monitoring might have errors or spikes due to sensor malfunction.
6. Interpretability and usability: The clustering results should be easy to interpret and actionable.
- Example: Clusters of website visitors that help a marketing team to tailor campaigns based on user behavior patterns.
Illustration: Imagine a retail company wants to segment its customers based on their shopping behavior. The dataset includes millions of transactions with various attributes such as purchase amount, frequency of visits, and product categories. The chosen clustering algorithm needs to process this large dataset efficiently, handle different data types, discover clusters of different shapes, be robust to noisy data, and provide interpretable results to inform marketing strategies.

18. Explain K-means Partition Algorithm.

The k-means algorithm takes the input parameter, k, and partitions a set of n objects intok clusters so that the resulting intracluster similarity is high but the intercluster similarity is low.
Cluster similarity is measured in regard to the mean value of the objects in a cluster, which can be viewed as the cluster's centroid or center of gravity.
The k-means algorithm proceeds as follows
First, it randomly selects k of the objects, each of which initially represents a cluster mean or center. For each of the remaining objects, an object is assigned to the cluster to which it is the most similar, based on the distance between the object and the cluster mean. It then computes the new mean for each cluster. This process iterates until the criterion function converges. Typically, the square-error criterion is used, defined as

$$E = \sum_{i=1}^{k} \sum_{p \in C_i} |p - m_i|^2,$$

Where E is the sum of the square error for all objects in the data set p is the point in space representing a given object mi is the mean of cluster Ci
Example: Clustering customers based on their spending patterns:
1. Suppose we have customer spending data for four customers: [100, 200, 300, 400]. We want to cluster them into two groups (K=2).
2. We randomly choose two initial centroids, say 100 and 400.
3. We assign each customer to the nearest centroid: [100] and [200, 300, 400].
4. We calculate new centroids: the mean of [100] is 100, and the mean of [200, 300, 400] is 300.
5. We repeat the assignment: [100] and [200, 300, 400].
6. We update the centroids: the centroids remain the same, so the algorithm stops.
The final clusters are [100] and [200, 300, 400], with centroids at 100 and 300.



Clustering of a set of objects based on the *k*-means method

19. Explain K-medoid Partition Algorithm.

The K-medoid algorithm is similar to K-means but is more robust to noise and outliers. It uses actual data points (medoids) as cluster centers.

1. Initialization: Select K representative objects (medoids) randomly.
2. Assignment: Assign each data point to the nearest medoid.
3. Update: For each cluster, choose the data point that minimizes the sum of dissimilarities to all other points in the cluster as the new medoid.
4. Repeat: Repeat the assignment and update steps until the medoids stabilize.

$$E = \sum_{j=1}^{k} \sum_{p \in C_j} |p - o_j|,$$

whereE is the sum of the absolute error for all objects in the data set pis the point inspace representing a given object in clusterCj ojis the representative object of Cj

Algorithm

Input:

- *k*: the number of clusters,
- *D*: a data set containing *n* objects.

Output: A set of *k* clusters.

Method:

(1) arbitrarily choose *k* objects in *D* as the initial representative objects or seeds;
(2) **repeat**
(3) assign each remaining object to the cluster with the nearest representative object;
(4) randomly select a nonrepresentative object, o_{random};
(5) compute the total cost, *S*, of swapping representative object, o_j, with o_{random};
(6) if $S < 0$ **then** swap o_j with o_{random} to form the new set of *k* representative objects;
(7) **until** no change;

Example: Clustering cities based on travel data:

1. Suppose we have five cities with travel distances between them and we want to cluster them into two groups (K=2). Initial medoids are City A and City B.
2. We assign each city to the nearest medoid based on travel distance.
3. We update the medoids by selecting the city in each cluster that minimizes the sum of travel distances to all other cities in the cluster.
4. We repeat the assignment and update steps until the medoids stabilize.
The final clusters could be: {City A, City C, City D} and {City B, City E}, with medoids at City A and City B.

20. Explain 4 cases of cost function for k-medoid clustering.

The cost function in k-medoid clustering measures the total dissimilarity between points and their assigned medoids. Here are four specific cases:
Case 1: pcurrently belongs to representative object, oj . If ojis replaced by orandomasa representative object and p is closest to one of the other representative objects, oi ,i≠j, then p is reassigned to oi .
Case 2: pcurrently belongs to representative object, oj. If ojis replaced by orandomasa representative object and p is closest to orandom, then p is reassigned to orandom.
Case 3: pcurrently belongs to representative object, oi , i≠j. If ojis replaced by orandomas a representative object and p is still closest to oi , then the assignment does notchange.
Case 4: pcurrently belongs to representative object, oi, i≠j. If ojis replaced byorandomas a representative object and p is closest to orandom, then p is reassigned toorandom
Illustration: Suppose we are clustering delivery locations for a logistics company. We use the k-medoid algorithm to minimize travel distances. The cost function ensures that each cluster is represented by a central location (medoid) that minimizes the overall travel distance for deliveries within that cluster.
Each time a reassignment occurs, a difference in absolute error, E, is contributed to the cost function. Therefore, the cost function calculates the difference in absolute-error value if a current representative object is replaced by a nonrepresentative object. The total cost of swapping is the sum of costs incurred by all nonrepresentative objects. If the total cost is negative, then oj is replaced or swapped with orandom since the actual absolute error E would be reduced. If the total cost is positive, the current representative object, oj , is considered acceptable, and nothing is changed in the iteration. PAM (Partitioning Around Medoids) was one of the first k-medoids algorithms introduced (Figure 7.5). It attempts to determine k partitions for n objects. After an initial random selection of k representative objects, the algorithm repeatedly tries to make a better choice of cluster representatives. All of the possible pairs of objects are analyzed, where one object in each pair is considered a representative object and the other is not. The quality of the resulting clustering is calculated for each such combination. An object, oj , is replaced with the object causing the greatest reduction in error. The set of best objects for each cluster in one iteration forms the representative objects for the next iteration. The final set of representative objects are the respective medoids of the clusters. The complexity of each iteration is $O(k(n-k)^2)$. For large values of n and k, such computation becomes very costly.

21. Write a note on CLARA

CLARA (Clustering Large Applications) is an extension of the k-medoid algorithm designed to handle large datasets. It works by taking multiple samples of the dataset and applying the k-medoid algorithm to each sample.

Steps:
1. Sampling: Draw multiple samples from the dataset.
2. Clustering: Apply the k-medoid algorithm to each sample to identify the medoids.
3. Selection: Evaluate the clustering quality on the full dataset using the medoids from each sample, selecting the best medoids based on a cost function.
Example: Suppose we have a dataset of 10,000 customer transactions and want to cluster them into 5 groups. Instead of clustering all 10,000 transactions at once, CLARA takes multiple random samples (e.g., 100 samples of 200 transactions each), applies k-medoids to each sample, and then evaluates which clustering solution performs best on the entire dataset.

22. Compare Agglomerative and Divisive Hierarchical Clustering.

Agglomerative Hierarchical Clustering:
- Bottom-up approach: Starts with each data point as a single cluster and iteratively merges the closest clusters.
- Steps:
  1. Assign each data point to its own cluster.
  2. Compute the proximity matrix.
  3. Merge the two closest clusters.
  4. Update the proximity matrix.
  5. Repeat until all points are in a single cluster.
- Advantages: Simple to understand and implement, good for small datasets.
- Disadvantages: Computationally expensive for large datasets, requires a dendrogram to determine the number of clusters.
Divisive Hierarchical Clustering:
- Top-down approach: Starts with all data points in one cluster and recursively splits them into smaller clusters.
- Steps:
  1. Assign all data points to a single cluster.
  2. Select a cluster to split based on some criteria.
  3. Split the selected cluster into two sub-clusters.
  4. Repeat until each data point is in its own cluster or a stopping criterion is met.
- Advantages: Can be more efficient for certain types of data, allows for more control over the clustering process.
- Disadvantages: Computationally intensive, requires a method to select which cluster to split.
Example: Suppose we are clustering animal species based on genetic data. Agglomerative clustering would start with each species as its own cluster and merge similar species step-by-step. Divisive clustering would start with all species in one cluster and split them based on genetic dissimilarities.

23. Write a note on BIRCH

BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies) is an efficient clustering algorithm for large datasets. It incrementally constructs a clustering feature tree (CF Tree) while scanning the dataset.
Key Concepts:
- CF Tree: A balanced tree structure that stores summary information about sub-clusters.
- Clustering Features (CF): A triplet (N, LS, SS) where N is the number of data points, LS is the linear sum of the points, and SS is the square sum of the points.

$$x_0 = \frac{\sum_{i=1}^{n} x_i}{n}$$

$$R = \sqrt{\frac{\sum_{i=1}^{n} (x_i - x_0)^2}{n}}$$

$$D = \sqrt{\frac{\sum_{i=1}^{n} \sum_{j=1}^{n} (x_i - x_j)^2}{n(n-1)}}$$

where R is the average distance from member objects to the centroid, and D is the average pairwise distance within a cluster. Both R and D reflect the tightness of the cluster around the centroid. A clustering feature (CF) is a three-dimensional vector summarizing information about clusters of objects. Given n d-dimensional objects or points in a cluster, {xi}, then the CF of the cluster is defined as

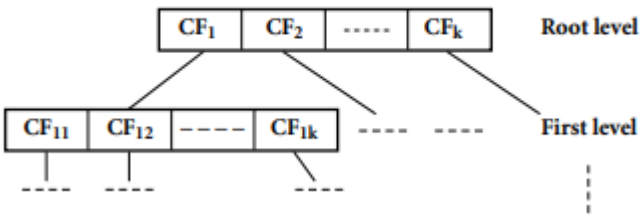
$$CF = \langle n, \boldsymbol{LS}, \boldsymbol{SS} \rangle,$$

where n is the number of points in the cluster, $\boldsymbol{LS}$ is the linear sum of the n points (i.e., $\sum_{i=1}^{n} x_i$), and $\boldsymbol{SS}$ is the square sum of the data points (i.e., $\sum_{i=1}^{n} x_i^2$).

Steps:

1. Build CF Tree: Insert data points into the CF Tree.
2. Condense CF Tree: Optionally condense the tree to fit into memory.
3. Global Clustering: Apply a global clustering algorithm to the leaf entries of the CF Tree.

Example: Suppose that there are three points, (2, 5), (3, 2), and (4, 3), in a cluster, C1. The clustering feature of C1 is CF1 = h3,(2+3+4,5+2+3),(2 2 +3 2 +4 2 ,5 2 +2 2 +3 2)i = h3,(9,10),(29,38)i. Suppose thatC1 is disjoint to a second cluster,C2, whereCF2 = h3, (35, 36), (417, 440)i. The clustering feature of a new cluster,C3, that is formed by mergingC1 andC2, is derived by adding CF1 and CF2. That is, CF3 = h3+3,(9+35,10+36),(29+417,38+440)i = h6,(44,46),(446,478)i.



24. Write a note on ROCK.

ROCK (Robust Clustering using Links) is a hierarchical clustering algorithm that merges clusters based on the number of shared neighbors (links) between data points.

Key Concepts:

- Links: The number of common neighbors between pairs of points.
- Threshold: A user-defined parameter that determines the minimum number of links required to merge clusters.

ROCK's concepts of neighbors and links are illustrated in the following example, where the similarity between two "points" or transactions, Ti and Tj , is defined with the Jaccard coefficient as

$$sim(T_i, T_j) = \frac{|T_i \cap T_j|}{|T_i \cup T_j|}.$$

ROCK's concepts of neighbors and links are illustrated in the following example, where the similarity between two "points" or transactions, Ti and Tj , is defined with the Jaccard coefficient as

Steps:

1. Compute Links: Calculate the number of links for each pair of data points.
2. Merge Clusters: Use a hierarchical approach to merge clusters based on link similarity.
3. Refinement: Optionally refine the clusters using additional criteria.

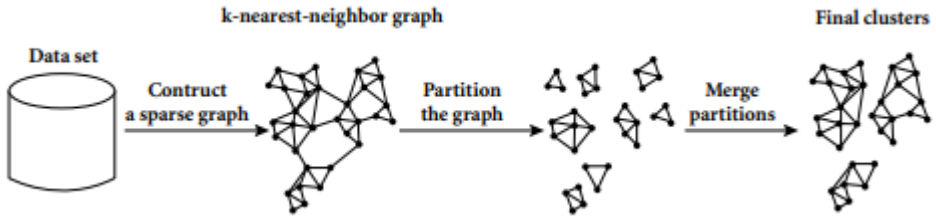
Example: Suppose we are analyzing purchasing behavior in an online store. ROCK calculates the number of common items bought by pairs of customers (links) and merges customers into clusters if they share many common purchases, identifying groups with similar shopping habits.

25. Write a note on Chameleon.

Chameleon is a hierarchical clustering algorithm that uses dynamic modeling to adaptively merge clusters based on their relative closeness and interconnectivity.

Key Concepts:

- Two-Phase Approach:
  1. Graph Partitioning: Initially, partition the dataset into many small clusters using a graph-partitioning algorithm.
  2. Dynamic Modeling: Merge clusters using a dynamic model that considers both the inter-connectivity (how well the points within the clusters are connected) and the relative closeness (proximity between the clusters).
- Self-Adaptive: The algorithm self-adapts to the internal characteristics of the clusters.



The relative interconnectivity, RI(Ci , Cj), between two clusters,Ci andCj , is defined as the absolute interconnectivity between Ci and Cj , normalized with respect to the internal interconnectivity of the two clusters, Ci and Cj . That is,

$$RI(C_i, C_j) = \frac{|EC_{\{C_i, C_j\}}|}{\frac{1}{2}(|EC_{C_i}| + |EC_{C_j}|)},$$

where EC{Ci , Cj} is the edge cut, defined as above, for a cluster containing bothCi and Cj . Similarly, ECCi (or ECCj) is the minimum sum of the cut edges that partition Ci (or Cj) into two roughly equal parts. The relative closeness, RC(Ci , Cj), between a pair of clusters,Ci

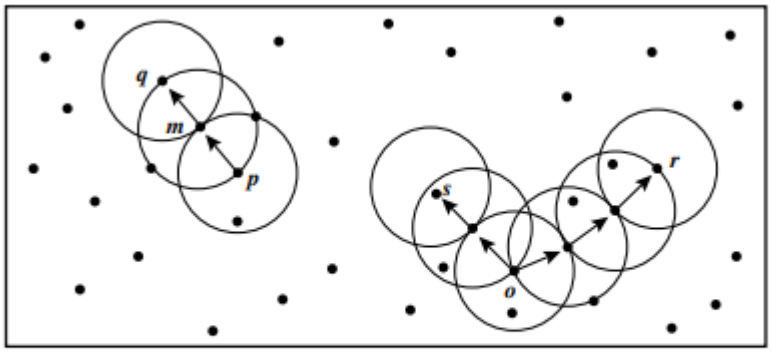
andCj , is the absolute closeness between Ci and Cj , normalized with respect to the internal closeness of the two clusters, Ci and Cj . It is defined as

$$RC(C_i, C_j) = \frac{\overline{S}_{EC_{\{C_i, C_j\}}}}{\frac{|C_i|}{|C_i|+|C_j|}\overline{S}_{EC_{C_i}} + \frac{|C_j|}{|C_i|+|C_j|}\overline{S}_{EC_{C_j}}},$$

Example: Suppose we are clustering images based on visual features. Chameleon first partitions the images into small groups based on feature similarity, then merges these groups based on both how well the features within each group are connected and how close the groups are to each other, resulting in meaningful image clusters.

26. Explain DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm that identifies clusters based on the density of points in the data space.
Key Concepts:
- Density Reachability: Points within a specified distance (ε) are considered to be in the same neighborhood.
- Core Points: Points with at least a minimum number of neighbors (MinPts) within the ε radius.
- Border Points: Points that are within the ε radius of a core point but do not have enough neighbors to be core points themselves.
- Noise Points: Points that are not reachable from any core points.



10 Density reachability and density connectivity in density-based clustering. Based on [EKSX96].

Consider Figure 7.10 for a given ε represented by the radius of the circles, and, say, let MinPts = 3. Based on the above definitions: Of the labeled points, m, p, o, and r are core objects because each is in an ε-neighborhood containing at least three points. q is directly density-reachable from m. m is directly density-reachable from p and vice versa. q is (indirectly) density-reachable from p because q is directly density-reachable from m and m is directly density-reachable from p. However, p is not density-reachable from q because q is not a core object. Similarly, r and s are density-reachable from o, and o is density-reachable from r. o, r, and s are all density-connected.
Steps:
1. Select an arbitrary point: If it is a core point, form a cluster.
2. Expand the cluster: Include all points density-reachable from the core point.

3. Repeat: Process all points to form clusters or label them as noise.
Example: Suppose we have GPS data of taxi rides in a city. DBSCAN can identify clusters of high activity (areas with many rides, such as downtown) while marking isolated rides as noise (e.g., rides to remote areas).

27. Explain OPTICS

OPTICS (Ordering Points To Identify the Clustering Structure) is an extension of DBSCAN that creates an augmented ordering of the database to represent its density-based clustering structure.
Key Concepts:
- Core Distance: The minimum radius that includes MinPts points.
- Reachability Distance: The smallest radius required to reach a point from another point.
- Ordering: The dataset is ordered based on reachability distances.

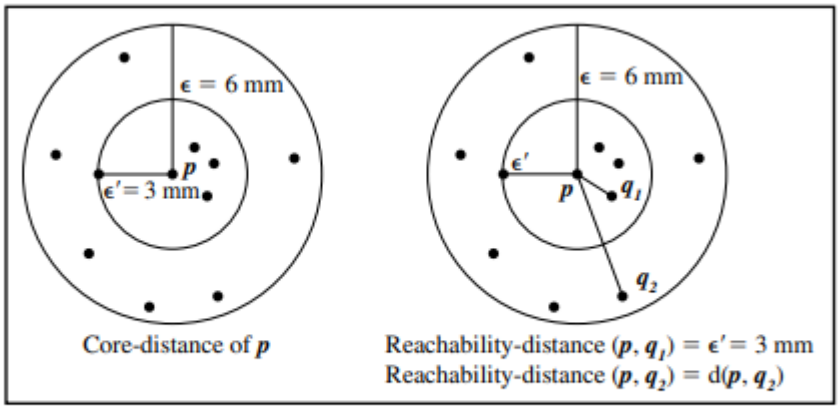


Figure 7.11 OPTICS terminology. Based on [ABKS99].

Figure 7.11 illustrates the concepts of coredistance and reachability-distance. Suppose that ε = 6 mm and MinPts = 5. The coredistance of p is the distance, ε 0 , between p and the fourth closest data object. The reachability-distance of q1 with respect to p is the core-distance of p (i.e., ε 0 = 3 mm) because this is greater than the Euclidean distance from p to q1. The reachabilitydistance of q2 with respect to p is the Euclidean distance from p to q2 because this is greater than the core-distance of p.
Steps:
1. Order Points: Calculate the core and reachability distances for each point.
2. Cluster Ordering: Create an ordered list of points based on their reachability distances.
3. Cluster Extraction: Identify clusters by analyzing the reachability plot.
Example: Suppose we are analyzing geological data to identify mineral deposits. OPTICS can order the data points based on density, revealing clusters of mineral deposits without needing to specify parameters for cluster separation.

28. Explain DENCLUE

DENCLUE (DENsity-based CLUstEring) is a clustering algorithm based on a density function, where clusters are determined by the density attractors (local maxima) of a density function.
Key Concepts:

- Density Function: Modeled using kernel density estimation.
- Density Attractors: Local maxima of the density function.
- Influence Function: Defines the influence of each data point on the density function.

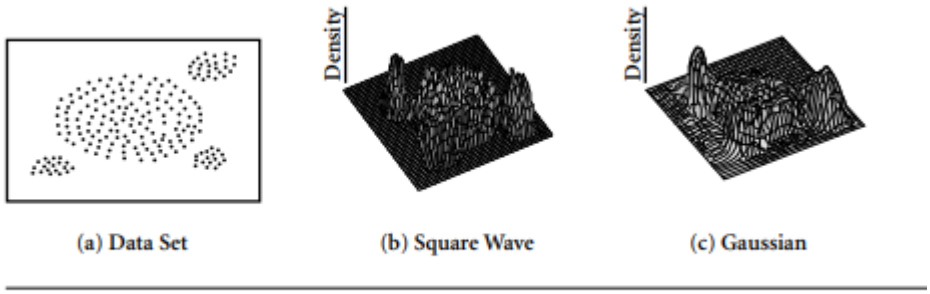


Figure 7.13 Possible density functions for a 2-D data set. From [HK98].

Figure 7.13 shows a 2-D data set together with the corresponding overall density functions for a square wave and a Gaussian influence function.

The density function at an object or point x ∈ F d is defined as the sum of influence functions of all data points. That is, it is the total influence on x of all of the data points. Given n data objects, D = {x1, ..., xn} ⊂ F d , the density function at x is defined as

$$f_B^D(\boldsymbol{x}) = \sum_{i=1}^{n} f_B^{\boldsymbol{x}_i}(\boldsymbol{x}) = f_B^{\boldsymbol{x}_1}(\boldsymbol{x}) + f_B^{\boldsymbol{x}_2}(\boldsymbol{x}) + \cdots + f_B^{\boldsymbol{x}_n}(\boldsymbol{x}).$$

For example, the density function that results from the Gaussian influence function (7.33) is

$$f_{Gauss}^D(\boldsymbol{x}) = \sum_{i=1}^{n} e^{-\frac{d(\boldsymbol{x},\boldsymbol{x}_i)^2}{2\sigma^2}}.$$

Steps:
1. Estimate Density: Calculate the density function for the data space using influence functions.
2. Identify Attractors: Find local maxima of the density function.
3. Cluster Assignment: Assign data points to the attractor they are influenced by.
Example: Suppose we are clustering gene expression data. DENCLUE can identify groups of genes with similar expression patterns based on the density of data points in the high-dimensional space, helping to understand gene regulatory mechanisms.

29. Explain STING

STING (Statistical Information Grid)
STING is a grid-based clustering algorithm used in data mining to efficiently cluster spatial data. It stands for STatistical INformation Grid and is designed to handle large datasets by partitioning the spatial area into a hierarchical grid structure.
Key Concepts:
1. Hierarchical Structure:
 - The spatial area is divided into a rectangular grid. This grid is hierarchical, meaning it consists of cells at different levels of resolution.
 - Each level in the hierarchy corresponds to a different resolution, with the bottom level having the finest granularity and the top level having the coarsest granularity.

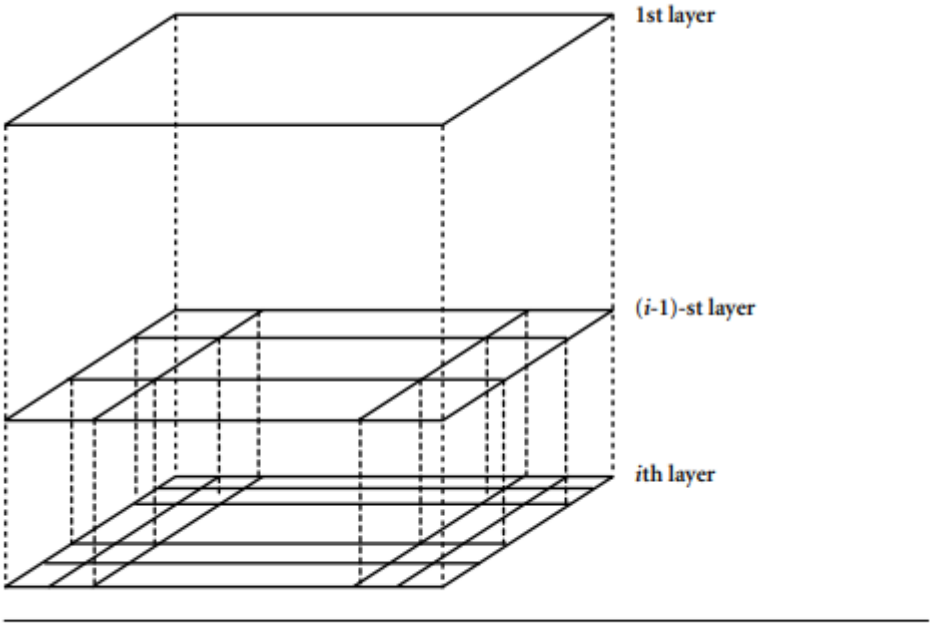


Figure 7.15 A hierarchical structure for STING clustering.

2. Statistical Information:
   - Each cell in the grid stores statistical information such as the number of objects, mean, standard deviation, and distribution of attribute values.
   - This information is precomputed and stored for each cell to speed up the clustering process.
3. Clustering Process:
   - The clustering process starts at a higher level (coarser granularity) and moves to lower levels (finer granularity).
   - At each level, cells that meet certain criteria (e.g., containing a sufficient number of objects) are selected for further processing.
   - Cells are merged or split based on statistical information to form clusters.
4. Advantages:
   - Efficiency: STING can handle large datasets efficiently due to its hierarchical structure and precomputed statistical information.
   - Scalability: It is scalable to both the number of objects and the number of dimensions.
   - Flexibility: It can handle different types of queries, such as range queries and region queries.

Example:

Suppose we have a dataset of geographical locations of restaurants in a city, and we want to cluster them based on their spatial proximity.

1. Grid Partitioning:
   - The city area is divided into a grid structure with multiple levels of resolution. For example, at the top level, the city might be divided into four large cells, and each of these cells might be further divided into smaller cells at the next level.
2. Statistical Information:
   - Each cell at each level stores statistical information about the number of restaurants, their average coordinates, and possibly other attributes like average rating.
3. Clustering:

- The algorithm starts at the top level and identifies which cells have a significant number of restaurants.
- It then moves to the next level, examining the finer cells within the significant cells, and continues this process until the desired level of granularity is reached.
- Finally, cells are merged or split based on the statistical information to form clusters of restaurants.

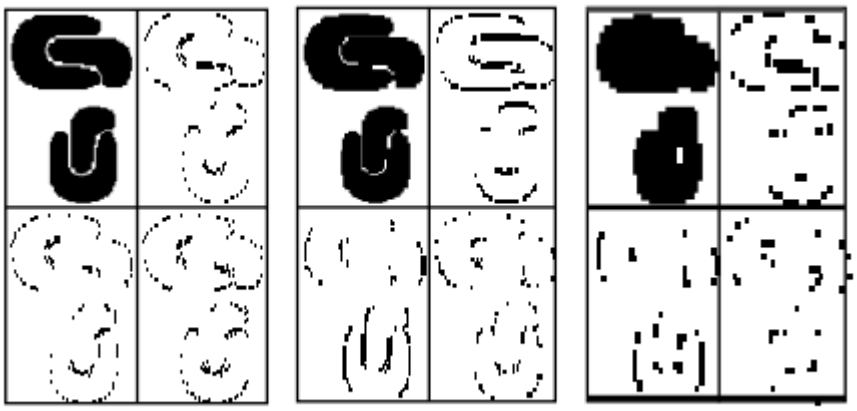
30. Explain Wave Cluster

Wave Cluster is a grid-based clustering algorithm that uses wavelet transforms to find clusters in a spatial dataset. It is particularly effective for detecting clusters of arbitrary shapes and handling noise.

Key Concepts:

1. Wavelet Transform:
   - Wavelet transforms are mathematical functions that decompose data into different frequency components and analyze each component with a resolution matched to its scale.
   - In Wave Cluster, wavelet transforms are applied to the spatial data to transform the original data space into a transformed feature space.
2. Grid-Based Approach:
   - The data space is divided into a multi-resolution grid structure.
   - Each cell in the grid represents a region in the spatial domain, and the data points falling within each cell are aggregated.

Figure 7.16 A sample of two-dimensional feature space. From [SCZ98].



For example, Figure 7.16 shows a sample of twodimensional feature space, where each point in the image represents the attribute or feature values of one object in the spatial data set. Figure 7.17 shows the resulting wavelet transformation at different resolutions, from a fine scale (scale 1) to a coarse scale (scale 3). At each level, the four subbands into which the original data are decomposed are shown. The subband shown in the upper-left quadrant emphasizes the average neighborhood around each data point. The subband in the upper-right quadrant emphasizes the horizontal edges of the data. The subband in the lower-left quadrant emphasizes the vertical edges, while the subband in the lower-right quadrant emphasizes the corners

3. Clustering Process:
   - Wavelet transforms are applied to the aggregated data in the grid. This process smooths the data and highlights regions with high density, effectively reducing noise.

- Clusters are identified by finding connected regions in the transformed feature space where the wavelet coefficients indicate high density.

4. Advantages:
   - Noise Reduction: Wave Cluster effectively reduces noise and outliers through the wavelet transformation.
   - Detection of Arbitrary Shapes: It can detect clusters of various shapes, not limited to spherical clusters.
   - Scalability: It can handle large datasets efficiently due to its grid-based approach and the efficiency of the wavelet transform.

Example:

Suppose we have a dataset of customer locations for a retail store chain, and we want to identify clusters of customers for targeted marketing.

1. Grid Partitioning:
   - The geographical area is divided into a grid structure. Each cell in the grid contains a number of customer locations.
2. Wavelet Transform:
   - A wavelet transform is applied to the data in the grid. This process transforms the data space, smoothing the data and highlighting areas with high customer density.
3. Clustering:
   - The transformed data space is analyzed to identify connected regions with high wavelet coefficients, indicating clusters of customers.
   - These clusters represent areas with a high concentration of customers, which can be targeted for marketing campaigns.
4. Result:
   - The final clusters might show that certain neighborhoods or regions have a high density of customers, allowing the retail store chain to focus its marketing efforts on these areas.

Comparison with STING:

- Both STING and Wave Cluster are grid-based algorithms, but Wave Cluster uses wavelet transforms to handle noise and detect clusters of arbitrary shapes, while STING relies on hierarchical statistical information to form clusters.